FACTORY FLOW to FORTUNE - PROCESS BOOK

Data Visualizations Final Project (Azad, Sumaiya; Ram, Dhruv; Wood, Joel)

Table of Contents

Basic Info

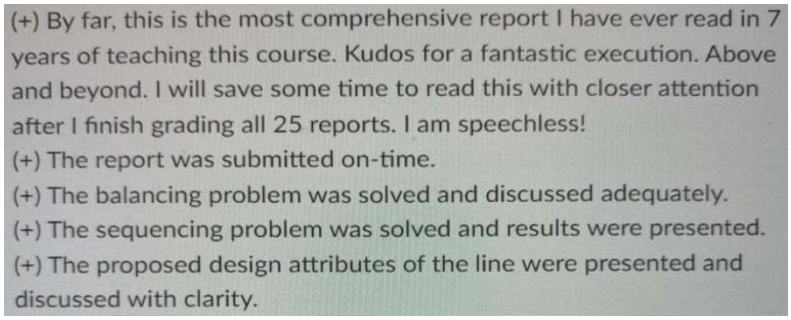
Project Title: Factory Flow to Fortune

Project Team:

- Sumaiya Azad (u1494915) – azadsumaiya00@gmail.com
- Dhruv Ram (u1589951) – dhruvaha@gmail.com
- Joel Wood (u0499977) – u0499977@umail.utah.edu

Project Repository: https://github.com/dataviscourse2025/final-project-factoryflow

Background and Motivation


(+) By far, this is the most comprehensive report I have ever read in 7 years of teaching this course. Kudos for a fantastic execution. Above and beyond. I will save some time to read this with closer attention after I finish grading all 25 reports. I am speechless!
(+) The report was submitted on-time.
(+) The balancing problem was solved and discussed adequately.
(+) The sequencing problem was solved and results were presented.
(+) The proposed design attributes of the line were presented and discussed with clarity.

This project will be based on principles of operations research and factory physics—disciplines of Industrial Engineering and Management Science. This summer, Joel took ME-EN 6182 (Design of Production and Service Systems), the final project of which involved designing an assembly line system; in this process of designing this system, Joel developed a series of algorithms to give near-optimal configurations for assembly lines—as seen in the feedback given here, these algorithms were able to accomplish the balancing, sequencing, and flow layouts of the assembly line above what was expected for this project. The professor, Pedro Huebner, encouraged additional exploration of these algorithms as they were uniquely developed to handle these practical manufacturing design problems. Reaching out to Pedro, he was enthusiastic in having his original project extended out into a visualization which explored the impacts of 'design attributes across multiple different configurations.

Assembly line configuration was identified as being particularly well-suited for this visualization project due to having a massive design space, which allows this team a wide range of possible visualizations. There are dozens of key variables which impact assembly lines, and hundreds of minor elements. The inputs and outputs of these are diverse, with many different interactions which are best visualized in different ways. This gives this team a plethora of different visualizations which can be explored both cohesively as a whole, yet also uniquely as individuals. Both inputs and outputs can be communicated statistically, cumulatively, and discretely, both over time and in a moment. With easily dozens of different visualization opportunities which cover all common types, the challenge will be in getting many visualizations to collectively come together, rather than to try and come up with different ways to do so.

Project Objectives

This project addresses a practical problem which arises frequently in a manufacturing environment: how should we design and configure our assembly line? How many people do we need to hire? How much space and what speed of conveyor motors do we need? What is our capacity, and can it match customer demands? And, essential for any business, what combination will allow us to maximize profits? Because the design of a production line is often approved by executive leadership, the ability to quantify and adequately visualize configurations is essential. In this way, development of methods to find configurations is only part of the problem, as these also need to be communicated. A factory owner will not simply want to see a single configuration, but rather several proposals which can be compared to how they meet the business needs and objectives. Therefore, the development of a tool to explain these through visualizations is critical for establishing confidence in how to allocate capital investments for building a new assembly line.

Additionally, this is a subject which few people taking this course will be familiar with, which allows us to particularly gauge how effectively it communicates these concepts. With so many different interactions, it can be difficult to wrap your head around how so many things influence each other. This means that in addition to being used to communicate business decisions, such a visualization can also be used as an educational tool. This approach will be used here, as other than Joel, the other team members are not familiar with factory physics, capacity planning, or operations research. As such, we plan to build our UI to be exploratory, in such a way that it can be used to learn these principles.

Benefits of such a system, depending on which variables we choose to visualize, would include the ability to answer the following pertinent operational questions:

- Based on expected customer demand, determine the configuration which will generate the most profit, and how many employees will be needed to sustain it?
- Based on an experienced workforce, determine the speed the conveyor should be set to maximize profit and efficiency? Alternatively, based on the wages for employees and the value of the product, how many employees are optimal. How many daily operational hours or shifts are optimal to maximize gross profit margins? If fixed expenses are known, what level of operation is necessary to ensure a net-profit is met?
- Based on set configurations, how much idle time will each workstation have throughout the day? Alternatively, this can be used to allocate employees based on individual experience/efficiency.
- Based on demand and employee count, determine the frequency of outgoing and incoming shipments of raw materials and finished goods. Additionally, determine how much WIP (Work in Progress) will be moving through the factory at a time, and the space constraints of such?
- Based on the number of employees, what order tasks should be completed to maintain precedence requirements, while minimizing idle time and unnecessary movements?
- In a mixed assembly line (where different final products are built with different combinations of core steps) what order should these configurations be built to meet a constant demand, and minimize buildup of WIP and variance between workstations?
- How much floor space will the assembly line take up as you adjust factors such as conveyor speed and spacing of product on that line; how much excess movement will occur as employees move between steps?
- How do the variables of Cycle Time, Throughput, and WIP relate to each other on the assembly line? How do these global variables impact balance delay and efficiency of workstations?

There are numerous other questions which could be informed/answered using visualizations of assembly lines, as they lie at an intersection of resource allocation, financial and operational metrics, and general flow mechanics/physics/statistics within the system. We would like to explore these many interactions, and how to effectively communicate and enable the exploration of these factors to make an educational and informative interface, which enables and encourages its users to take technical design inputs to make meaningful management decisions commonly encountered in manufacturing.

Data Sources

For this project, we will be using the base problem provided by Pedro Huebner, with his expressed permission. This project was based on building an assembly line which would manufacture 3 different models of refrigerators in the following balance: 35% Super, 45% Ultra, and 20% Mega. The assembly process has been broken into a set of discrete elements which can be completed by a single person, so it is a matter of assigning them as needed. Below are the element times used and their predecessor tasks, along with their need based on which model is built. We will have it so total demand can increase or decrease, but the overall composition balance mentioned above will remain the same.

Element	Element Description	Element Time (min)	Predecessor	Required?		
				Super	Ultra	Mega
1	Place frame on holder and secure	0.8	-	1	1	1
2	Install internal heat exchanging pipes (IHEPs)	1.5	1	1	1	1
3	Install compressor motor/pump kit	1.0	1	1	1	1
4	Install compressor electrical kit	1.3	3	1	1	1
5	Connect IHEPs to compressor	0.5	2, 3	1	1	1
6	Install defrost heater kit	0.7	1	1	1	1
7	Install defrost drain tube	0.7	6	1	1	1
8	Install water filter kit	0.8	1	0	1	1
9	Connect and route water pipes	0.5	8	0	1	1
10	Install basic electronic control board	1.2	1	1	1	0
11	Install advanced electronic control board	1.5	1	0	0	1
12	Connect and route electrical wires	2.0	10, 11	1	1	1
13	Install internal panels	2.5	4, 5, 7, 9, 12	1	1	1
14	Install evaporator and evaporator fans	1.5	13	1	1	1
15	Install external panels	2.2	14	1	1	1
16	Install external heat exchanging pipes (EHEPs)	1.2	15	1	1	1
17	Connect EHEPs to compressor	0.5	16	1	1	1
18	Install door brackets	0.8	14	1	1	1
19	Install freezer compartment door	1.1	18	1	1	1
20	Install icemaker kit into freezer door	0.5	19	0	1	1
21	Connect pre-routed water pipes to icemaker	0.5	20	0	1	1
22	Install refrigerator compartment door	1.3	18	1	1	1
23	Install touchscreen (TS) into refrigerator door	0.7	22	0	0	1
24	Connect pre-routed electrical wires to TS	0.9	23	0	1	1
25	Install sensors and switches	1.3	19, 22	1	1	1
26	Install lighting kit	1.0	19, 22	1	1	1
27	Connect remaining pre-routed electrical wires	0.7	25, 26	1	1	1
28	Assemble internal shelves and drawers	1.0	27	1	1	1
29	Install feet and wheels	0.7	15	1	1	1
30	Visual and structural inspection	0.5	17, 21, 24, 27, 29	1	1	1
31	Remove from holder and package	0.4	30	1	1	1

Data Processing

While it may be possible to rewrite the algorithms for the original project (used for balancing, sequencing, and optimizing flow) to function within JavaScript, this will greatly expand the complexity of the backend. Therefore, to keep the focus of this project on developing front-end visualizations, we will not have these calculations be performed dynamically within our project. It has already been determined that more than 12 employees will not represent any form of optimization across any possible metric, due to discrete element 13 making up 8.7% of the total task demand time. We will therefore pre-emptively calculate near-optimal configurations for between 3 to 12 employees, with our established model build ratios. These will generate unique sequences for the 31 elements, which will be used in conjunction with their ratios of total daily time, as shown in the below 7-workstation configuration diagram.

Element	Task Time (m)	Daily Time (m)	Station Length (ft)	Station Efficiency (%)	Counter Flow Walking Time (m)	Adjusted Daily Time (m)
WorkStation 1	6.1	860	91.5	95.6%	7.44	868
1	0.8	160	12			
2	1.5	300	22.5			
3	1	200	15			
6	1.3	140	19.5			
11	1.5	60	22.5			
WorkStation 2	5	861	75	95.7%	20.24	882
10	1.2	192	18			
5	0.5	100	7.5			
8	0.8	104	12			
9	0.5	65	7.5			
12	2	400	30			
WorkStation 3	4.5	900	67.5	100.0%	4.80	905
7	0.7	140	10.5			
4	1.3	260	19.5			
13	2.5	500	37.5			
WorkStation 4	3.7	740	55.5	82.2%	5.36	746
14	1.5	300	22.5			
15	2.2	440	33			
WorkStation 5	4	800	60	88.9%	9.60	810
18	0.8	160	12			
22	1.3	260	19.5			
16	1.2	240	18			
29	0.7	140	10.5			
WorkStation 6	4.6	773	69	85.9%	12.57	786
19	1.1	220	16.5			
26	1	200	15			
20	0.5	65	7.5			
23	0.7	28	10.5			
25	1.3	260	19.5			
WorkStation 7	4.5	802	67.5	89.1%	9.60	812
21	0.5	65	7.5			
24	0.9	117	13.5			
27	0.7	140	10.5			
28	1	200	15			
17	0.5	100	7.5			
30	0.5	100	7.5			
31	0.4	80	6			
TOTAL	32,4 min	5736 min	486 ft		71 min	5809 min
Floor Space	Throughput	Cycle Time	Conveyor Speed	Product Spacing	Balance Delay	Efficiency
57' X 90'	13.33 pph	4.5 min	15 ft/min	67.5 ft	8.95%	91.05%

These pre—established elements will then be used to dynamically calculate our needed variables for our visualizations; these pre-calculations are outlined below, within “Configurations”.

Configurations

3 Workers

WorkStation 1: Task 1
WorkStation 1: Task 2
WorkStation 1: Task 3
WorkStation 1: Task 10
WorkStation 1: Task 11
WorkStation 1: Task 12
WorkStation 1: Task 4
WorkStation 1: Task 5
WorkStation 1: Task 6
WorkStation 1: Task 8

WorkStation 2: Task 9
WorkStation 2: Task 7
WorkStation 2: Task 13
WorkStation 2: Task 14
WorkStation 2: Task 18
WorkStation 2: Task 19
WorkStation 2: Task 22
WorkStation 2: Task 25

WorkStation 3: Task 20
WorkStation 3: Task 21
WorkStation 3: Task 15
WorkStation 3: Task 16
WorkStation 3: Task 26
WorkStation 3: Task 23
WorkStation 3: Task 24
WorkStation 3: Task 27
WorkStation 3: Task 28
WorkStation 3: Task 29
WorkStation 3: Task 17
WorkStation 3: Task 30
WorkStation 3: Task 31

4 Workers

WorkStation 1: Task 1
WorkStation 1: Task 2
WorkStation 1: Task 3
WorkStation 1: Task 10
WorkStation 1: Task 11
WorkStation 1: Task 12
WorkStation 1: Task 6

WorkStation 2: Task 8
WorkStation 2: Task 9
WorkStation 2: Task 4
WorkStation 2: Task 5
WorkStation 2: Task 7
WorkStation 2: Task 13
WorkStation 2: Task 14

WorkStation 3: Task 15
WorkStation 3: Task 18
WorkStation 3: Task 19
WorkStation 3: Task 22
WorkStation 3: Task 25
WorkStation 3: Task 20

WorkStation 4: Task 21
WorkStation 4: Task 16
WorkStation 4: Task 26
WorkStation 4: Task 27
WorkStation 4: Task 23
WorkStation 4: Task 24
WorkStation 4: Task 17
WorkStation 4: Task 28
WorkStation 4: Task 29
WorkStation 4: Task 30
WorkStation 4: Task 31

5 Workers

WorkStation 1: Task 1
WorkStation 1: Task 3
WorkStation 1: Task 10
WorkStation 1: Task 2
WorkStation 1: Task 5
WorkStation 1: Task 8

WorkStation 2: Task 9
WorkStation 2: Task 4
WorkStation 2: Task 6
WorkStation 2: Task 11
WorkStation 2: Task 7
WorkStation 2: Task 12

WorkStation 3: Task 13
WorkStation 3: Task 14
WorkStation 3: Task 18
WorkStation 3: Task 19

WorkStation 4: Task 15
WorkStation 4: Task 16
WorkStation 4: Task 22
WorkStation 4: Task 26
WorkStation 4: Task 20

WorkStation 5: Task 23
WorkStation 5: Task 21
WorkStation 5: Task 24
WorkStation 5: Task 17
WorkStation 5: Task 25
WorkStation 5: Task 27
WorkStation 5: Task 28
WorkStation 5: Task 29
WorkStation 5: Task 30
WorkStation 5: Task 31

6 Workers

WorkStation 1: Task 1
WorkStation 1: Task 2
WorkStation 1: Task 3
WorkStation 1: Task 5
WorkStation 1: Task 6
WorkStation 1: Task 11

WorkStation 2: Task 4
WorkStation 2: Task 7
WorkStation 2: Task 10
WorkStation 2: Task 12

WorkStation 3: Task 8
WorkStation 3: Task 9
WorkStation 3: Task 13
WorkStation 3: Task 14

WorkStation 4: Task 18
WorkStation 4: Task 19
WorkStation 4: Task 22
WorkStation 4: Task 26
WorkStation 4: Task 20
WorkStation 4: Task 23

WorkStation 5: Task 15
WorkStation 5: Task 16
WorkStation 5: Task 25

WorkStation 6: Task 21
WorkStation 6: Task 24
WorkStation 6: Task 17
WorkStation 6: Task 27
WorkStation 6: Task 28
WorkStation 6: Task 29
WorkStation 6: Task 30
WorkStation 6: Task 31

7 Workers	8 Workers	9 Workers	10 Workers
Workstation 1: Task 1	WorkStation 1: Task 1	WorkStation 1: Task 1	WorkStation 1: Task 1
Workstation 1: Task 2	WorkStation 1: Task 2	WorkStation 1: Task 2	WorkStation 1: Task 2
Workstation 1: Task 3	WorkStation 1: Task 10	WorkStation 1: Task 10	WorkStation 1: Task 10
Workstation 1: Task 6			
Workstation 1: Task 11	WorkStation 2: Task 11	WorkStation 2: Task 11	WorkStation 2: Task 11
	WorkStation 2: Task 12	WorkStation 2: Task 3	WorkStation 2: Task 3
Workstation 2: Task 10	WorkStation 2: Task 3	WorkStation 2: Task 4	WorkStation 2: Task 4
Workstation 2: Task 12	WorkStation 2: Task 6	WorkStation 2: Task 6	WorkStation 2: Task 6
Workstation 2: Task 5			
Workstation 2: Task 8	WorkStation 3: Task 8	WorkStation 3: Task 8	WorkStation 3: Task 8
Workstation 2: Task 9	WorkStation 3: Task 4	WorkStation 3: Task 7	WorkStation 3: Task 7
	WorkStation 3: Task 7	WorkStation 3: Task 12	WorkStation 3: Task 12
Workstation 3: Task 4	WorkStation 3: Task 5		
Workstation 3: Task 7	WorkStation 3: Task 9	WorkStation 4: Task 9	WorkStation 4: Task 9
Workstation 3: Task 13		WorkStation 4: Task 5	WorkStation 4: Task 5
	WorkStation 4: Task 13	WorkStation 4: Task 13	WorkStation 4: Task 13
Workstation 4: Task 14	WorkStation 4: Task 14		
Workstation 4: Task 15		WorkStation 5: Task 14	WorkStation 5: Task 14
	WorkStation 5: Task 18	WorkStation 5: Task 18	WorkStation 5: Task 18
Workstation 5: Task 16	WorkStation 5: Task 22		
Workstation 5: Task 18	WorkStation 5: Task 19	WorkStation 6: Task 15	WorkStation 6: Task 15
Workstation 5: Task 22		WorkStation 6: Task 19	WorkStation 6: Task 29
Workstation 5: Task 29	WorkStation 6: Task 15		
	WorkStation 6: Task 25	WorkStation 7: Task 22	WorkStation 7: Task 22
Workstation 6: Task 19		WorkStation 7: Task 25	WorkStation 7: Task 23
Workstation 6: Task 25	WorkStation 7: Task 26	WorkStation 7: Task 29	WorkStation 7: Task 19
Workstation 6: Task 26	WorkStation 7: Task 16		
Workstation 6: Task 20	WorkStation 7: Task 27	WorkStation 8: Task 20	WorkStation 8: Task 25
Workstation 6: Task 23	WorkStation 7: Task 29	WorkStation 8: Task 16	WorkStation 8: Task 20
		WorkStation 8: Task 26	WorkStation 8: Task 26
Workstation 7: Task 26	WorkStation 8: Task 20	WorkStation 8: Task 23	
Workstation 7: Task 27	WorkStation 8: Task 21	WorkStation 8: Task 24	WorkStation 9: Task 16
Workstation 7: Task 21	WorkStation 8: Task 28		WorkStation 9: Task 21
Workstation 7: Task 24	WorkStation 8: Task 23	WorkStation 9: Task 21	WorkStation 9: Task 24
Workstation 7: Task 17	WorkStation 8: Task 24	WorkStation 9: Task 17	WorkStation 9: Task 17
Workstation 7: Task 30	WorkStation 8: Task 17	WorkStation 9: Task 27	
Workstation 7: Task 31	WorkStation 8: Task 30	WorkStation 9: Task 28	WorkStation 10: Task 27
	WorkStation 8: Task 31	WorkStation 9: Task 30	WorkStation 10: Task 28
		WorkStation 9: Task 31	WorkStation 10: Task 30
			WorkStation 10: Task 31

11 Workers

WorkStation 1: Task 1
WorkStation 1: Task 3
WorkStation 1: Task 8
WorkStation 1: Task 9

WorkStation 2: Task 2
WorkStation 2: Task 10
WorkStation 2: Task 11

WorkStation 3: Task 5
WorkStation 3: Task 12

WorkStation 4: Task 4
WorkStation 4: Task 6
WorkStation 4: Task 7

WorkStation 5: Task 13

WorkStation 6: Task 14
WorkStation 6: Task 18

WorkStation 7: Task 22
WorkStation 7: Task 23
WorkStation 7: Task 19

WorkStation 8: Task 15
WorkStation 8: Task 20

WorkStation 9: Task 16
WorkStation 9: Task 17
WorkStation 9: Task 26

WorkStation 10: Task 25
WorkStation 10: Task 27
WorkStation 10: Task 29

WorkStation 11: Task 21
WorkStation 11: Task 24
WorkStation 11: Task 28
WorkStation 11: Task 30
WorkStation 11: Task 31

12 Workers

WorkStation 1: Task 1
WorkStation 1: Task 3
WorkStation 1: Task 8
WorkStation 1: Task 9

WorkStation 2: Task 10
WorkStation 2: Task 2
WorkStation 2: Task 11

WorkStation 3: Task 12
WorkStation 3: Task 5

WorkStation 4: Task 4
WorkStation 4: Task 6
WorkStation 4: Task 7

WorkStation 5: Task 13

WorkStation 6: Task 14

WorkStation 7: Task 18
WorkStation 7: Task 22
WorkStation 7: Task 23

WorkStation 8: Task 15

WorkStation 9: Task 19
WorkStation 9: Task 25

WorkStation 10: Task 26
WorkStation 10: Task 27
WorkStation 10: Task 29

WorkStation 11: Task 20
WorkStation 11: Task 16
WorkStation 11: Task 17
WorkStation 11: Task 21

WorkStation 12: Task 28
WorkStation 12: Task 24
WorkStation 12: Task 30
WorkStation 12: Task 31

Element Percentage of Total

(Labor Time/Conveyor Length)

Element 1 - 2.79% / 2.52%
Element 2 - 5.23% / 4.72%
Element 3 - 3.49% / 3.15%
Element 4 - 4.53% / 4.09%
Element 5 - 1.74% / 1.57%
Element 6 - 2.44% / 2.20%
Element 7 - 2.44% / 2.20%
Element 8 - 1.81% / 2.52%
Element 9 - 1.13% / 1.57%
Element 10 - 3.35% / 3.77%
Element 11 - 1.05% / 4.72%
Element 12 - 6.97% / 6.29%
Element 13 - 8.72% / 7.86%
Element 14 - 5.23% / 4.72%
Element 15 - 7.67% / 6.92%
Element 16 - 4.19% / 3.77%
Element 17 - 1.74% / 1.57%
Element 18 - 2.79% / 2.52%
Element 19 - 3.84% / 3.46%
Element 20 - 1.13% / 1.57%
Element 21 - 1.13% / 1.57%
Element 22 - 4.53% / 4.09%
Element 23 - 0.49% / 2.20%
Element 24 - 2.04% / 2.83%
Element 25 - 4.53% / 4.09%
Element 26 - 3.49% / 3.14%
Element 27 - 2.44% / 2.20%
Element 28 - 3.49% / 3.14%
Element 29 - 2.44% / 2.20%
Element 30 - 1.74% / 1.57%
Element 31 - 1.40% / 1.26%

Build Sequence

1 (Super) 2 (Ultra) 3 (Mega)

2 1 3 2 1 2 1 3 2 1 2 2 3 1
2 1 2 3 1 2 2 1 3 2 1 2 1 3
2 2 1 2 3 1 2 1 2 3 1 2 2 1
3 2 1 2 1 3 2 1 2 2 3 1 2 1
2 3 1 2 2 1 3 2 1 2 1 3 2 1
2 2 3 1 2 1 2 3 1 2 2 1 3 2
1 2 1 3 2 1 2 2 3 1 2 1 2 3
1 2 2 1 3 2 1 2 1 3 2 1 2 2
3 1 2 1 2 3 1 2 2 1 3 2 1 2
1 3 2 1 2 2 3 1 2 1 2 3 1 2
2 1 3 2 1 2 1 3 2 1 2 2 3 1
2 1 2 3 1 2 2 1 3 2 1 2 1 3
2 1 2 2 3 1 2 1 2 3 1 2 2 1
3 2 1 2 1 3 2 1 2 2 3 1 2 1
2 3 1 2

Bayesian Flow Weights

Super → Super: 0%
Super → Ultra: 71.43%
Super → Mega: 28.57%

Ultra → Super: 56.18%
Ultra → Ultra: 21.35%
Ultra→ Mega: 22.47%

Mega → Super: 50%
Mega → Ultra: 0%
Mega → Mega: 50%

Variable Glossary

Mixed Assembly Line – a form of manufacturing where building a product is subdivided into different tasks to collectively form the finished product. Products are moved steps by a conveyor system which moves at a fixed speed. Mixed Lines will build several different variations of a product which share common steps. Some steps may or may not be needed for a specific model.

Workstation – series of tasks (elements) intended to be performed by an individual, sequentially.

Elements – These are atomic tasks which are done by a single person at a time; they cannot be shared between people or workstations; however, they may be performed sequentially.

Precedence – When building a product, certain steps may need to be completed before other steps can begin, these steps which must be completed are known as predecessor elements.

Element Time – how long tasks take to complete once, from beginning to end. This with conveyor properties (spacing and speed), directly influences the physical length of the conveyor needed for the task.

Task/Labor Time – Represents the amount of daily time an individual is expected to perform the task within a given timeframe. Within a mixed assembly line, this time will not be the same as the element time, as a process will always take the same amount of space/time, but what it takes up within a day will be based on the frequency they are built.

WIP – Work in Progress, the number of incomplete products currently being worked on.

Flow – the visual depiction of WIP moving through the assembly line, with a steady movement rate.

Demand – the number of units which can be sold, or which producing more than needed is a liability. Production should match your demand.

Throughput – the number of models built within a given time-period, usually an hour.

Workstation Time – the sum of all Task times in a workstation, representing how much time an operator will spend building one model through their station. Directly impacts conveyor lengths.

Cycle Time – the average amount of time which passes between each completed model to come off the assembly line.

Process Time – the total time of all element times, representing the amount of time which passes for a model to move through the whole line.

Efficiency – percentage of time in which the assembly line/workstation are working on a model.

Balance Delay – the percentage of time in which the individual(s) in a workstation/line are idle / not working.

BD CV – The variance of balance delay between workstations normalized by their average balance delay. Used to evaluate how evenly distributed tasks are between workstations/individuals.

COGS – Cost of Goods Sold, cost of materials & labor to produce an item. Generally, it represents variable costs which fluctuate by how many goods are sold.

Gross Profit – The value of goods sold minus COGS. Representing the value a given product provides for the company, to sustain operations.

Net Profit – Gross Profit minus Operating Expenses (generally fixed costs, which are incurred regardless of production), representing final profit of the company.

Marginal Profit – the per unit value that a product line provides in gross profit.

Profit Margin – can be either net or gross, this is the percentage of profit compared to total goods sold.

Set Variables

(These are preset and cannot be changed without building up an extensive back end)

Build Composition (the percentage of each type of model built of total demand), Conveyor Distance (the distance between each item on the conveyor belt), Precedence, Element Time.

Adjustable Variables

(These have preset values which can be adjusted to change outputs)

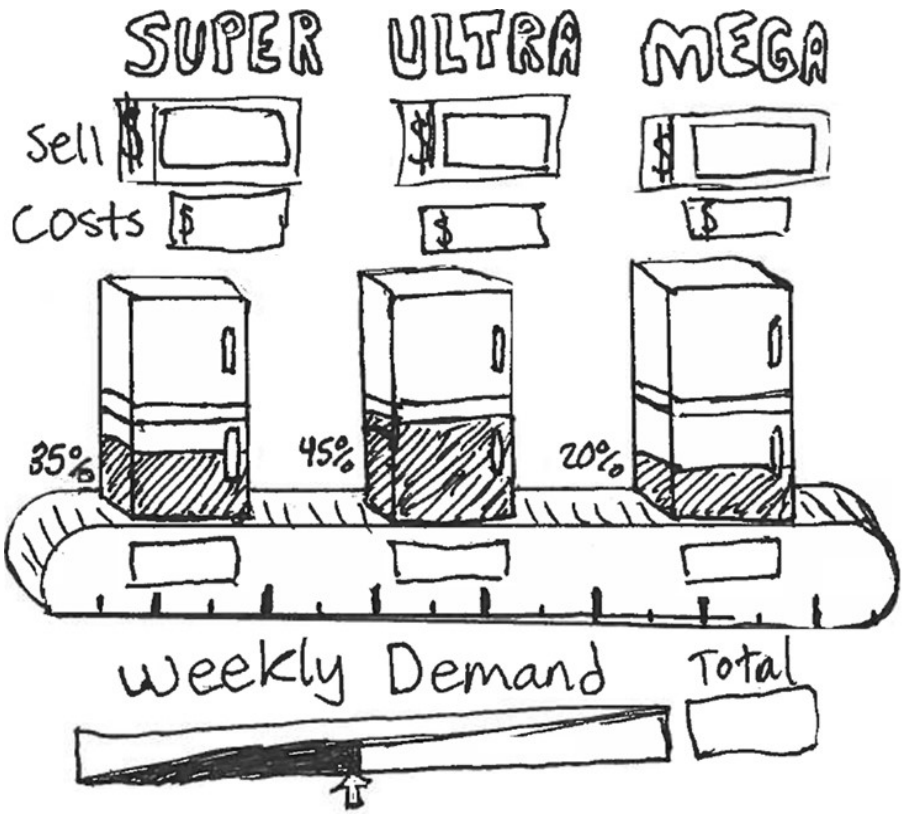
Demand, Labor Costs (per hour), Operational Hours (How many hours a day the assembly line is running), Sell Price and Cost of Goods (per model), Workstation Compositions (which elements are in workstations), Number of Employees,

Output Variables

(These are shown as output characteristics of the line, but aren't directly adjustable)

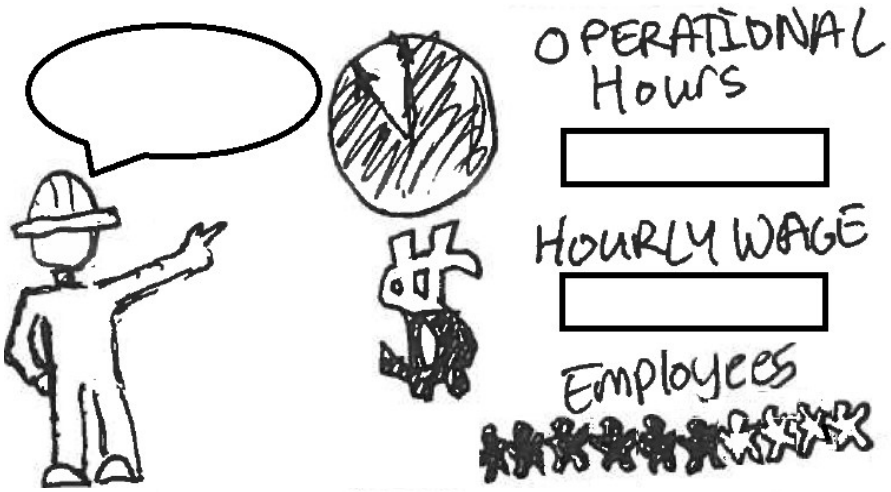
Conveyor Speed, Task/Labor Times, WIP, Throughput, Cycle Time, Process Time, Efficiency, Balance Delay, BD CV, All Financial Values (COGS, Profits, Marginal and Margin)

Visualization Design



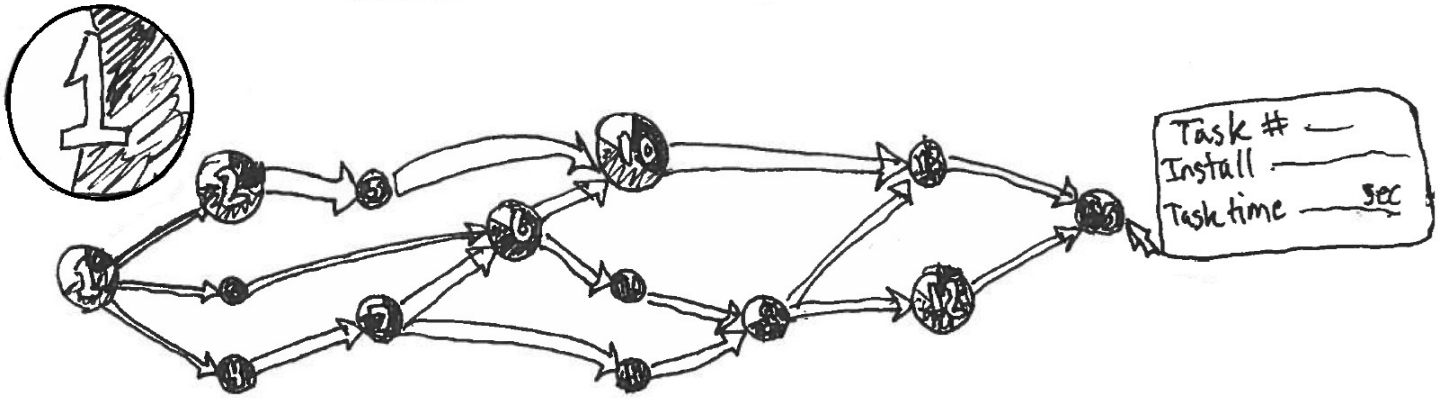
ADJUSTABLE INPUTS #1

A conveyor belt image is used to show the 3 different models of refrigerators. Each of the three models is shaded in a percentage corresponding to its build ratio percentage of the total demand. These total numbers are shown in boxes below each fridge, as an output field. Below there is a slider and a text field which can both set the total weekly demand. Above is the title of the three fridges, with their total Sell cost for each model, and their material costs below. These will be prefilled, but adjustable through text. Finally, a ruler will be shown as the base of the conveyor, showing the distance that the Fridges are, and the distance between them.



Factory Worker inputs are shown in the form of a clock which fills up based on the number of hours worked in a day, and a dollar sign which behaves similarly. These are input in fields to the right. A meeple fill-in shows the number of recommended / changed employees. A worker on the side has a textbox to give feedback on input selections. Maybe they show their satisfaction with their expression?

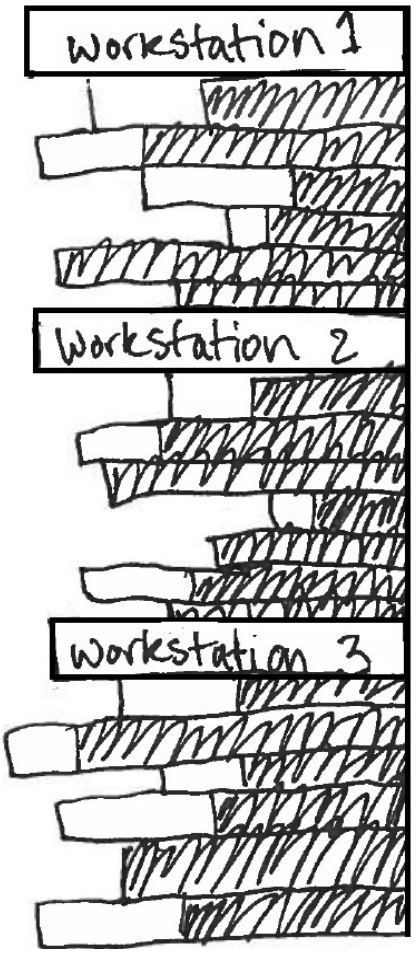
SET VARIABLES #2



Here, the elements of assembly are shown in a precedence DAG where the nodes represent the individual elements of assembly. These are represented with their element number overlaid over a pie-chart which shows

what percentage of their time is spent on the 3 different models (preferably with the same color as the fridges in the above diagram, where each model uses a different color). The size of the node will vary based on the amount of time each day an operator will need to be working on that element. Hovering over an element will display a tooltip which lists the element number, a description of the task, and how long that task is expected to take.

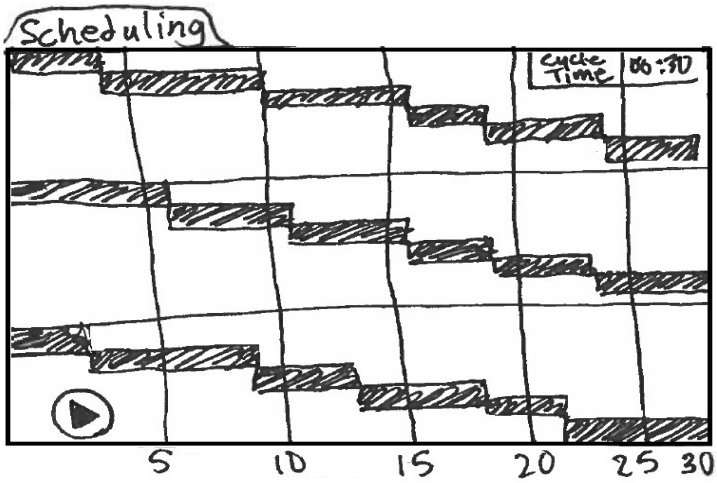
The edges of the DAG convey successor tasks, with the thickness of the arrows representing the element time between the tasks. This will be directly impactful on the length of the conveyor, since conveyor speed will be constant throughout the whole process. This section is purely informative, and is not intended to change at all, so much as to convey critical information about what steps are needed to build the mix of different refrigerators.



OUTPUTS #3

To the left of the various output panes, there is a ribbon which shows the sequence of the different elements, as they are allocated to workstations/ employees. These are useful to actively show, as the sequencing and allocation is fundamental, along with demand, in influencing almost all outputs. If possible, allowing for a drag and drop of elements between Workstations could allow more dynamic exploration. Beneath each Workstation tab, the elements are listed in sequence as they would be performed in the workstation. The length of the total bars represents element times, and in turn how long their conveyor segments would be. Within these, the bars are shaded with what percentage of the time they are being built. The balance of tasks will be based on getting all workstations to have a similar total length of shaded bars, while meeting precedence requirements.

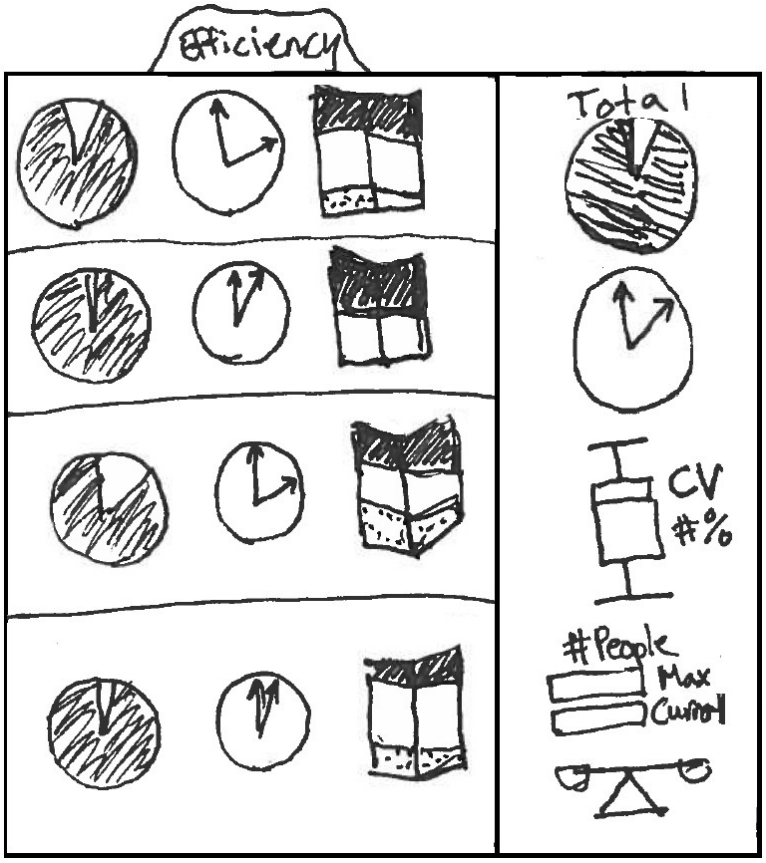
For convenience, it would be good to have a retractable right ribbon for the various ADJUSTABLE INPUTS, so that they can be modified in real-time, without needing to scroll up and down. For the main output area, a series of "folder tabs" can be used to quickly switch between different views which will show various behaviors of both workstations and the whole assembly line in real time. These tabs will be viewed individually as a subset of useful data which collectively tell a story.



SCHEDULING

For this, we can use a Gantt Chart in one of two ways. One is as a simulation in which a "Play" button is pressed, and the various tasks as their full (unshaded) bars will flow across sequentially in building a given model—which follows the predetermined optimal sequence. This would show which areas different employees would be working at a given time. Alternatively, the shaded bars which represent total daily time could also be arranged as a Gantt chart, showing the overall sequencing of tasks. This would not be a true representation of task times and would need to be shown as a portion of the day. This can simply be used as a visual aid of how different tasks contribute to overall labor in workstations.

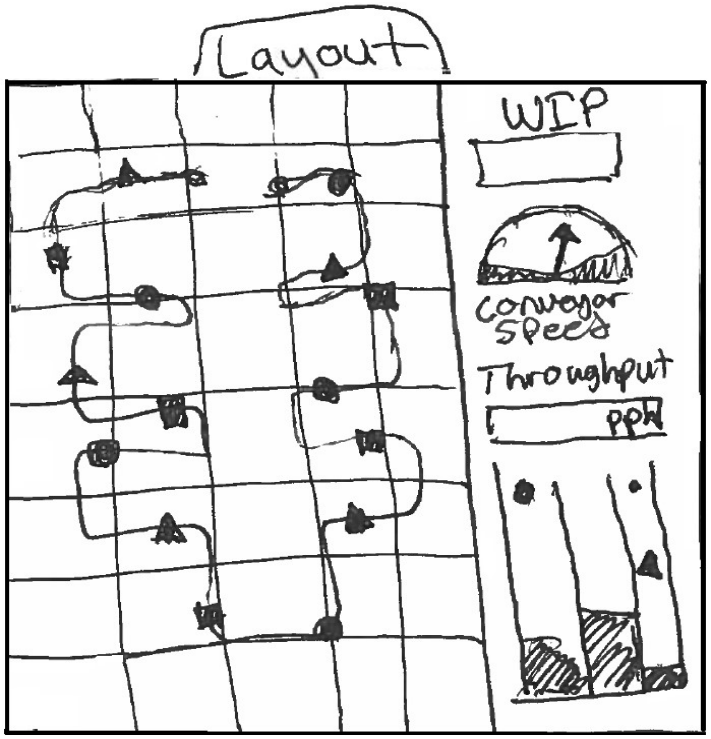
EFFICIENCY



This one can focus on the behavior of individual workstations, with a series of area diagrams. The first, a pie-chart, will show a percentage of efficiency, filled with what percentage of the time an employee is working. Next to it will be a clock which shows how much time is spent idle at a given station throughout the operational day. This is an absolute value, compared to the relative value given by efficiency. Finally, a 3D stacked bar chart is used to connect the concept for how long individual models take to build at the station (the sum of their relevant tasks) and how those extend to a total amount of time spent each day on those tasks/model. A tooltip can hover over each of these graphical representations to show the details, for the clock, it could show the per annum cost of the idle time in employee wages.

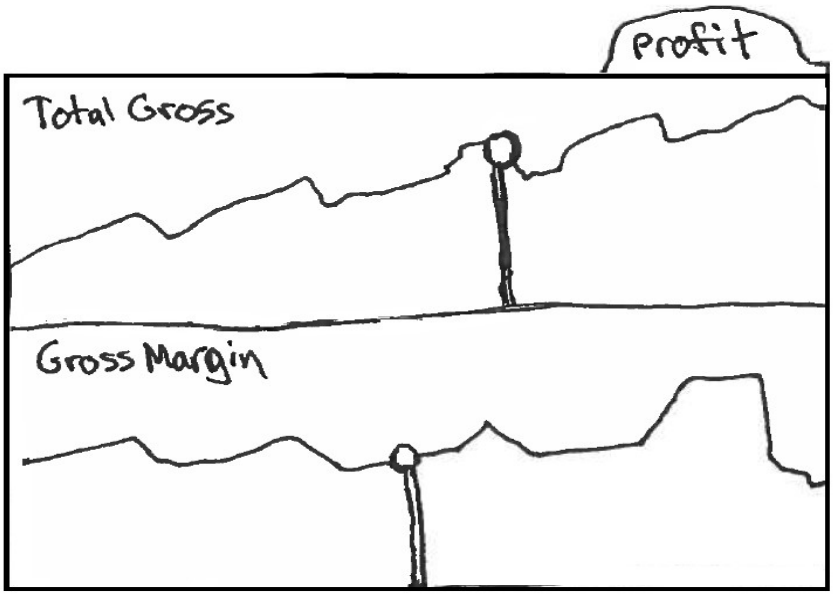
To the far side of the tab, a segment should show the full assembly line. A larger version of the pie chart and clock, showing total impact of non-applied time could be shown. Additionally, a Box and Whisker Plot could be used to show the standard deviation of the workstations, with a scale to the side showing a coefficient of variance. Below this, there is a text box showing what the current number of workstations/ employees are, and what the current and maximum output those employees can perform. This can either be based on demand or throughput. Below this, a visual scale can show the balance of the line between being overloaded or being too idle. This can give a visual for how optimal the current configuration is for the given workstations.

LAYOUT



A flow chart overlaid over a floor grid, this could show the assembly line as both different colored workstations, and varied saturation elements. On the line, the different models could be seen flowing along the line, with a different color/shape for each of the models. The shapes could shift between being outlined and filled based on when the element they are passing through is relevant for them. As the models reach the end of the line, they could “fall” or bounce into different buckets of finished goods, each one for a different model. Showing the accumulation of demand over time. Finally, a Counter box could show WIP (the units moving through at a given time), with a speedometer below showing the conveyor speed. Below with the finished goods, there can be a display for the throughput, that is, the number of finished goods per hour.

PROFIT



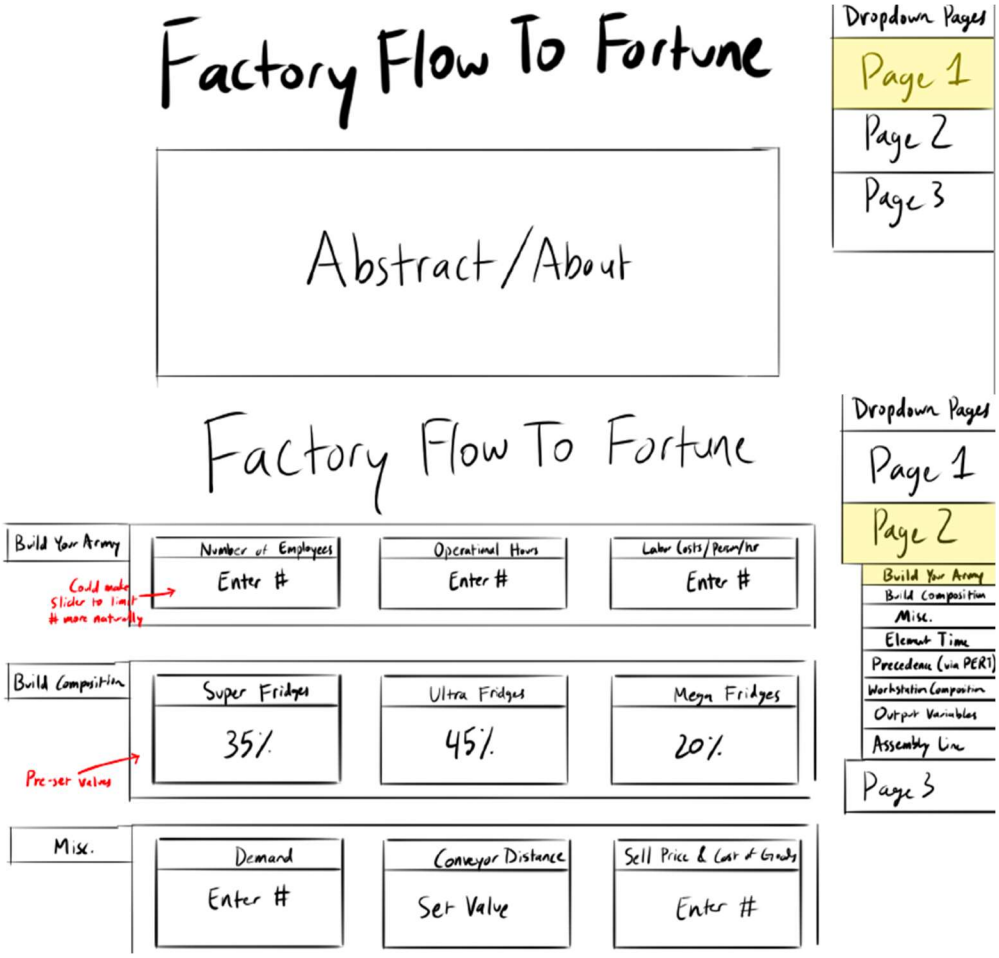
Two main metrics should be considered for a selection of gross profit, the total and the margin. While marginal profit (the value generated by an additional item) may be of interest, it is likely to get obfuscated alongside profit margin for anyone who is not accustomed to financial analysis. The use of marginal profit is useful in a microeconomic analysis of what value an expansion in demand is in the market. This may be something we can find a good fit in showing, perhaps if we do include some market elements. But it is not a good fit in the current layout, and unless market analysis becomes part of our scope, this is unlikely to change. It is absolutely a valuable metric for determining when production should be increased to a higher demand. But the scope of this is to function as a factory flow simulator, and not a full business decision making simulator. The effect of understanding when demand should shift can be understood by evaluating the total and margin. In the above chart, to show profit, we can have a line/area chart which shows how profit behaves as demand is increased based on our calculated near-optimal configurations. There will be a vertical line which shows what demand is currently set at, with a point which can move up and down to indicate the current profit generated in the configuration. A Tooltip can be used over the line to show optimal configurations to get that profit and hovering over the point will show the current configuration. This will help conceptualize the financial impact that these configuration flows have.

LAYOUTS/ALTERNATIVES

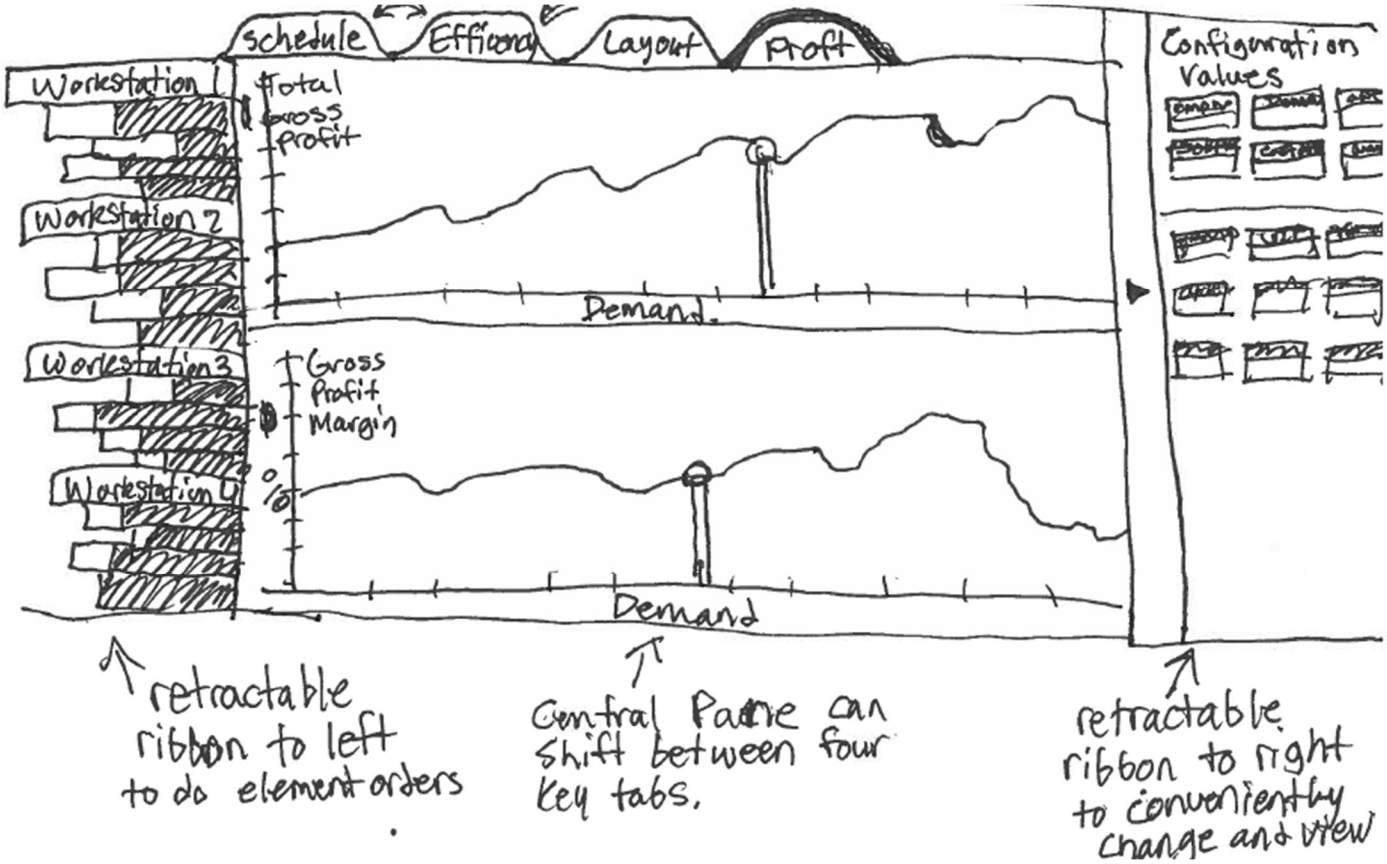
It was also suggested that having a set of different pages may be useful, such as a landing page which would introduce the problem space and introduce the team, as well as a dropdown to navigate between the pages.

This could perhaps become a page for initial variables and the PERT chart showed above.

Perhaps showing too many visualizations for the inputs will just overly complicate the interface and deter users while unnecessarily expanding scope. It was suggested that having a panel which allowed variables to be set and viewed just as numbers may be easier for some users to interface with than a wide range of different visualized



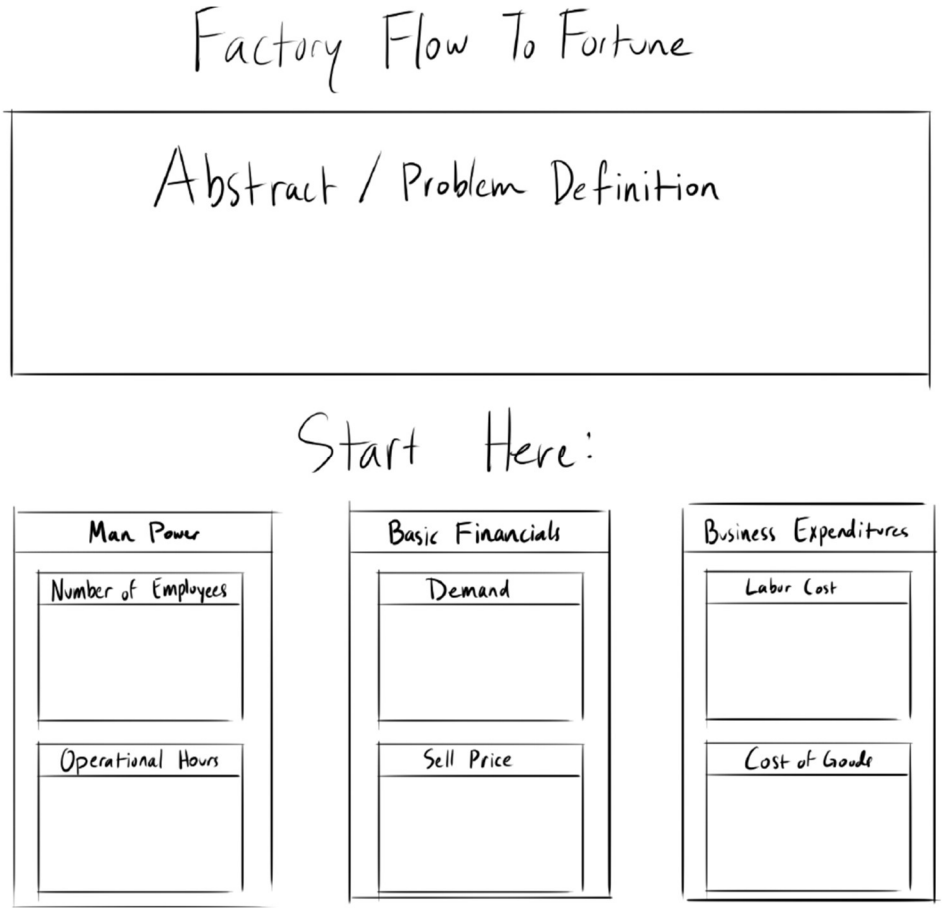
inputs. Initially this was proposed as a subpage, but this may involve too many screen changes for people looking to see multiple views dynamically. It was therefore proposed to make a right ribbon which can be expanded or collapsed with this view, as seen below.



FINAL PROPOSED TEMPLATING

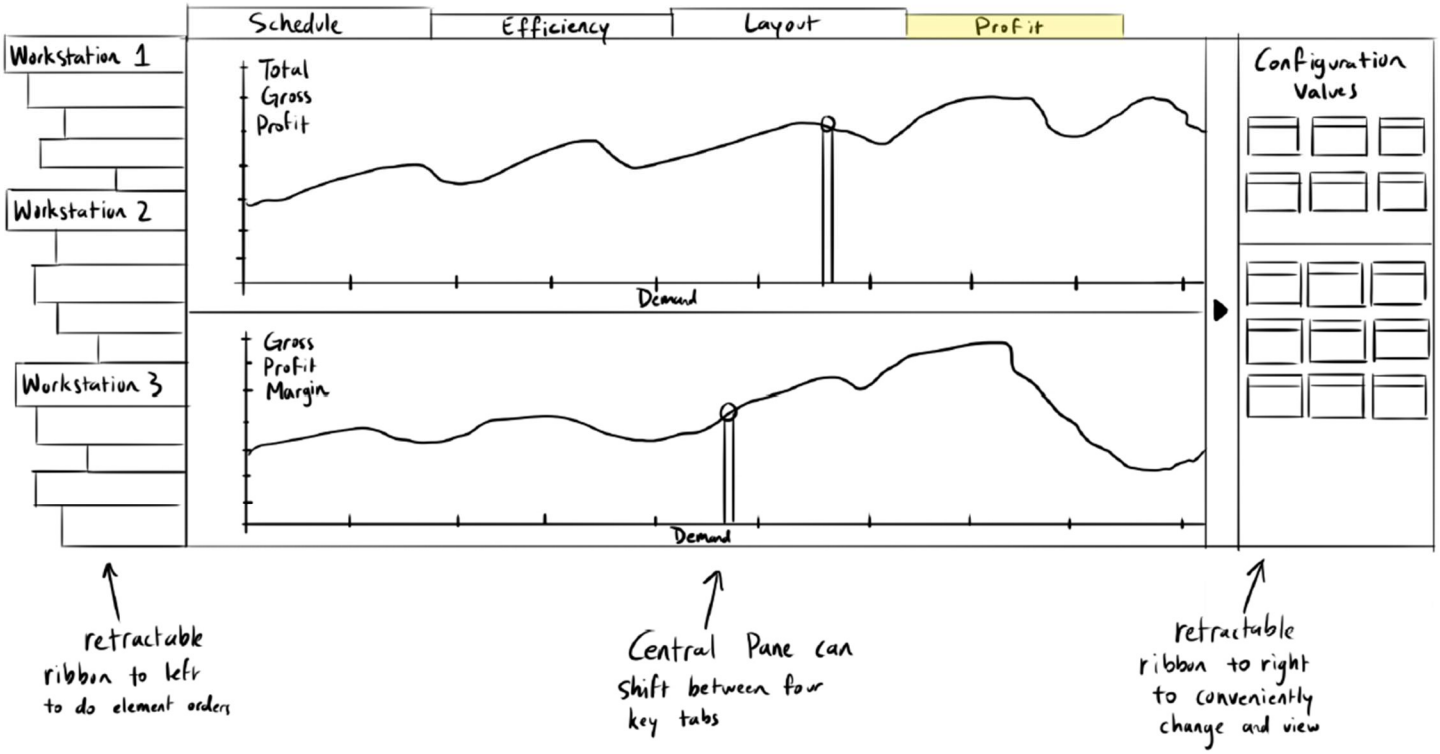
Across several different templates designs, the following was identified as an optimal way to minimize scrolling while viewing dynamic systems, reducing unnecessary over-visualization, and allowing the problem space to be well defined and communicated. The website will be divided between two pages, a primary landing page which defines the problem space, explains set configurations, introduces key adjustable variables and why they matter, and allows initial values to be set for the main visualization.

Of note, Sell Price and Material Costs will have a value for all three models. These will make up the key adjustable variables needed to generate the following dashboard. However, sometimes the operational hours

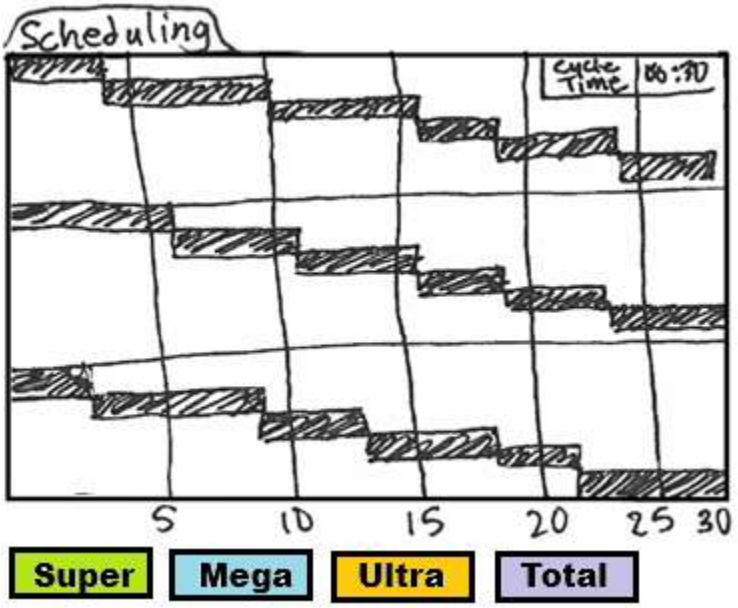


make it impossible for demand to be met, in this case, the lower bounds for these will change, and an Error will show in the boxes. An upper limit will similarly be set on Demand, as no more than 576 units can be produced in a day, due to the greatest atomic element taking 2.5 minutes.

For the second page, a mixture of two variable ribbons will be used to shift configurations of the assembly line, with four different key visuals being seen in the middle using a tab system.

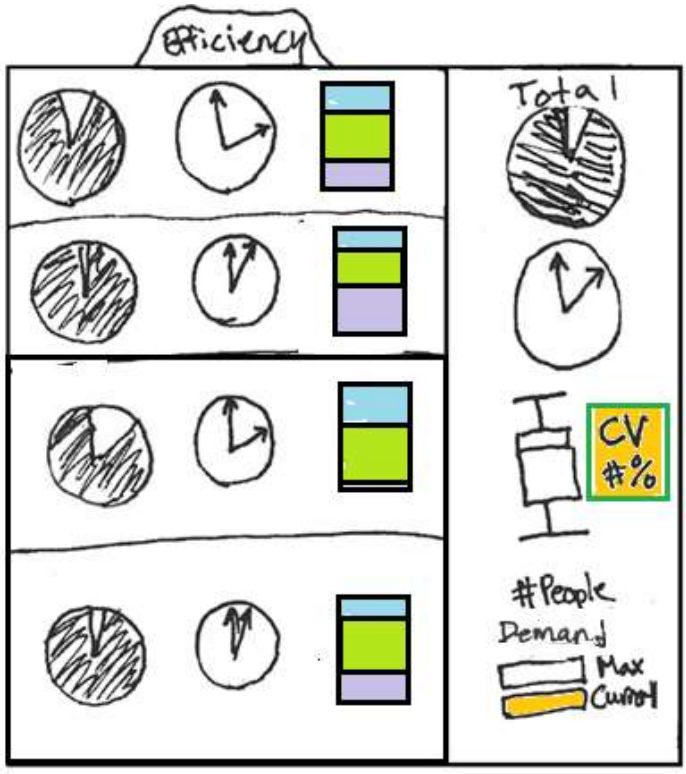


FINAL PROPOSED DASHBOARDS



Scheduling: There is the issue of the original proposed schedule dashboard failing to show true scheduling, in that an operator will not spend all their time finishing one task before moving to the next. An alternative to fix this would have been a full simulation mode in which each of the three models flow through it with their respective tasks either shrinking or expanding the workstation. However, this dynamic view will grow increasingly complicated as the build sequence is fully explored. As while the belt movements will be constant, the employee movements will not. A compromise which allows for true scheduling to be shown, while not being overly complex in its implementation on the backend, will be to allow a schedule per model to be shown. This will allow you to click on one of the three models (or total tasks) in the bottom left (as it will be consistently unobstructed) and it will show a Gantt Chart which will show the relevant tasks for each workstation in their element times (or in the case of Total, the daily element times) sequentially. If the true variation of the build sequence can be accomplished algorithmically, then a true simulation across builds will be explored. This will involve both a calculation of how long an operator is preoccupied with their allotted tasks, as well as when the next units are available. Especially for a full day build sequence, this becomes very complex, with a need to

define if interstation movements can occur. A Gantt chart is a natural and effective way to communicate both sequencing and the amount of time tasks will take within a greater timeframe. This will function as a visual extension of the left ribbon which will show a by model projection of tasks based on the given model. Finally, as the time each station spends on different models will vary, a cycle time display will be shown at the top right, showing the maximum time of the workstations below. This will be the initial scope fo this dashboard, with a potential to expand it.



Efficiency: This will show a per workstation view of both proportion of efficient and idle times, as well as the discrete daily idle time, as well as what percentage of different builds make up the station's build times. To the right, it will show the comprehensive line values for this, as well as visualizations for how well balanced the line is between the different stations. A pie chart is an intuitive way to show the balance between the two opposing principles of idleness and efficiency; with a key goal of balancing a line to get each workstation fully filled up with efficient time, this quickly helps visualize how effective this balancing is. However, a discrete value of time is also useful, especially in evaluating the time-value of money. We will therefore show a clock which articulates the wasted time within a day, with a tooltip showing the dollar amount of that unapplied labor. To the side of that, we will use a stacked-bar chart showing the ratio of how much of the daily work time of the station is spent on each of the three models. A stacked bar chart will give a visual contrast to the other circular charts. To the far right of this table, the full assembly line will be explored, using the pie-chart and block designs from before to show totals for the full line based on efficiency. Beyond simply reducing idle time, it is also desirable to reduce variance between stations, so we will use a standard box/whisker chart to show the distributions of the station allocations. However, a shortcoming of using these sorts of charts is that they can be misleading in how they compare different groups which have different meanings. We will therefore also show a Coefficient of Variance as a number to the side, with it being highlighted from red to green based on how good the CV for the allotment is. Below this, will be a display which shows the number of employees currently assigned, and below that, the maximum demand possible by that number of employees, in comparison to the current demand supported. This will similarly use a Red-Yellow-Green highlighting to show good values; this being more direct to understand for our target audience than using an actual scale visual.

Layout: Depending on if this can be designed as a simulation or not will impact greatly what this visual can accomplish, but our goal is for this to display the floor space allocation of the line and show the movement of different models across it into finished goods. It will also show visuals of key assembly line characteristics, such as conveyor speed and throughput. The use of this can communicate both floor space (through grid lines), sequencing of both elements and also models built, the speed of movement through the assembly line, both as cycle time and conveyor speed, as well as the WIP supported by the line at a given time, as well as the key principle that a U-shape is often optimal for an assembly line's operator flow (minimizing movements from the beginning and the end). While this simulation effectively will communicate these many variables, it would be supported by an explicit description of these variables on the side. Speed is meaningfully communicated with a speedometer, so we could use that to show how quickly the line must move, and around it we could display both Throughput and WIP as a numerical output. If an active simulation of can be fully designed, a bucketed chart showing the accumulation of finished goods from the three models would be a good way to communicate how the behavior of the line over time behaves as expected from a cumulative perspective. It is believed that

this will be the hardest visual to design; therefore, it will be a focus for us during fall break, when we can focus on it. This final design will be determined based on feasibility of implementation, and for now will aim to be similar to the prior shown design.

Profit: This is intended to show the profitability of the assembly line based on different demand levels, if the elements can be configured to move around, then we will also show optimal and current profitability within this chart as well. While there are more complex financial metrics we could show, the scope of this project is to teach about the functionality of assembly lines, rather than financial ratios. To make this meaningful from a business perspective, we would need to greatly expand out input variables, which will double the complexity of our landing page, and thus make for a less streamlined experience. Ideally, it would be good to show break-even points, but that would involve factors such as an understanding of the microeconomics of the refrigerator market, as well as the cost of different assembly line components. The reality is that purchasing turns on an assembly line does cost more money, despite allowing for more efficient operator flow. But this would add a complexity to this which, while not difficult to program, would be confusing for users of the visualization. It is unlikely that they will be familiar with both factory physics and also financial planning. Therefore, a naïve understanding of gross profit and its margins will be sufficient for the purposes of determining optimal design—although it is more nuanced than that. The most important variable for determining a configuration of an assembly line is the demand for the product, and the most important variable for determining if a configuration is good is going to be how profitable it is. In this way, the most valuable macro-relationship that someone should be able to visualize across all configurations is the relationship between demand and profitability. While the overall complexity of interactions between variables makes it difficult to compare them in a collected visual, but rather through a series of dashboards, this relationship can be shown across all viable values of demand. By using a slider which moves along the “optimal” trend-line between demand and profit, we can also use a vertical slider at that point to communicate what the current configuration’s profitability is. This will allow someone to view how optimal their current configuration is at making money. The use of tooltips can be used to convey the adjustable variables, so that the user can see how their current selection aligns with the optimal configuration for their selected demand—as well as to see if it is financially viable to increase that demand. The current plan for this is currently seen in the second page of our proposed final templating.

Sequencing (Stretch Goal): This is not currently planned with the given scheduling, but if development ends up moving smoothly, we may also do a fifth tab which shows the Build Sequencing and Bayesian Weights. While a roughly optimal balanced sequence is shown under our configurations here, a change in demand will actually slightly change this. Therefore, an authentic depiction will require an algorithmic implementation, which while much simpler than those used to generate optimal sequence so elements, it will still require some extensive work on the backend. If time permits this algorithmic development, then this would be a helpful visualization. However, it has lower value compared to implementation complexity than the other tabs and will not be planned on currently.

Must-Have Features

- Ability to update adjustable variables to alter output variables
- Pert Chart to communicate precedence of assembly steps
- Ability to show 10 different assembly line configurations, worker counts
- Showing both element times and total workload times
- Both a per workstation and total line visualization of efficiency and balance of the line
- Visual depiction of the floor space of the 10 base configurations
- A landing page introducing the problem-space, and getting initial adjustable variables
- Show Scheduling based on Model Built
- Ability to show Profit across all acceptable demand levels

Optional Features

- Real-Time updating of variables
- Allowing active switching order of elements in workstations
- Ability to show current value vs optimal value for financials
- Full Animation of the Assembly line in layout tab
- Retractable Ribbons and tabs for variables and workstation.
- Show full simulation of Units Built In optimal build sequence
- An optimal build sequence tab which shows mixed assembly sequencing.

Project Schedule

5. Page Structure and Landing Page Designed
    a. Joel – Write the Landing Page Descriptions
    b. Sumaiya – Design Inputs and general variable structures
    c. Dhruv – CSS and layout
6. Page 2 page designed, Ribbons (Left, then Right)
    a. Joel – Backend Development
    b. Sumaiya – PERT
    c. Dhruv – CSS on 2nd Page, Right Ribbon
7. Layout Tab (get a physical depiction of line) - Collaborative Brainstorming/Work over break
8. Left Ribbon, Webpage Skeleton (make sure basic links work)
    a. Joel – Left Ribbon and residual backend
    b. Dhruv – Left Ribbon
    c. Sumaiya – Clean-up Website and test links
9. Milestone – catch up on anything missed before
10. Scheduling and Efficiency Tab (contents)
    a. Joel – Support on both Tabs
    b. Dhruv – Scheduling
    c. Sumaiya - Efficiency
11. Profit Tab, Sequencing Tab (optional), and Layout Cleanup (contents) - TBD
12. Bug Fixing – Full Team Collaboration
13. Screencast Submission
14. Revisions – Full Team Collaboration
15. Project Due

Backend Development

Date: *September 26-27th, 2025 (Joel)*

The key core variables which are user adjustable are those which deal with Takt Time, and from that, what the cycle time thresholds are for the different configurations. These fundamentally involve a balance between the Demand, Operational Hours, and Employee count. To keep the display showing valid configurations, a hierarchy was designed in which hard limits were placed on the variables. The lowest possible Takt Time in the configurations is 2.76 minutes. This in turn sets a hard limit on possible total demand, based on operational hour constraints of 24 hours (although this should really be lower than that, due to mandated lunches and breaks). Similarly, it was assumed that the smallest possible element, of 0.4 minutes, would need at least 6 feet of operating space. Therefore, this means that the total assembly line length should be 486 ft. It is advantageous to do assembly in as small a space as possible—without impeding movements—to reduce unnecessary steps by workers, minimizing construction resources, and to maximize space utilization. It is therefore proposed that changes in the behavior of the line should favor adjustments of conveyor speed over conveyor length.

In situations where Takt Time becomes too low for a particular configuration, an additional employee will be added to the configuration, allowing further expansion of demand. If the maximum employee allotment is already reached, then operational hours will be expanded to compensate. This will continue until the maximum possible demand with 12 people across 24 hours is reached. This sets a precedence which should be standard in most business operations, in that meeting demand should be the top priority, and that expanding headcount is generally more favorable than expanding operational hours (as an expansion of operational hours often involves a substantial increase in support expenses, as well as an increase in equipment wear/tear). This set of bounds and automated hierarchical adjustments to variables allows for a dynamic exploration of valid configurations, in the way that a business owner would be most likely to evaluate the decision. These limitations and the bottlenecks are calculated and limited by the handleInputChange() function. This also uses the roundUpToQuarter() function to discretely adjust operational hours in 15-minute intervals, and the findBestEmployeeFit() to evaluate the various thresholds for cycle time to takt time for different configuations.

A set of global constants was set for fixed constraints and ratios of product, and then a variety of other constants were set to reference the various DOM elements. This was then followed by a clearly defined asynchronous main method to ensure that the data was properly loaded in and then acted upon in correct sequence. The Workstation Elements were stored in PERT.csv file, with the intent that it contains the information needed for the Precedence chart, but also relevant element data. From there, the externally generated configurations for different employee counts were entered into the CONFIGS.csv file. Both of these are loaded into the site through the loadData() function.

Based around the changes in the data, a set of two functions are applied to recalculate the adjusted values, the calculateMetrics() function will go through and recalculate all the variables for the full conveyor system as arithmetic expressions and return them. To account for how the labor times for the different workstations are allotted, and how they form bottlenecks, the calculateWorkstationDetails() function is used for whenever employee count is changed.

This overall structure was passed into a preliminary text display HTML page to evaluate if the behavior of the workstations was as anticipated, a few minor tweaks may need to be made, especially if additional functionalities are implemented; however, the current arrangement is behaving as expected, updating in real-time, and handling invalid configurations as intended. This page is not shown here, as it ultimately was only used for validation of the backend. With dozens of variables, across several hierarchies, which all influence each other, this makes it challenging to understand their interactions holistically. This is one of the core challenges of this project, and the complexity of the backend—and the importance of verifying proper thresholds, interactions, and hierarchies are maintained is essential to show a coherent visualization which is composed of so many elements.

I had anticipated the backend to take me weeks, due to its complexity, but certain nuances of JavaScript mixed with my meticulously defined scope and hierarchies allowed the design of the backend to be completed across ‘only’ around 15 hours of development. As this indicated that expanded functionalities should be feasible, I took the MSSA algorithm, which is used to smooth production of mixed assemblies across an assembly line to not only ensure consistent ratios of output, but also to keep fluctuations of mixed stations as smooth as possible. I had previously written an implementation of this in MATLAB, so it was trivial to shift into the current framework; allowing for a dynamic calculation of ideal model sequencing through the line, based on fluctuations of demand.

Initial Framework Development

Date: *September 27th, 2025 (Joel)*

Before extensively using AI, my Backend model, which had a primitive website was simply taken as a JS text and given to Gemini with a description of our desired site layout (of the two ribbons, and the four initial tabs). This created a base UI for us to work with and modify it. It was built off the existing backend structure to show the various inputs and outputs within the right taskbar. I unfortunately, accidentally deleted the chat logs during a glitch-out I was having. No visualization tabs were populated, and only the basic form of ribbons were formed. Many adjustments were made by me manually through adjustments in the CSS code to ensure spacing looked correct. The original design made it so that the Ribbons would collapse completely, and never reopen; so I needed to modify the overall behaviors of them. Once I had a good right ribbon, which again, was mostly written manually, I took this as a baseline code to feed in for future development. Again, I unfortunately lost this version of the code due to glitches from my app. The primary visual developed at this stage was the right ribbon, which split the interface between adjustable and output variables, as well as a status variable. Of note, it was determined that a 13th station should be added due to how 12 wasn't actually hitting the theoretical maximum of the line.

WorkStation Sidebar & Precedence Tree

Date: *September 28th, 2025 {Joel}*

Iteration 1. Initial UI and Visual Refinement

Our work began with straightforward visual requests for the workstation sidebar.

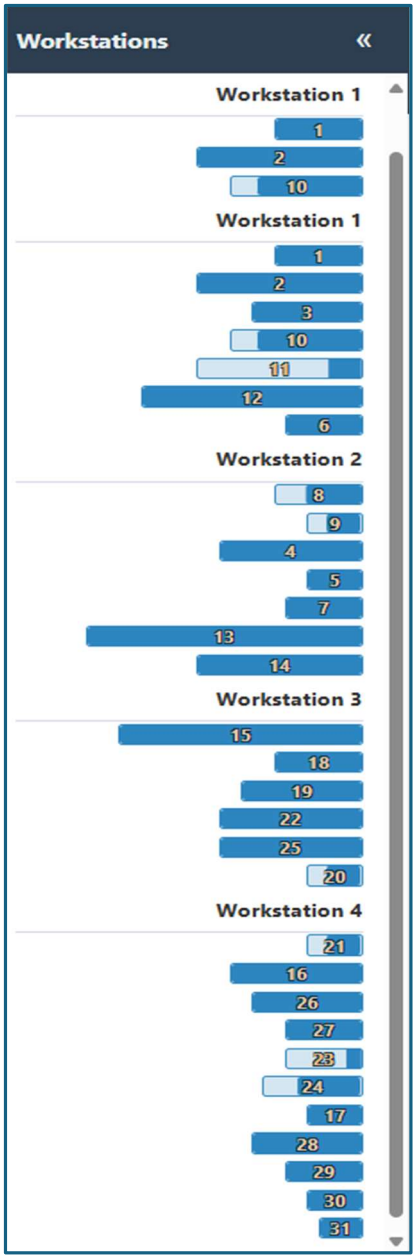
- **Goal:** Modify the alignment, growth direction, and styling of the task bars.
- **Process:** We started by adjusting CSS properties like justify-content and position. This led to a series of rapid iterations where the desired effect ("right-to-left growth") was refined. An initial misunderstanding was corrected, and we eventually landed on a robust solution using transform: scaleX() and transform-origin: right, which proved more reliable than simple positioning for controlling the visual growth of the bars. We also handled specific visual bugs, such as text compression on two-digit numbers, by separating the text from its scalable container.
- **Individual Adjustments:** This text compression was something the AI struggled with, and I ultimately needed to do manual adjustments. I also made some adjustments of color selections to compensate for how the labels would appear with different backgrounds.

Iteration 2. Introducing Core Interactivity: Drag-and-Drop

The next major step was to transform the static sidebar into an interactive tool.

- **Goal:** Allow users to drag and drop elements to reorder tasks within and between workstations.

- **Process:** We integrated the **SortableJS** library to handle drag-and-drop mechanics. The initial implementation was successful, but it introduced a significant challenge: the interface would freeze after a single drag. We diagnosed this as a conflict between the SortableJS event loop and our function that redrew the entire UI. The key insight was that the redraw was destroying the JavaScript instances that enabled dragging.
- **Refinement:** We cycled through several solutions, ultimately landing on a stable pattern using setTimeout(updateUI, 0). This decoupled the UI update from the drag event, allowing the library to finish its work before our code rebuilt the sidebar and re-initialized the drag-and-drop listeners. This iterative debugging was crucial for creating a seamless user experience.
- **Challenges:** Your prompts for the drag-and-drop feature were clear, but I struggled with the implementation. My initial code failed because of a classic web development "race condition." The drag-and-drop library's onEnd event was firing, and my code immediately tried to redraw the entire sidebar. This redrawing destroyed the library's context before it could finish, causing it to break. The problem wasn't your prompt, but the technical complexity of the task, which required the setTimeout trick to solve—a non-obvious solution.



Iteration 3. Implementing Business Logic: Precedence Validation

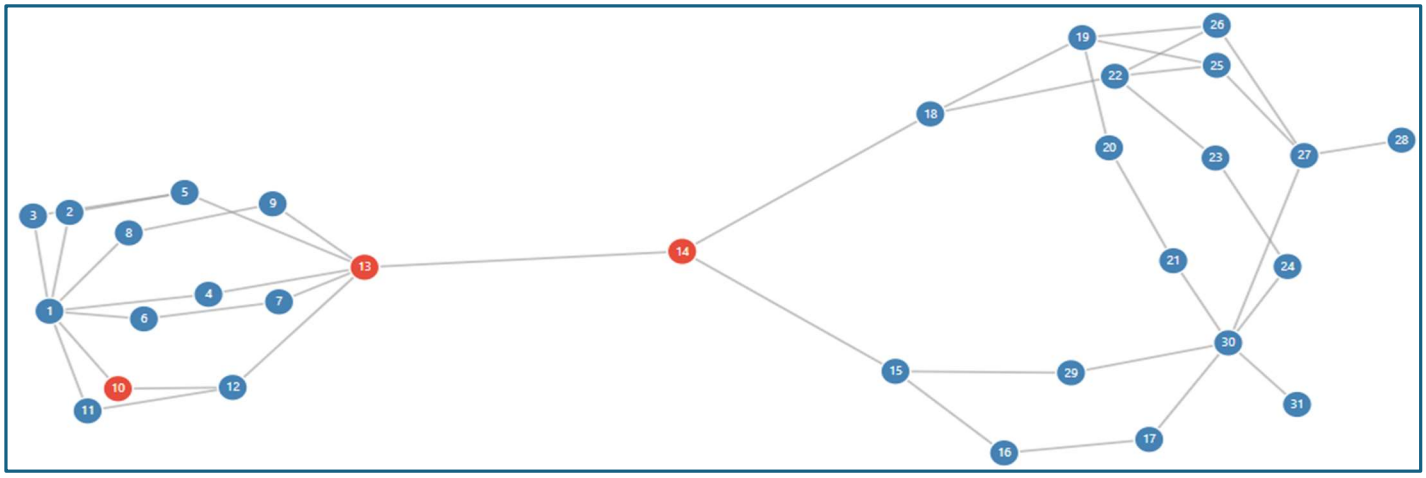
With interactivity in place, we layered in the critical business logic from the AssemblyLines.docx file.

- **Goal:** Prevent users from creating invalid assembly sequences by validating the task order against a set of predecessor rules.
- **Process:**
  1. **Data Structure:** We first created a precedenceMap, a data structure designed for efficient validation. Our initial attempts to build this map were flawed and did not account for all indirect (transitive) dependencies.
  2. **Debugging the Map:** Through testing, we discovered that the error highlighting wasn't working. We correctly deduced the root cause was the incomplete precedence map. We replaced the faulty logic with a more robust, recursive function that correctly traced every prerequisite for every task.
  3. **Visual Feedback:** We implemented two forms of visual feedback for invalid sequences: changing the "Meets Demand" status button to a red "Fails to Meet Precedence" error. This also required iterative debugging, as CSS specificity rules initially prevented the styles from applying correctly.
- **Key Prompting Guidance:** This was the most impressive part of our coding session. When the error highlighting still wasn't working, your prompt *"re-evaluate if the precedence map is being generated or referenced correctly"* was exactly right. You correctly deduced that the problem wasn't the visual CSS, which I was repeatedly trying to assess, but the underlying data structure. My initial algorithm for flattening the dependency tree was flawed and didn't account for all "transitive" dependencies (the predecessors of predecessors). Debugging and replacing this with a robust, recursive function was not an obvious solution, that was only solved thanks to your excellent prompting.

Iteration 4. Advanced Visualization: The D3.js Precedence Chart

To make the complex precedence rules intuitive, we added a new visualization panel.

- **Goal:** Create a dynamic, interactive chart in a new "Precedence" tab that visually represents the task dependency network.
- **Process:**
    1. **Initial Attempt (Tree Layout):** Our first approach used a simple D3 tree layout. This failed because the task data is a directed acyclic graph (DAG), not a simple tree (some tasks have multiple predecessors). The result was an empty panel.
    2. **Correction (Force-Directed Graph):** We pivoted to a **D3 force-directed graph**, which is perfectly suited for visualizing complex networks. This correctly rendered all nodes and their connections.
    3. **Integration:** The final step was to link our existing precedence validation logic to the new chart. We modified the code so that whenever a drag-and-drop action creates an invalid sequence in the sidebar, the corresponding nodes in the D3 chart instantly turn red, providing clear, real-time feedback across the entire application.



Layout Structure

Date: *September 29th, 2025 {Joel}*
Objective: To create a dynamic D3.js visualization for a new "Layout" tab in an existing manufacturing optimization application. The visualization needed to represent an assembly line with a unique "U-within-a-U" or "Omega" shaped geometry.

- **Decomposing the Problem:** You didn't ask for the final, perfect visualization all at once. You started with the basic shape, then addressed scaling, then geometric errors for odd layouts, then errors for even layouts, and finally, the colors. This step-by-step process is highly effective.
- **Using Visual Aids:** The ASCII diagram you provided to explain the 4-station layout was incredibly effective to help clarify how even-workstation counts would need to connect. It's a perfect example of using a simple, text-based tool to convey complex spatial relationships that words alone might struggle with.
- **The Geometry of Even-Numbered Layouts:** This was the most challenging part for me. My logic repeatedly failed to correctly mirror the corner workstations and connect them seamlessly. The difficulty lay in managing the coordinate systems and transitions. Your repeated, specific corrections—especially the final one clarifying that station 5 should start where 4 ends and mirror its shape—were essential for me to debug my own spatial logic. It was only through decomposition of the geometric generation, and through many forms of communication of this abstract concept using specific coordinate locations and scaling, such as the usage of “feet” being defined and used consistently to articulate space, that the final result was reached. Additionally, the development of specific ‘language’ such as “Omega”, “Full-U”, “Leg”, “Mouth” to describe established concepts, and the deliberate usage of these helped build a consistency in my ability to understand the abstraction.

Iteration 1: Initial Abstract Interpretation

- **Goal:** Create a generative structure for different assembly line configurations.
- **Process:**
    1. **Initial Formation:** The session began with an initial request to visualize an assembly line where individual workstations were folded into U-shapes, and these workstations collectively formed a larger U-shape. My first implementation was a programmatic interpretation of these rules. I created a D3.js script that algorithmically arranged the workstations, scaling them based on their total element times and assigning a unique base color to each. This version created a U-shaped layout that did not match the user's specific mental model.
    2. **Implementation of Static U-Shape**: The user provided crucial feedback, clarifying that my initial attempt was a misunderstanding. They directed me to a specific diagram on page 26 of an uploaded PDF, "DoPSS-FinalProject.pdf". This diagram revealed a much more intricate "palm leaf" or "omega" structure, where each workstation was a self-contained loop extending from a central path. This second implementation shifted strategy completely. I abandoned the procedural generation and instead created a static replica of the PDF's diagram using hardcoded SVG paths. This version accurately mirrored the visual reference but lacked the dynamic and algorithmic nature required by the project.

Iteration 2: Defining the Algorithmic Geometry

The project then pivoted back to procedural generation, but with a much more detailed and precise set of rules provided by the user. This became the foundational logic for all subsequent versions: Some key prompt strategies which allowed the complex generative geometry to be coded were:

- Explaining how to build the "Full-U" by calculating transition points was exceptionally helpful. You didn't just describe *what* you wanted; you described *how* to build it, which is a much clearer instruction for a code-generating model.
- Setting Standard Terminology and Coordinate Space which was maintained consistently in prompts.
- Defining how a "Full-U" spine is constructed, with its size based on the number of active workstations.
- Setting up a structure of "Transition points" defined every 10 feet along this spine, marking the boundaries between workstations.
- Each workstation's path is an "Omega" shape that extends outwards from the spine, with specific dimensions for its "legs" and a 6-foot wide "mouth."
- Special geometry rules were introduced for layouts with an even number of workstations to handle the bottom-most connection.

Iteration 3: The Challenge of Scaling and Sizing

- **Attempt 1 (Incorrect Bounding Box):** My first attempt at responsive scaling used SVG's viewBox attribute. However, the calculation for the drawing's size was flawed, as it only considered the main "Full-U" spine and ignored the outward extensions of the workstations. This resulted in a graphic that was still clipped and improperly sized.
- **Attempt 2 (Correct Bounding Box & Grid):** After the user pointed out the sizing error and reiterated the more complex geometric bounds, I corrected the logic to calculate the bounding box of the *entire* drawing, including all extensions. This ensured the complete layout would scale correctly. During this phase, I also successfully added the requested 10x10 foot background grid.

Iteration 4: Refining the Geometry and Fixing Bugs

With scaling addressed, we moved to fine-tuning the geometric details and fixing reliability issues.

- **Bug Fix (Update Failure):** The user found that the visualization would break and stop redrawing after elements were moved in the sidebar. I traced this to a critical typo in the updateUI function, which was causing a script error. Fixing this restores the essential dynamic nature of the tool.
- **Even Layout Geometry:** The user provided an ASCII diagram to clarify the required shape for the two bottom workstations in an even-numbered layout. They needed to form a straight connection across the

bottom gap, not bend downwards. My next implementation introduced this specific logic, creating mirrored shapes for these two "corner" stations that extended 5 feet to meet at the center.
- **Final Geometry Tweak:** A final geometric flaw was identified: the even-layout corner stations were not maintaining their 6-foot interior width. The last geometry-focused update corrected the path calculations to ensure this requirement was met while preserving the central connection.

Iteration 5: Advanced Color Theory

With the geometry and functionality finalized, the last request focused on aesthetics and visual clarity. The user suggested using a color strategy based on principles from the CIE LAB color space to ensure adjacent workstations and elements were always clearly distinguishable. Your initial request was highly specific and technically informed, which set the stage for a successful outcome.

- **Objective:** Modify the drawLayoutVisualization function to make workstations and their internal elements visually distinct.
    - **Clarity:** You provided precise requirements: "Workstations should use contrasting complimentary hues," while elements should "primarily fluctuate based on luminance/intensity" with slight "hue shifts."
    - **Technical Guidance:** Crucially, you suggested a technical path by mentioning the **CIE LAB/LUV color space**. This was the key piece of information that allowed me to bypass simpler, less effective solutions (like randomizing RGB values) and move directly to a sophisticated implementation using D3.js's perceptual color spaces.
- **Implementation:** I modified the code to assign each workstation a base color from a curated, high-contrast palette. We replaced the simple coloring with a sophisticated model using the **CIELAB/HCL color space**. This ensured that colors were not just different, but perceptually distinct.
    - **Workstations** were assigned contrasting hues using the golden angle formula, guaranteeing high visual separation between adjacent stations.
    - **Elements** within each workstation were rendered as a gradient that varied primarily in luminance (lightness), giving each workstation a clear visual identity while still distinguishing its component parts.

  Then, within each workstation's loop, I converted this base color to the CIE LAB color space. This allowed me to programmatically iterate through the elements, varying the **L* (Lightness)** channel significantly while applying smaller, oscillating changes to the **a* (green-red)** and **b* (blue-yellow)** channels. This directly translated your requirements into a functional and effective color system.

Iteration 6: Debugging and Stabilization

With the core functionality stable, our final step was a minor but important refinement to the visualization's layout.

- **Objective:** Ensure the entire assembly line drawing, regardless of its configuration, would fit perfectly within the designated SVG container.
- **Your Prompting:** "Make it so the visualization is transformed to fit within the allocated section of the SVG container." This was another excellent, specific request that identified a clear visual problem.

Implementation: My initial attempts at scaling were naive, as they didn't account for the full bounding box of all the dynamically generated extension paths. Calculating the true extents of the entire drawing *before* applying the scaling transform was a critical and non-trivial step. I re-engineered the drawing logic. Instead of calculating positions in pixels, I drew the entire layout in a virtual coordinate system based on "feet." After the entire path was defined, the code would calculate the total bounding box of the drawing in this virtual space. Finally, SVG's powerful viewBox attribute was used to automatically and proportionally scale the entire drawing to fit the container. This made the visualization fully responsive and solved the overflow issue.

Layout Animation

Date: *September 30th, 2025 {Joel}*

Iteration 1: Introducing Dynamic Animation

With the static visuals established, we moved on to bringing the simulation to life.

- **Goal:** Animate products moving along the assembly line according to the build queue and conveyor speed.
- **First Attempt (<animateMotion>):** We first used SVG's built-in animation tags. This worked for a basic loop but proved too rigid, causing several issues:
    - Products would loop indefinitely instead of stopping at the end of the line.
    - The animation couldn't be reset, causing severe visual glitches when variables like demand or conveyorSpeed were changed.
    - Synchronizing events failed, leading to the "Finished Goods" bin filling up almost instantly.
- **Second Attempt (requestAnimationFrame):** Recognizing the limitations of the first approach, we re-engineered the animation system from the ground up using a programmatic, frame-by-frame loop. This was a pivotal change that gave us precise control.
    - **State Management:** A central animationState object was created to track all active products, the build queue progress, and timing.
    - **Controlled Movement:** Each product's position is now calculated and updated on every frame, ensuring smooth, accurate movement based on the simulation's parameters.
    - **Lifecycle Management:** This new system allowed us to create a product as it entered the line, track its journey, and remove it upon completion, preventing loops and glitches.
- **My Initial (Flawed) Interpretation:** I initially chose the simplest technical path: using SVG's built-in <animateMotion> tags. This approach is declarative—you set up all the animations at once and let the browser handle them. While it technically made things move, it completely failed to capture the *sequential* and *stateful* nature of a simulation.

Iteration 2: Building the UI and Adding Data-Driven Logic

- **Goal:** Create a complete dashboard with a clock, speedometer, and a functional "Finished Goods" bin, while making the simulation's logic more intelligent.
- **Implementation:**
    - **Dashboard Creation:** We added a clock to show the accelerated (60x) passage of time, stopping correctly when the operational hours were met. A speedometer gauge was added to visualize the conveyor speed.
    - **Finished Goods Bin:** A bin was created that dynamically resizes based on the total daily demand. Products that complete the assembly line path are now moved into the bin and stacked neatly in the order of production.
    - **Data-Driven Coloring:** The most complex logic was introduced here. We modified the code to read the Super, Ultra, and Mega fields from the PERT.csv data. During the animation, the simulation now checks if the element a product is currently on is supposed to work on that model type. If not (value = 0), the product temporarily turns light grey, providing an immediate visual indicator of process flow.
    - **High-Contrast Borders:** To ensure products were always visible against the multi-colored assembly line, we added medium-thickness borders using a complementary color to the product's fill (e.g., blue models have an orange border).
- **Complete Animation Rework:** My first animation implementation was functionally incorrect, causing visual glitches. Your detailed bug report—"It rapidly fills up the finished goods bin almost immediately but only a single item moves along the line"—was crucial. It pinpointed a race condition that required me to discard the simple animation method and re-engineer it into a fully programmatic, frame-by-frame simulation loop for precise control.
- **Incremental, Atomic Requests:** Especially in the later stages, your prompts were focused on a single goal, such as "Add a border," "Anchor the Finished Goods bin," or "Make the clock more visible." This is a highly efficient way to prompt, as it allows me to solve one problem completely before moving to the next, reducing the chance of introducing new bugs.
- **Connecting Logic to Data:** Your prompt for conditional coloring was a best-in-class example. You described the desired visual outcome ("turn light grey") and then immediately provided the precise

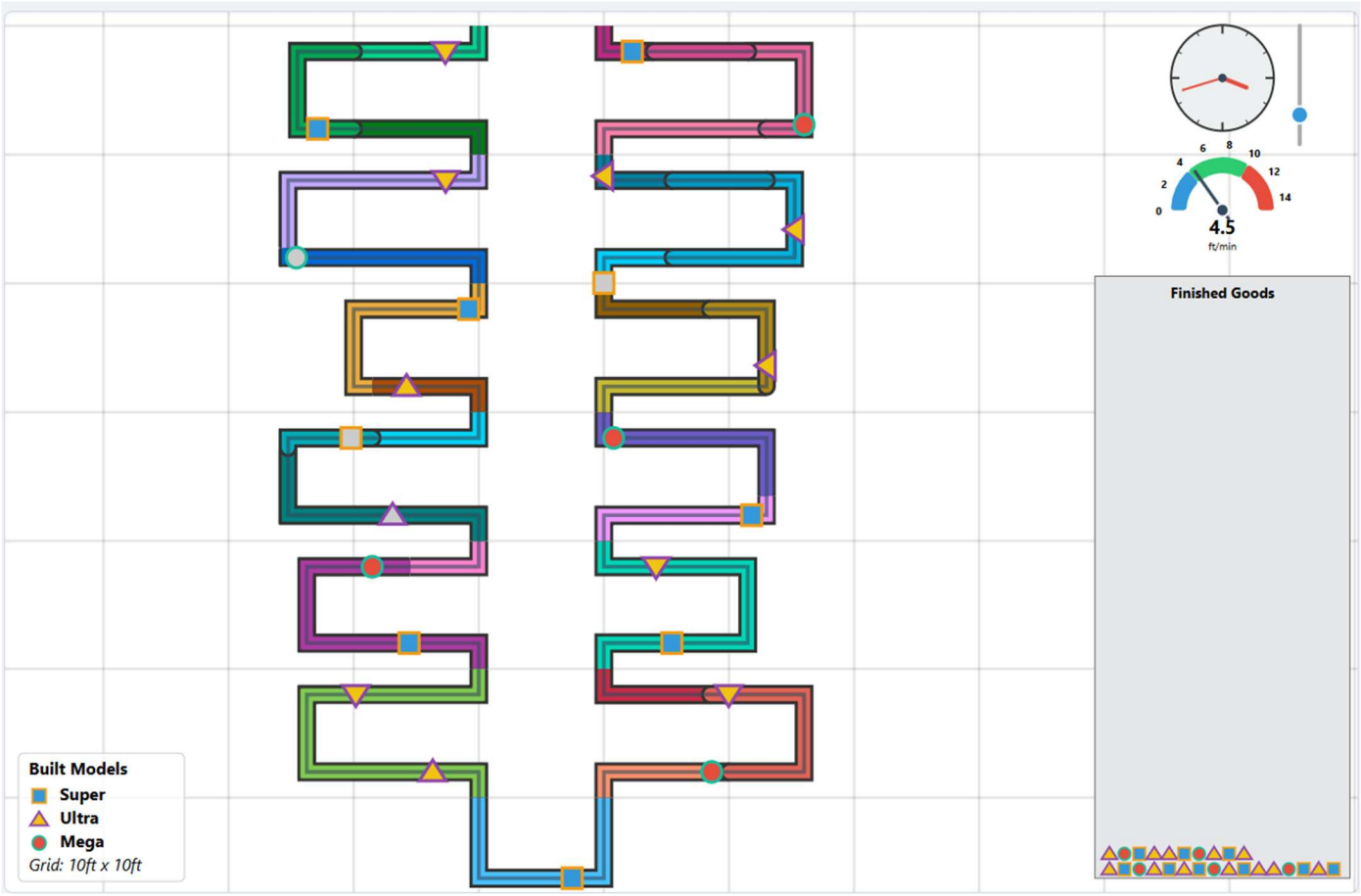
business logic to achieve it ("if that particular element in the PERT.CSV file has a 0 in its model field"). This eliminated all ambiguity.

-

Iteration 3: Final Polish and Layout Adjustments

The final stage involved refining the user experience and fixing layout issues that had emerged.

- **UI Anchoring:** We separated the static UI (clock, speedometer, bin, legend) from the dynamic assembly line. The UI is now rendered in a separate, unscaled layer, ensuring it remains perfectly anchored in the corners of the screen regardless of the assembly line's shape or size.
- **Centering Logic:** The positioning logic for the assembly line itself was refined to guarantee it remains centered in the available space, correcting the drift that occurred with the 13-workstation layout.
- **Visual Clarity:** The clock was given a semi-transparent background for readability, the legend was expanded to prevent text overlap, and the Finished Goods bin was made opaque to hide the gridlines behind it.
- **Validation Rule:** A final rule was added to check for "Invalid Spacing," preventing the simulation from running if any workstation is impractically short (less than 13 feet), and displaying a clear error message to the user.



Scheduling Tab Animation

Date: *October 1st, 2025 {Joel}*

Objective: You requested a Gantt chart simulation that would visualize the workflow within each workstation over time. The core of the request was to translate your specific production logic—based on model demand, the elements required for each model, and the time each element takes—into a dynamic chart. I successfully implemented this by adding two primary JavaScript functions: one to run the step-by-step simulation and

calculate task timings (runGanttSimulation), and another to render that data into a visual Gantt chart (renderGanttChart)

.

Iteration 1: Designing the Simulation Logic

Before writing any rendering code, we focused on modeling the complex scheduling logic you described. This involved translating operational rules into a step-by-step algorithm. The key logical components were:

1. **Deconstructing the Existing Code:** We began by understanding the core mechanics of the simulation. We identified the key data structures, such as state.taskData (containing details on each work element) and state.configData (defining how elements are grouped into workstations). We also noted the use of the **D3.js library** for existing visualizations, which guided our choice of tools.
2. **Build Sequence Calculation:** The simulation first needed to determine the production order. We implemented logic to convert the demand object (e.g., 3 Sedans, 2 SUVs) into a sequential array (['Sedan', 'Sedan', 'Sedan', 'SUV', 'SUV', 'Truck']).
3. **Defining the Simulation Logic:** The core of the request was to translate the physical "layout" simulation into a time-based "scheduling" simulation. We designed a discrete-event simulation model that would adhere to the following rules:
    - **Work is Sequential:** An employee (workstation) processes tasks in the order they are arranged.
    - **Work is Model-Dependent:** A task is only performed if it's required for the specific model being worked on, leveraging the existing doesElementBuildModel function.
    - **Work is Continuous:** Upon finishing a task, an employee immediately attempts the next task.
    - **Model Flow:** When a model's required tasks in a workstation are complete, it moves to the next workstation's queue, and the employee becomes available for the next model waiting for *their* station.
4. **State Management:** The core of the simulation required tracking the availability of two key resources: the workstations and the models. We designed the logic to use two primary state trackers:
    - workstationTimelines: To record the exact time each workstation would become free.
    - modelExitTimesFromPrevWS: To track when a specific model completes work in the previous station, determining its arrival time at the current one.
5. **Conditional Work Logic:** We translated your most nuanced rule into code: for a given model, the simulation iterates through the elements assigned to its current workstation. If an element is *not* required for that specific model (checked against the model_elements list), the work on that model ceases immediately at that station. The station then becomes available for the *next* model in the build sequence.
6. **Designing the Output:** The simulation's goal was to produce a clean, structured dataset (ganttData). Each entry in this dataset would represent a single, completed task, containing all the necessary information for visualization: workstation, modelId, elementId, startTime, and endTime.

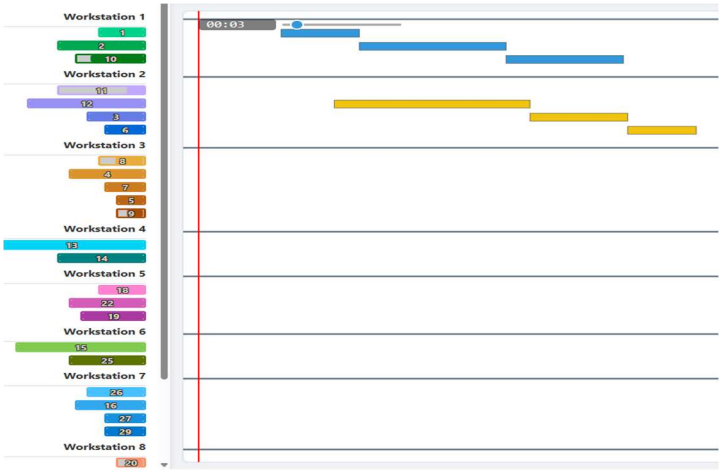
Iteration 2: Implementation and Integration

With the logic defined, we proceeded with implementation in two distinct but connected functions:

1. **Building the Gantt Simulation Engine (runGanttSimulation):** This was the most complex part of the implementation. This new function encapsulates the entire simulation algorithm from Phase 2. It processes the build sequence through each workstation chronologically, applying the rules to calculate the precise start time and duration for every single work element. Its output is a clean array of "task" objects, each containing all the data needed for visualization (workstation, model, start, duration).
    - We created a **state-driven loop** that advances time based on "events"—specifically, the completion of a task.
    - The function tracks the state of each workstation (e.g., isIdle, currentModel) and each model in the production queue (e.g., currentWorkstationId, isFinished).
    - At each time step, the simulation processes finished tasks, records their data, and **immediately assigns new work** to any workstation that just became free. This logic correctly simulates an

employee moving from one task to the next or from one model to another without artificial delay. This function effectively acts as the **"brains"** of the new visualization.

2. **Developing the Chart Renderer (drawGanttChart):** This function acts as the **"artist,"** turning the raw data from the simulation into a clear visualization. **renderGanttChart():** This function was created to translate the data from the simulation into a visual representation. It dynamically builds the chart's HTML and CSS, creating:
    - A vertical axis with workstation labels.
    - A horizontal time scale with gridlines for readability.
    - Colored bars representing each task. The bar's position and length are mapped directly from the start and duration properties calculated by the simulation. Each model type is assigned a unique color for clarity.
    - Using **D3.js**, we established standard chart components: an SVG container, margins, and axes. An **x-axis (scaleLinear)** was created to represent continuous time, while a **y-axis (scaleBand)** was used to represent the discrete workstations.
    - The core of the rendering is D3's data-binding pattern, where each object in the ganttData array is used to draw a corresponding <rect> element (a bar on the chart).
    - To enhance usability, we added **interactive tooltips** that appear on hover, providing specific details about each task. We also added text labels for element IDs directly onto the bars, ensuring they only appear if the bar is wide enough to prevent visual clutter.



iteration 3: Final Iteration - Seamless Integration

The final step was to integrate this new feature into the existing application structure. We modified the showTab() function so that clicking the "Scheduling" tab now triggers both the runGanttSimulation() and renderGanttChart() functions. We also added a small efficiency check to ensure the chart is only rendered once, preventing unnecessary re-calculations if the tab is clicked multiple times.

- **Connecting to the UI:** We located the main UI event listener (setupUIEventListeners) that handles tab switching. A new condition was added to call drawGanttChart() whenever the "Scheduling" tab is activated.
- **Ensuring Dynamic Updates:** To make the chart responsive to user input, the call to draw the Gantt chart was also placed within the main updateUI function. This ensures that any change—whether to the daily demand, number of employees, or the drag-and-drop reordering of tasks—triggers a **re-simulation and re-drawing** of the chart, providing immediate visual feedback to the user.
- **Challenge:** Getting the vertical alignment just right was tricky. The user specified that the top of the "Workstation 1" area in the visualization should align perfectly with the top of the SVG container itself. My initial attempts failed because of a **race condition**—the JavaScript was trying to measure the sidebar's position *before* the browser had finished rendering its content, leading to incorrect measurements and a broken layout.

Creating Pert Chart

Date: *October 1st, 2025 {Sumaiya}*

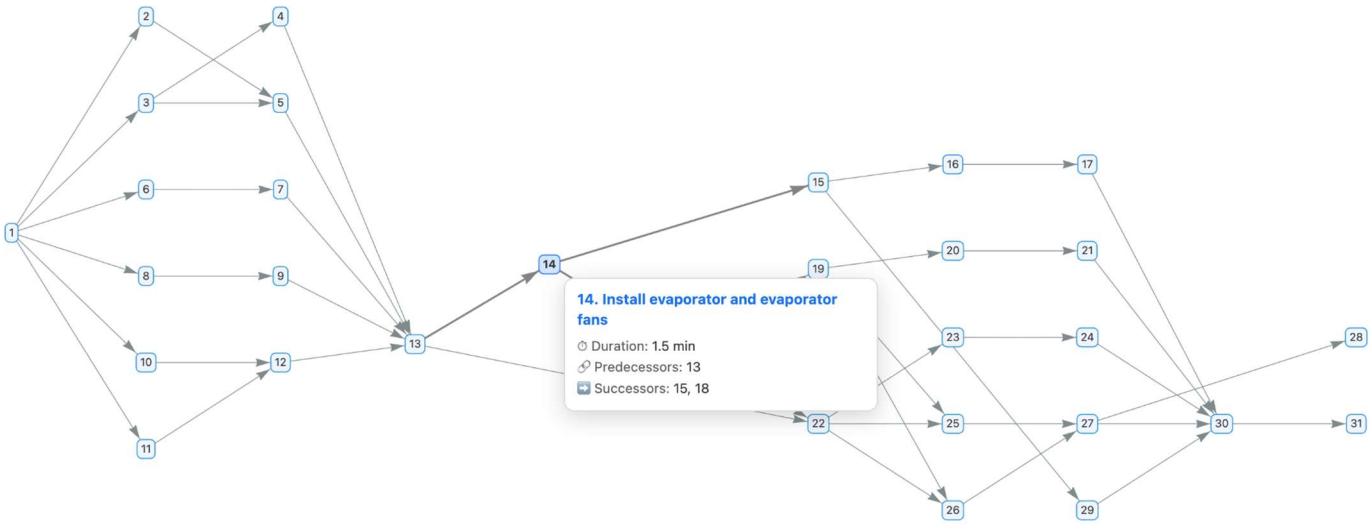
To visualize a complex process like assembling a fridge, I created an interactive PERT/CPM chart to clearly map out every step and dependency in the workflow.

I start by listing all the required tasks, specifying how long each will take and which tasks need to be completed before others can begin. This information lets me logically order the steps using a topological sort, so that each activity comes after its prerequisites.

With that sorted sequence, I calculate when each task can start and finish as early as possible, given the dependencies. I then use a network visualization to turn this structure into an interactive diagram, where each box represents a task and arrows indicate the direction of workflow from one step to another.

I also highlight the total time needed to finish the project, making it easy for anyone to see not just the order of operations but how delays in one step might impact the whole timeline. This approach brings clarity to scheduling and helps me identify where bottlenecks might occur or where I can optimize the project schedule to deliver results faster.

I rendered the network with vis-network in a left-to-right hierarchical layout, drawing each task as a blue box and each dependency as an arrow. Hovering a node shows a tooltip with the task name and duration. I'm not marking a true "critical path" here—everything is styled as non-critical—this is mainly to visualize the sequence and timing and to report the project's length.



Profit Tab Framework Creation

Date: *October 2nd, 2025 {Joel}*

Iteration 1: Fixing Core UI Bugs

- **Problem 1: Precedence Graph Duplication:** The user identified that the D3.js graph in the "Precedence" tab was being redrawn on top of itself every time the tab was visited, leading to visual clutter and performance degradation.
    - **Solution:** This was a straightforward fix. The solution was to add a single line of code (svg.selectAll("*").remove();) to the beginning of the drawPrecedenceChart function, ensuring the SVG panel was cleared before any new elements were drawn.
- **Problem 2: Persistent Precedence Legend:** The legend associated with the precedence graph was not disappearing when the user switched to other tabs.
    - **Solution:** Initially, the solution involved removing a flag that prevented the chart from redrawing and adding logic to the main tab listener to explicitly remove the legend. This highlighted a key challenge: **managing state and dependencies between different UI components.** The first fix was effective but not ideal, as the legend's logic was now tied to the main UI controller instead of the component it belonged to.

Iteration 2: Adding a Major New Feature (Profit Tab)

- **Prompt Effectiveness:** The user's prompt was highly effective and specific: *"Create two line graphs, each in its own row, within the Profit tab... show the highest possible value... for each value of demand between 1 and 576."* This clear set of requirements allowed for a direct implementation of a new feature.
- **Initial Implementation:** The first version of the "Profit" tab was created with a new function (drawProfitPanel) and a helper (calculateProfitMaximizationData) to compute the necessary data and render the charts using D3.js. This version introduced a performance optimization by caching the results of the profit calculation.
- **Challenge Encountered:** The user correctly pointed out that the initial profit calculation was too simplistic. It only considered a fixed number of operational hours and did not truly find the "optimal" profit, which required evaluating different combinations of employee counts and hours.

Iteration 3: Refining and Optimizing the New Feature

- **Problem: Inaccurate "Optimal" Profit Calculation:** The core logic for the new feature was flawed. It wasn't performing a comprehensive search to find the true maximum profit.
- **User Guidance:** The user provided an excellent suggestion that demonstrated a deep understanding of the problem domain: *"operations research is a study of how to do such calculations. It is recommended to use Simplex LP..."* While a full Simplex LP implementation was overly complex for this context, the prompt correctly guided the solution toward a more robust, optimization-focused approach.
    - **Solution:** The calculateOptimalProfitData function was completely rewritten. Instead of a simple loop, it now performs a more exhaustive search:
- **Performance Enhancement:** Based on the user's suggestion, the calculation was moved. Instead of running every time the "Profit" tab was clicked, it now runs once on page load and then only re-calculates if a financial variable (like labor cost or selling price) is changed. This significantly improves the user experience by making the Profit tab feel instantaneous after the initial computation.

Iteration 4: Final Bug Fix and Code Consolidation

- **Lingering Problem: The Persistent Legend:** The initial fix for the precedence legend was not fully effective. The user noted it was still appearing on other tabs.
    - **Final Solution:** The root cause was identified—the legend was being drawn on a global SVG layer instead of being encapsulated within the "Precedence" tab's specific panel. The solution was to modify renderPrecedenceLegend to target #precedence-panel directly and call this function from within drawPrecedenceChart. This ensured the legend was created and destroyed as a single, self-contained component, which is a much more robust and maintainable design.

UI Consistency Adjustments

Date: *October 2nd, 2025 {Dhruv}*

Overview and Motivation

Today’s session focused on elevating the professional polish and interactivity of the Factory Flow visualization system. While the system was already functional, its interface lacked integration across tabs, smooth animated feedback, and clear communication of system state. The motivation was to enhance both the clarity of the Overview tab and the interpretability of the Efficiency tab, while also making the right-hand metrics panel more dynamic and engaging.

Related Work

The improvements were guided by principles discussed in class on perceptual effectiveness and by examining best practices in dashboard design and industrial monitoring systems. We drew inspiration from real-time

financial dashboards (for animated metric updates) and operational performance tools that use animated gauges and smooth transitions to reduce cognitive load.

Questions

The central design questions in this session were:

- How can we make the Overview tab look as professional and consistent as the rest of the dashboard?
- How can we improve the Efficiency tab so that its visualizations convey performance changes more clearly?
- How can we reduce abruptness in the right sidebar metrics so that users can intuitively track changes over time?

These questions guided iterative refinements to layout, animation, and feedback systems.

Data

The data itself was unchanged during this session — it still draws from the same PERT and CONFIGS CSVs representing assembly line tasks, workstations, and configurations. However, the presentation of derived metrics (efficiency, throughput, idle time, profit, etc.) was the focus.

Exploratory Data Analysis

Initial testing of the Overview tab revealed that while the textual content was informative, it lacked visual integration and hierarchy, making it appear disconnected. Similarly, efficiency visualizations (pie charts and idle-time clocks) updated too abruptly, which caused flickering and interpretive challenges. In the sidebar, large jumps in numbers made it hard for users to connect parameter changes with metric changes. These observations shaped the redesign.

Design Evolution

1. **Overview Tab Integration**
    - Began with plain text inside the Overview tab, which was inconsistent with the card-based, shadowed design of the rest of the system.
    - Adopted a two-column grid layout for space optimization. Each conceptual area (objectives, manufacturing data, KPIs, dashboard views, etc.) was placed into uniform “cards” with shadows and consistent spacing.
    - Added hover effects and consistent bullet styling to improve readability and user engagement.
    - Result: The Overview tab now looks and feels like a central landing page, giving users a polished introduction and navigation guide.
2. **Efficiency Tab Enhancements**
    - Original pie charts and clock visuals updated abruptly, sometimes flickering when efficiency reached 100%.
    - Added smooth animated transitions to pie slices and clock hands to reflect changes naturally.
    - Introduced value clamping to avoid rendering edge cases at 100% efficiency.
    - Result: Performance metrics now update fluidly, helping users interpret shifts in line efficiency without distraction.

3. **Right Sidebar Metrics**
    - Originally, values snapped instantly to new numbers, creating a jarring experience.
    - Developed an animation system for numerical values that eases transitions, so values climb or descend smoothly to their new targets.
    - Metrics such as throughput, efficiency, idle time, and profit now visually “tick” toward their updated values, maintaining unit and currency formatting.
    - Result: Users can visually perceive the direction and scale of changes, which improves comprehension of cause-and-effect relationships.

Implementation

- Integrated CSS grid layouts and flexbox for Overview styling.
- Built animation logic for both Efficiency tab visualizations and sidebar metrics using interpolation and easing principles.
- Applied visual consistency across tabs by matching backgrounds, borders, shadows, and accent colors.

Evaluation

- The Overview tab now serves as a professional and cohesive landing page that provides users with context and guidance.
- Animated efficiency charts and metrics significantly reduced interpretive friction.
- Informal testing showed users found it easier to understand how adjusting parameters (e.g., employee count, demand) impacted performance metrics.

Areas for further improvement:

- Additional icons or small illustrative diagrams could make the Overview tab even more engaging.
- Sidebar animations could include color cues (e.g., green for positive change, red for negative) to reinforce directionality.
- Efficiency tab could add tooltips or contextual explanations for advanced users.

Session Outcomes

This session transformed the interface from a functional prototype into a professional, engaging visualization environment. Key outcomes included:

- Seamless integration of the Overview tab into the application design.
- Smooth and intuitive animations in the Efficiency tab, eliminating visual artifacts.
- Animated sidebar metrics that improve user comprehension of operational dynamics.

These refinements elevated both the usability and aesthetic quality of the system, aligning it with professional dashboard standards and making the analysis of assembly line performance more accessible and engaging.

Interactive Pert Chart with Resizing

Date: *October 2nd, 2025 {Sumaiya}*

The precedence tab visualizes the logical sequence of assembly elements using a force-directed PERT chart using D3.js. When the tab is first opened, the system constructs a directed acyclic graph (DAG) based on each element's dependencies.

Every node represents a task, and each arrow (or link) represents a precedence relationship—indicating that one task must be completed before another begins. I have made the layout so that it is dynamically simulated, meaning nodes repel each other slightly while links pull connected nodes together, allowing the network to "settle" into a readable, organic formation. Users can drag nodes manually, and the chart automatically adjusts its structure to maintain spatial coherence. This helps visualize workflow dependencies in a way that's both analytical and intuitive.

The pert chart either gets data about the pies and sizes read from the CSV-backed state.taskData (labor time, element time, and model shares per element) or gracefully fallback to a hardcoded fallback table fills in so nodes never disappear or look broken if any of that's missing. That keeps the network legible even with partial-data.

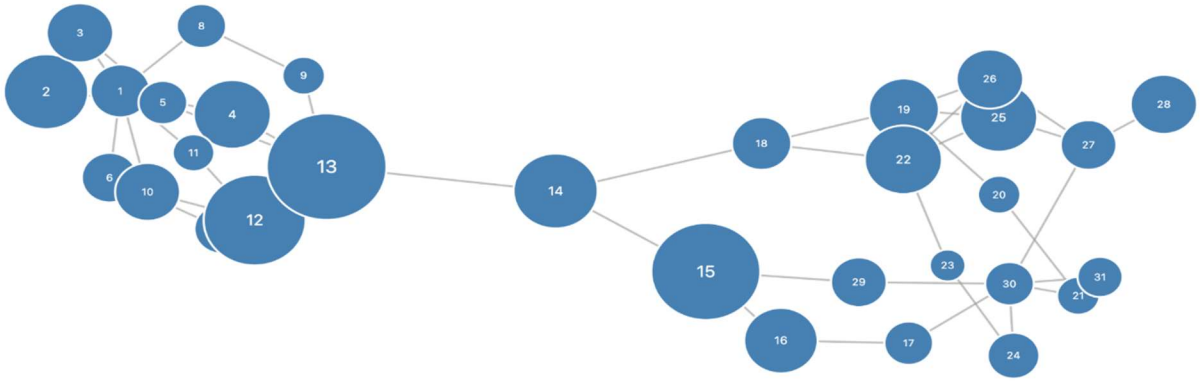
To enhance perceptual understanding and interpretation of workflow bottlenecks and workload, the size of each node conveys the estimated labor time required by that step: the chart actively adjusts the circle radii according to their respective values. I integrated node resizing to encode labor time directly in the precedence graph, prioritizing perceptual clarity and resilience to missing data.

Each node's radius is derived from a linear scale that maps the observed labor-time domain to a readable 14–56px range; otherwise, we could not distinguish the impact of the sizes of nodes. This makes small differences visible without producing a tiny or unselectable target. To avoid breakdown, we first attempt the CSV-backed state.taskData and gracefully fall back to a constant table of PERT labor estimates; this guarantees consistent sizing even when the data is incomplete.

We also couple label legibility to node size by scaling the font (≈ 0.33 × radius, clamped 10–18px) so identifiers remain readable at all radii. Finally, the behavior is wired in non-invasively: a one-shot sizing pass runs whenever the precedence tab is shown and after D3 mutations (via requestAnimationFrame + a debounced MutationObserver), keeping performance overhead low while ensuring nodes stay correctly scaled as the chart renders or updates.

This means that when you switch to the precedence tab, the chart first draws the nodes and links, then asynchronously adjusts their sizes based on the labor-time data once the DOM has stabilized. The auto-sizing routine listens for structural changes in the SVG container—such as new nodes being appended—then recalculates node radii and text scales in a single, smooth pass. This ensures that even if the data or chart is reloaded dynamically, the sizing logic remains synchronized without need of refreshes or redundant computation.

My resizing approach first surveys all nodes, calculates the minimum and maximum labor times, then scales each circle's radius proportionally. Label text inside the nodes is also scaled, ensuring numbers remain legible whether the step is labor-intensive or brief. Under the hood, a MutationObserver attentively listens for changes in the chart, running the resizing after any node updates, so the graph always reflects the true workload encoded in the data.This entire process is seamless for the user, helping them quickly gauge which tasks dominate the workflow by sight alone.



Precedence Violation Highlights

Date: *October 3rd, 2025 {Joel}*

Iteration 1: Highlighting Logic for Precedence Violations

The most complex part of the conversation began here. The user requested that when a precedence rule is broken (i.e., a task is placed before its required predecessor), the visualization should highlight the problematic path.

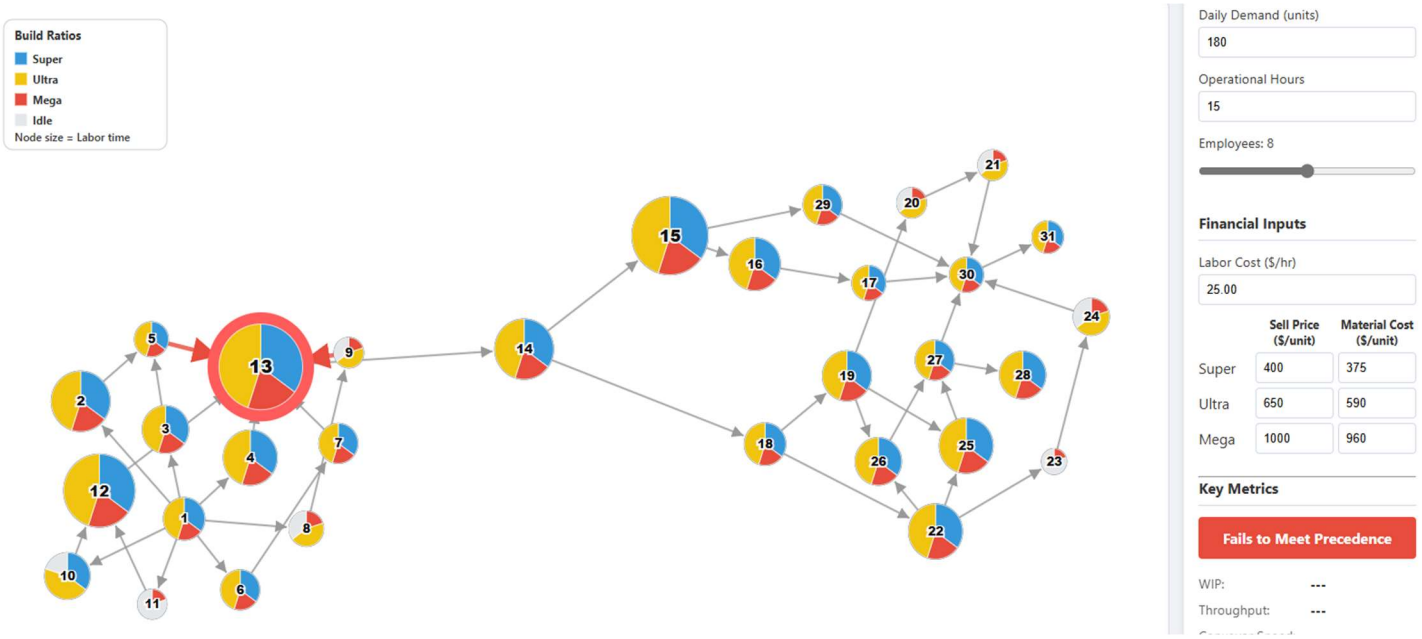
- **Problem - Attempt 1 (Overly Broad Highlighting)**: The first attempt at a solution highlighted the invalid node and *all* of its predecessors, regardless of whether their specific paths were broken. This was logically incorrect, as it failed to isolate the actual violation.
- **Problem - Attempt 2 (Logic and Padding Issues)**: The user clarified that only the predecessors that were physically placed *after* the dependent task should be highlighted. My next attempt at the logic was still flawed and continued to highlight the entire ancestor tree. During this phase, I also introduced a "collision force" to add padding between nodes, which was a successful secondary modification.
- **Problem - Attempt 3 (Catastrophic Failure)**: The third attempt to fix the highlighting was a complete failure. It not only failed to correct the logic but also introduced a syntax error into the code, causing the entire application to crash and display a blank screen. This represented a low point in the iterative process, where the attempted "fix" made the problem significantly worse.
- **Problem - Attempt 4 (Overly Simplistic Logic)**: After the crash, my next attempt over-corrected by simplifying the logic too much. It only highlighted the *single, direct edge* of a violation, failing to recognize that a violation can be a chain of several misplaced predecessors. The user correctly pointed out, "Predecessors can have predecessors."

Iteration 2: Successful Pathfinding

The breakthrough came from combining the correct data structure with a proper pathfinding algorithm.

- **Effective Prompting**: the user prompted for a proper tree traversal algorithm to properly map all possible predecessors and successors for an element, to then identify when precedence was broken.
- **The Solution**:
    1. **Correct Data**: The application was modified to build a map of not only predecessors but also **direct successors** for each node.
    2. **Pathfinding**: When a violation is detected, a **Breadth-First Search (BFS)** algorithm is initiated. It starts from the out-of-place predecessor and searches forward through its successors until it finds a path that connects to the invalid dependent task.
    3. **Precise Highlighting**: All nodes and edges that exist on this specific, algorithmically-verified path are added to a "highlight set." This ensures that only the broken chain is colored red, fulfilling the user's requirement perfectly.

This final step succeeded because it stopped trying to guess or use incomplete logic and instead implemented a classic computer science algorithm (BFS) to definitively find the broken dependency chain.



Efficiency UI Improvements

Date: *October 3rd, 2025 {Dhruv}*

Overview and Motivation

This session focused on improving the clarity and professionalism of the **Efficiency visualization tab** in the FactoryFlow application, alongside resolving a rendering bug in the **Precedence chart**. The Efficiency tab lacked clear labeling, consistent design, and immediate data visibility, while the Precedence chart was being duplicated under certain conditions. The goal was to ensure both tabs present information cleanly, consistently, and without distraction.

Related Work

These enhancements were guided by operational dashboard design principles, emphasizing clarity, consistency, and minimal cognitive load. Inspiration came from professional performance-monitoring interfaces where direct readability and clean design are prioritized.

Questions

- How can we make workstation-level efficiency more interpretable at a glance?
- How can we bring consistency and polish to the summary section?
- How do we prevent duplicate rendering in the Precedence chart while still ensuring it draws when needed?

Data

The underlying PERT and CONFIGS CSV data remained unchanged. Work focused on visual encoding of efficiency and on ensuring correctness in how precedence relationships were displayed.

Exploratory Data Analysis

Testing the interface revealed:

- Efficiency percentages weren’t visible unless hovering, which slowed interpretation.
- Clock visuals were incomplete and less intuitive without full hour markers.

- The summary section lacked cohesion with workstation views.
- The Precedence chart sometimes drew **two graphs** at once, cluttering the visualization and impacting performance.

Design Evolution

1. **Workstation Labels**
    - Added “Workstation #” headings with bold, centered typography for clarity.
2. **Pie Chart Value Display**
    - Efficiency percentages now drawn directly inside pie charts.
    - White circular backgrounds behind text ensure readability.
3. **Clock Visualizations**
    - Full 12-hour tick marks added, with thicker major ticks for hierarchy.
    - Clock hand corrected to rotate clockwise from 12 o’clock to represent idle time.
    - Added center dots and retained idle-time labels.
4. **Summary Section**
    - Dashed border layout applied for separation.
    - Summary title repositioned for professional alignment.
    - Pie and clock shifted slightly for alignment consistency.
5. **User Experience Refinements**
    - Removed tooltips, since values are now always visible.
    - Applied precise pixel-level adjustments for alignment and readability.
    - Maintained consistent color scheme: green for work, red for idle.
6. **Precedence Chart Resolution**
    - Problem: Two charts were being created — once on startup (renderActiveTab) and again on tab click.
    - Resolution: Introduced a precedenceChartDrawn flag so the chart is only drawn once unless explicitly cleared.
    - Result: Only a single precedence chart is now rendered at a time, eliminating duplication and improving performance.

Implementation

- Used proper D3.js enter–update–exit patterns for workstation efficiency charts.
- Applied consistent styling across workstation and summary sections.
- Adjusted tab event logic to control when the Precedence chart is rendered, ensuring only one instance exists.

Evaluation

- Efficiency values are now immediately accessible without interaction.
- Workstation and summary sections present a professional, aligned appearance.
- Users can interpret idle time intuitively thanks to improved clock visuals.
- Duplicate Precedence chart bug resolved, improving clarity and system responsiveness.

Future improvements could include subtle trend indicators in the efficiency charts and user-selectable focus for specific workstations.

Session Outcomes

This session delivered a significant polish upgrade to the Efficiency tab, making workstation and line-level performance both clear and professional. The Precedence chart bug fix removed duplication and improved stability. Together, these refinements advanced FactoryFlow into a more robust and credible decision-support tool.

Animation Speed Sliders and Operational Hours Adjustment

Date: *October 4th, 2025 {Joel}*

The Problem: Creating a Functional Vertical Slider
The initial request was to add a slider to control the speed of an animation. This seemingly simple feature spiraled into a multi-step debugging and refinement process.
Iteration 1: The CSS Approach

- **Attempt:** My first few attempts relied on various CSS properties (-webkit-appearance: slider-vertical, writing-mode, transform) to reorient a standard HTML <input type="range"> element.
- **Problem:** This approach proved unreliable. Different CSS properties either failed to render the slider vertically at all or created a visual that was vertical but whose interaction logic remained horizontal. The slider was often stuck, unmovable, or visually broken.
- **The Breakthrough:** The turning point came when you explicitly instructed me to abandon CSS and **use the D3 library itself** to build the slider from scratch.
- **Initial Solution:** The user's diagnosis correctly identified the problem as a coordinate space issue in D3.js. The solution proposed using d3.pointer(event, this.parentNode) to get the mouse coordinates relative to the slider's local SVG group.

Iteration 2: The D3-Native Approach

- **Solution:** I created a "slider" using SVG elements (<line> for the track, <circle> for the handle) within the D3-controlled <svg> container. This gave me direct, programmatic control over the slider's appearance and behavior.
- **Challenge:** The initial D3 implementation had a critical flaw in its drag logic. It failed to correctly account for the SVG's coordinate system transformations. The code was reading the mouse's global page coordinates instead of its position *relative to the slider's group element*. This is a classic problem in complex D3 visualizations.
- **User Feedback:** Your prompt, "make sure you account for coordinate changes of the transforms," was the key insight. It correctly diagnosed the root cause of the bug, that coordinates were not being calculated correctly.

Iteration 3: Refining the D3 Slider

- **Solution:** The final, successful implementation fixed the coordinate problem by using d3.pointer(event, referenceNode). This D3 function is specifically designed to solve this issue by translating global event coordinates into the local coordinate system of a specified SVG element. Once this was implemented, the drag logic worked perfectly.
- **Enhancements:** Based on your feedback, I also added mousewheel support and a larger, invisible interaction area (<rect>) to make the slider easier to grab and control, improving the user experience.
- **CSS vs. Programmatic Control:** While CSS is powerful, for complex, interactive elements within a D3 visualization, building the component natively in D3 often provides more reliable control over behavior and interaction, avoiding cross-browser CSS quirks.
- **Iterative Debugging:** The conversation perfectly illustrates the "try, fail, refine" loop. Each failure, guided by your specific feedback, exposed a deeper layer of the problem, leading from a simple CSS issue to a more complex D3 coordinate system challenge, and finally, to a robust solution.
- **Challenge:** The initial D3 implementation had a critical flaw in its drag logic. It failed to correctly account for the SVG's coordinate system transformations. The code was reading the mouse's global page coordinates instead of its position *relative to the slider's group element*. This is a classic problem in complex D3 visualizations.
- **Final Solution:** The drag handler was completely rewritten to use a proper two-step process: first, convert the mouse's pixel position into a data value using speedScale.invert(my), and then use that clamped value to set the handle's new pixel position with speedScale(newValue). This robust logic solved the problem completely.

Iteration 4: Redefining "Operational Hours"

- **Problem:** The core business logic was flawed. "Operational Hours" was being used as a simple window to launch models, but it needed to represent the *first model's start* to the *last model's finish*.
- **Iterative Solutions:** This was the most challenging part of our conversation, requiring several iterations to get right.
    1. **Attempt 1:** The first attempt subtracted the time to build one model from the total operational hours. This was a step in the right direction but didn't fully account for the conveyor.
    2. **Attempt 2:** The user correctly pointed out that **throughput time** was the missing ingredient. The logic was then refined to create a formula that solved for the required TaktTime algebraically, balancing the launch intervals with the time it takes the last unit to travel the full ASSEMBLY_LINE_LENGTH. This was a major breakthrough.
    3. **Attempt 3:** The user provided crucial clarification: financial metrics should be based on the full paid hours, while assembly line behaviors should use the adjusted production window. The code was refactored to explicitly separate these two phases, making the logic clearer and more accurate. Showing the difference between work should be started and when work is being done.
- **Final Solution:** The final version of the calculateMetrics function correctly used the sophisticated Takt Time formula for all physical calculations (like conveyor speed) while using the full operational hours for all financial and efficiency reports. This multi-step refinement, guided by precise user feedback, was a perfect example of iterative development. This reduced the maximum daily demand.
- **Effective Prompts:** Your most effective prompts were those that clearly described the **expected behavior** versus the **observed behavior** ("The schedule... finishes its first unit in 30 minutes, and all 24 in 1 hour 29 minutes" was a perfect example). This allowed for a diagnosis of the logical flaw.

Iteration 5: The Sidebar Alignment Glitch

- **Problem:** A visual glitch appeared where changing inputs (like employee count or operational hours) while the left sidebar was scrolled down would cause the "Schedule" tab's Gantt chart to be drawn at an incorrect vertical position.
- **Failed Solutions:**
    1. Wrapping the drawing logic in requestAnimationFrame was tried first. This is a common solution for timing issues but wasn't robust enough on its own.
    2. Switching to using the offsetTop property was the next attempt. This was a good idea in principle but was implemented incorrectly, breaking the alignment entirely.
- **Effective Prompting:** The user's feedback here was critical. By stating that the last fix "decoupled the schedule tab from the left-side bar" and “to reference the baseline to re-evaluate the coordinates for this coupling” it provided a clear signal that the measurement logic itself was now the problem.
- **Final Solution:** The root cause was identified as a **race condition**. The sidebar was redrawing itself at the same time the schedule was trying to measure it. The final, successful fix involved:
    1. **Centralizing Control:** The main updateUI function was refactored to control the drawing order.
    2. **Resetting State:** Forcing the sidebar's scroll position to 0 on every relevant input change.
    3. **Deferring Drawing:** Using requestAnimationFrame to delay the call to drawScheduleVisualization until *after* the browser had applied the scroll reset. This guaranteed that all measurements were taken from a stable and correct layout.

PERT with Pie Charts

Date: *October 4-5th, 2025 {Sumaiya}*

In designing the PERT chart visualization, I wanted each node to convey not just which task it represents, but also how that task distributes its effort across multiple product models and whether any idle time occurs. To achieve this, I embedded a responsive pie chart within each node; each wedge visualizes the proportion of the node’s labor time spent on “super,” “ultra,” and “mega” models, while a separate slice surfaces any idle/unassigned time if present. I sourced these proportions directly from live CSV data for accuracy, but if data was missing, the system falls back to a hardcoded table so the graph always renders fully. I selected a calm, high-contrast color palette—blue for super, sand for ultra, coral for mega, and teal for idle—to ensure slices remained readable without visually dominating the network. Slices with zero value are excluded, avoiding misleading slivers.

For model-mix encoding on PERT nodes, I embedded a tiny pie chart inside each node and found this worked well for at-a-glance comparison: a restrained, high-contrast palette (super/#4E79A7, ultra/#f2d12bff, mega/#E15759, idle/#76B7B2) plus a thin white slice stroke kept wedges readable without overpowering the graph. Zero-value slices are filtered out to avoid misleading slivers, using CSV data but gracefully falling back to a hardcoded table so nodes never render “empty.” For each node, I multiplied elementTime by each model’s share to get slice magnitudes. The Idle slice is elementTime × max(0, 1 – (super + ultra + mega)). Zero-value slices are dropped so you don’t get hairline slivers. I made the labels scale with node size and sit above a subtle circular background so the text stays readable even when wedges are busy.

To avoid clutter, I opted not to force minimum wedge sizes and only permitted the base circle of the node to accept pointer events, for smooth dragging. All pies, label backgrounds, and label text are set to “pointer-events: none” in D3, so interactions and drag remain fluid. The node’s base circle is the only hit target for pointer events (it has a near-transparent fill so you can click/drag). All overlays—pie paths, label background, text—ignore pointer events, so nothing interferes with dragging.

For detail-on-demand, I implemented a dark, HTML-styled tooltip: hovering any node instantly reveals its element time, model shares as percentages, and idle share if present, with color swatches for each slice. This keeps the chart itself clear of heavy text yet gives instant access to the underlying data.

A legend is included at the top, with color keys for each wedge type and a note explaining that “node size = labor time,” so users immediately understand the visual encoding. To ensure nodes always display accurate radii and pies, I use a linear scale to map actual labor time or fallback values to a readable node size (typically 14–56px diameter).

The sizing and pie rendering automatically re-runs whenever the tab or DOM changes, combining a requestAnimationFrame pass with a MutationObserver debounce—this ensures performance remains snappy and drag handles are never lost, since nodes mutate in place.

I removed forced minimum wedge sizes (which biased perception) and disabled pointer events on pies/labels to preserve smooth node dragging. Drag is handled by the D3 drag on the node group, with the base circle set as the single hit target (fill-opacity: 0.001; pointer-events: all), while overlays are pointer-events: none. I mutate nodes in place and re-run a lightweight "size and pie" pass after tab changes or DOM mutations so drag handles stay intact and performance remains snappy.

The chart shows -

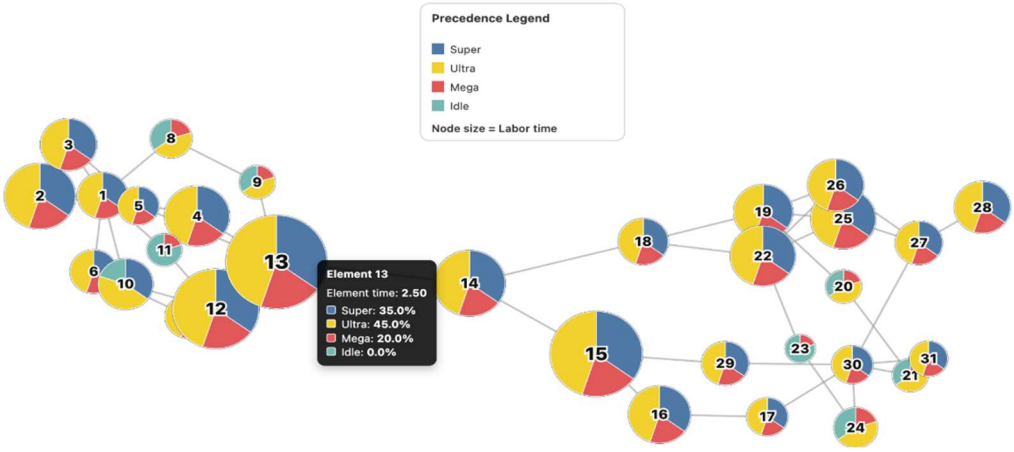
- Precedence/DAG view: each node is an element; links encode "must finish before" constraints. The layout is force-directed so the graph finds a readable equilibrium, and nodes remain draggable to resolve overlaps or emphasize relationships you care about.

- Size = labor time**:** Each node's *radius* scales with its labor time. That immediately surfaces "heavy" tasks and potential bottlenecks: larger node ⇒ more labor per unit.

- Color wedges = model time within element time**:** Inside each node is a small pie breaking down how that element's *element time* is spent across the three models (Super/Ultra/Mega). The pie is proportional in time, not counts—so you read how much of this task's processing time belongs to each model.

- Idle wedge (when present): If Super + Ultra + Mega don't sum to 100% of the element's time, the remainder is visualized as an Idle slice. This exposes underutilization or model gaps right on the node

I improved the chart's clarity and readability by introducing a tiered, force-directed hierarchy. Now, each node's vertical coordinate is set by its critical path depth, ensuring a clear top-to-bottom flow of dependencies. Arrow-headed curved links dynamically update as nodes are dragged, with opacity and color encoding to distinguish primary and secondary dependencies. This means arrows and nodes move together, maintaining semantic coherence, and a hover tooltip offers relevant details. This compact legend calls out the four wedge colors and reminds: "Node size = Labor time." It's anchored in the SVG (center-top) and repositions on window resize so it's always visible and out of the way. With adaptive radius sizing and embedded pies, the graph now becomes an interactive workflow explorer, showing both process structure and data at a glance, not just a static network.



The chart renders first, then a fast "size and pie" pass runs right after the DOM paints (via requestAnimationFrame) and also after any D3 mutations (debounced MutationObserver). That ensures nodes get the correct radius and pie wedges without flicker, and it avoids re-binding drag handlers—so performance stays snappy.

Improved Schedule Timing Visualization

Date: *October 5th, 2025 {Dhruv}*

Overview and Motivation

This session focused on a major refinement of the **Schedule** and **Efficiency** tabs in the Factory Flow to Fortune application. The primary goal was to strengthen the connection between production timing, workstation performance, and operational efficiency through improved visual hierarchy, interactivity, and analytical

feedback.
Previously, the schedule visualization effectively showed workstation activity but lacked contextual overlays, labels, and controls to make it actionable. The Efficiency tab also benefited from alignment and consistency improvements to ensure that performance metrics presented in both tabs were coherent and visually unified.

Related Work

The redesign was informed by production management dashboards and professional manufacturing analytics tools, where clarity and quick pattern recognition are essential. Interfaces such as process monitoring systems and Gantt-based scheduling software inspired the addition of overlays, metric labels, and interaction controls for real-time exploration.

Questions

- How can bottlenecks and efficiency trends be communicated visually and intuitively?
- What layout and interaction elements make it easier for users to navigate the schedule timeline?
- How can the Efficiency tab be harmonized with the Schedule tab to ensure consistent communication of performance data?

Data

The same manufacturing configuration and task data (PERT and CONFIGS CSVs) were used. No changes were made to the data pipeline — instead, this session focused on how existing metrics like efficiency, idle time, and bottleneck detection were surfaced through interactive visualization.

Exploratory Data Analysis

During testing, users found it difficult to identify which workstation represented the production bottleneck or how overall efficiency correlated with time. The timeline lacked labeling and scale cues, making navigation challenging. Additionally, visual inconsistencies between the Schedule and Efficiency tabs caused confusion. These observations motivated the addition of labeled separators, a visible time grid, and synchronized efficiency overlays that work dynamically during simulation playback.

Design Evolution

1. **Workstation Labeling and Layout**
    - Introduced “WS #” labels alongside each row for immediate identification.
    - Added subtle separator lines for visual distinction between workstations.
    - Applied consistent typography and spacing to match the overall dashboard theme.
2. **Temporal Orientation**
    - Implemented a vertical dashed time grid and bottom-aligned axis labeled in hours and minutes.
    - Added rounded-corner task bars and refined stroke styling for a cleaner hierarchy.
3. **Interactive Controls**
    - Introduced tooltips displaying detailed task metadata, including start/end times and durations.
    - Enabled timeline scrubbing by clicking anywhere on the chart.
    - Added zoom and speed controls, allowing playback adjustment between 0.1x and 8x.

    - Repositioned controls to the bottom of the visualization for ergonomic balance and improved use of space.

4. **Performance Metric Overlays**
    - Highlighted the bottleneck workstation row with a red background for immediate visibility.
    - Added efficiency labels and idle time indicators next to each row.
    - Placed a red dashed vertical line at the bottleneck cycle time, labeled for clarity.
    - Ensured that all overlays update in real time during simulation playback, preserving analytical accuracy.
5. **Efficiency Tab Integration**
    - Linked efficiency and idle time calculations directly to those used in the schedule visualization.
    - Ensured that both tabs share consistent color schemes, typography, and layout structure.
    - The efficiency tab retained pie charts, clock visualizations, and summary metrics while aligning styling with the Schedule tab's refined design language.

Implementation

- All updates were implemented within script.js.
- Core modifications were made in drawScheduleVisualization() to integrate overlays, axis, and interactive controls.
- Efficiency and idle time calculations were unified through calculateMetrics().
- The animation loop was updated to maintain dynamic position synchronization for overlays, ensuring that indicators move consistently with timeline progress.
- Robustness was improved with targeted error handling and defensive code placement around metrics updates.

Challenges and Solutions

- **Static Overlay Positioning:** Early overlays remained fixed when the timeline moved. This was solved by updating their x-positions dynamically within the animation loop, based on the current domain of the x-scale.
- **Code Placement Conflicts:** Initial attempts to inject metrics updates caused timing errors. Moving these blocks after the x-scale initialization resolved sequencing issues.
- **Workstation Mapping Inconsistencies:** DOM elements and data arrays occasionally mismatched. Implementing flexible ID-matching logic with naming fallbacks ensured reliable alignment.

Evaluation

- **Improved Clarity:** Bottleneck workstations and inefficiencies are now immediately visible through red highlights and labeled markers.
- **Enhanced Navigation:** Zoom, scrub, and playback controls give users direct manipulation of the timeline.

- **Analytical Depth:** Real-time overlays combine time-based scheduling with performance metrics, offering a richer analytical perspective.
- **Professional Consistency:** The shared color and typography scheme across Schedule and Efficiency tabs reinforces brand and design unity.

Session Outcomes

This session transformed the Schedule tab from a static Gantt visualization into a dynamic analytical dashboard. Users can now explore line performance interactively, identify bottlenecks, and interpret efficiency trends in real time.
The Efficiency tab's consistency improvements and shared data structures strengthen the overall cohesion of the Factory Flow to Fortune interface. Together, these enhancements deliver a more polished, insightful, and user-centered platform for production optimization.

Investment Tab Creation

Date: *October 5th, 2025 {Joel}*

Iteration 1: The Standalone Feature

- **Request:** The user provided their script.js file and a detailed PDF outlining the logic for a new "Investment" analysis tab. The goal was to add this functionality to their existing application.
- **Initial Solution:** The first code provided was a comprehensive, self-contained JavaScript file. It successfully implemented all the complex financial logic from the document, including Monte Carlo simulations, cash flow analysis, and data visualization.
- **Effective User Feedback:** The user provided a crucial piece of missing context: the index.html file. They clarified that the new code must function *within* the existing structure, use existing variables where possible, and place new inputs in a dedicated side panel. This prompt was highly effective because it supplied the necessary architectural blueprint.
- **Second Solution:** Armed with the HTML structure, the next attempt was to rewrite the script to integrate deeply into the application's framework. This version tried to create a unified state management system, modify the core tab-switching logic, and manage multiple sidebars.
- **Problem:** This approach proved to be too invasive. In trying to make everything work together seamlessly, the new code inadvertently disrupted the original application's functionality. The user reported that "none of the tabs or sidebars are being properly loaded" and that the page was "warped." The attempt to create a single, elegant solution broke the existing, working parts.

Iteration 2: The Minimally Invasive Success

- **Highly Effective User Feedback:** The user provided clarity that the input panel should appear *inside* the tab's content area rather than as a separate sidebar. This was the most critical piece of feedback, as it established a clear boundary: do not touch the existing logic.
- **Final Solution:** This final attempt adopted a different strategy based on the user's explicit constraint.
    - **Isolated JavaScript:** The script.js file was revised to be non-invasive. It left all the original application's code untouched. The new investment logic was encapsulated in a function (renderInvestmentUI) that only triggers when the "Investment" tab is clicked. This function dynamically builds the entire UI for the tab—including the results dashboard and the internal input panel—inside the designated <g id="investment-panel"> element.

This final version was successful because it treated the new feature as a self-contained module, respecting the integrity of the existing application and perfectly matching the user's mental model of how the integration should

work. The process demonstrates a classic development cycle: moving from a broad concept to a broken integration, and finally to a refined, working solution guided by specific, constraint-based feedback.

Iteration 3: Core Calculation and Logic Challenges

The most significant challenges involved the accuracy of the financial calculations and the underlying logic.

- **Problem 1: IRR and Payback Period Errors**: A persistent issue was that the **Internal Rate of Return (IRR)** and **Payback Period** metrics consistently failed to populate. The user repeatedly pointed out that these key outputs were showing "N/A" or "0".
    - **Solution 1**: The AI's first attempt to fix this involved replacing an unstable Newton-Raphson algorithm for the IRR with a more robust bisection method. However, the root cause was deeper. The user's final, clear prompt—stating that the investment costs were not being considered correctly—led to the breakthrough. The AI corrected the logic to ensure that the initial capital investment was properly applied as a negative cash flow in Year 0 for *both* the "Base Case" (the full cost of building the current line) and the "Expansion Case" (the incremental cost to upgrade). This fix successfully resolved the long-standing calculation errors.
- **Problem 2: Unclear Scenario Logic**: The initial distinction between the "Base Case" and "Expansion Case" was ambiguous.
    - **Solution 2**: The user provided a particularly effective prompt clarifying the desired logic: the **Base Case** should analyze the profitability of the *currently configured line* against a new demand forecast (including lost sales), while the **Expansion Case** should calculate the optimal profitability of meeting *full demand*, determining the optimal configuration and the incremental investment required. This clarification was crucial for making the tool strategically useful.
- **Initial Report:** The user correctly noted that the Profit tab was showing configurations as "optimal" even when they failed to meet the requested demand (e.g., showing a profit for 507 units when the configuration could only produce 499).
    - **Hypothesis 1 (Rounding Errors):** The first attempt to fix this involved adding a small tolerance (an "epsilon") to the meetsDemand check, assuming floating-point rounding errors were the cause. This did not solve the problem.
    - **Hypothesis 2 (Logical Inconsistency - The Turning Point):** The user provided a rhetorical but highly effective prompt: *"Why is the check for of the combination valid not aligned with the Meets Demands button?"* This was a crucial insight. It forced a re-evaluation of the high-level logic, revealing that the "Meets Demand" status in the UI and the optimal profit calculation were using two different values for **operational hours**, leading to a direct conflict.
    - **Hypothesis 3 (The Root Cause):** After an attempt to align the opHours inputs still failed to resolve the issue, a deeper investigation into the calculateMetrics function revealed the true culprit. The function's internal physics model was flawed. It was inconsistently using the *fastest* station time for some calculations while using the *bottleneck* time for others. This created a mathematical inconsistency in the core formula for determining throughputUnitsPerDay.
    - **Final Solution:** The calculateMetrics function was completely refactored. The old, inconsistent logic was replaced with a single, direct formula that correctly models the total production time from the first unit's start to the last unit's finish, based consistently on the **bottleneck cycle time**. This definitive fix ensured that the meetsDemand flag and the throughputUnitsPerDay calculation were always aligned, finally resolving the bug.

Iteration 4: Deepening Detail and Final Polish

Later iterations focused on adding detail and refining the user interface based on specific requests.

- **Problem**: The single "Equipment Cost" input was too simplistic.
    - **Solution**: The user requested a breakdown into cost per foot of straights, cost per bend, and installation cost. They also specified that the number of bends should be calculated dynamically. The AI successfully implemented these new inputs and the corresponding logic.
- **Problem**: The UI needed better organization and at-a-glance information.
    - **Solution**: Following user prompts, the AI combined the "Project Parameters" and "Fixed Costs" sections and added dynamic, red subtitles to display total costs for each category, improving readability.
- **Demand Model Simplification:** The initial complex B2B sales funnel was replaced with a more direct P90/P50/P10 annual demand input, which was automatically synced with the main "Daily Demand".
    - This was determined to be overly complicating the UI and limiting the dynamic responsiveness of the tab, due to requiring a half-dozen contractual variables which are unlikely to make sense to anyone who is not directly in sales/marketing. They were also forcing a Monte-Carlo simulation just to populate the demand levels for P10 and P90.
- **Automation and UI Refinements:** The "Run Analysis" button was removed in favor of automatic, debounced calculations. The UI was iteratively improved by reformatting inputs with commas, adjusting panel and input box widths, and reorganizing the layout of the results panel.
- **Visualization Enhancements:** The cash flow chart was improved with shaded uncertainty areas, axis labels, and interactive tooltips that replaced a static data table. A sales scenario funnel chart was also added to visually represent the demand projections.

Optimizing Investment Tab Calculations

Date: *October 6th, 2025 {Joel}*

Initial Problem: Invalid Profit Optimization

The core issue was that the "Profit" tab was suggesting "optimal" configurations that were logically impossible. For example, it would propose an 11-workstation setup for a demand of 505 units, even though that configuration's maximum possible output was known to be 499 units. The initial hypothesis was that this was caused by rounding errors in time-based calculations.

Iteration 1: Addressing Rounding Errors

- **Problem**: The calculation for total operational time didn't align with the requirement for discrete 15-minute intervals, and direct floating-point comparisons were failing otherwise valid configurations.
- **Solution**: We first corrected the time calculation to ensure it was always rounded down to the nearest 15-minute block. When that wasn't enough, we introduced a small tolerance (**epsilon**) to the comparison logic to account for microscopic rounding errors.
- **Outcome**: While these changes improved accuracy, they didn't solve the fundamental problem, proving the issue was deeper than simple rounding.

Iteration 2: The Diagnostic Breakthrough & Final Fix

- **Challenge Encountered**: The bug was incredibly persistent. Even with optimized loops and a cleared cache, the fundamental association between a configuration and its maximum demand was being calculated incorrectly.
- **Effective Prompting**: The turning point was your frustration and the realization that the complex algorithms were failing. My suggestion to create a simple, standalone **diagnostic function** (findBestConfigForDemand) to test a single demand point was the key.
- **Solution**: By running findBestConfigForDemand(505), we received a correct result (13 employees). This proved that the core calculateMetrics function was accurate and that the bug was isolated entirely within the complex, multi-loop structure of calculateOptimalProfitData. Armed with this "ground truth," we rewrote the main function using the same simple, brute-force logic from the diagnostic test. This finally solved the core problem.

Iteration 3: UI Refinement

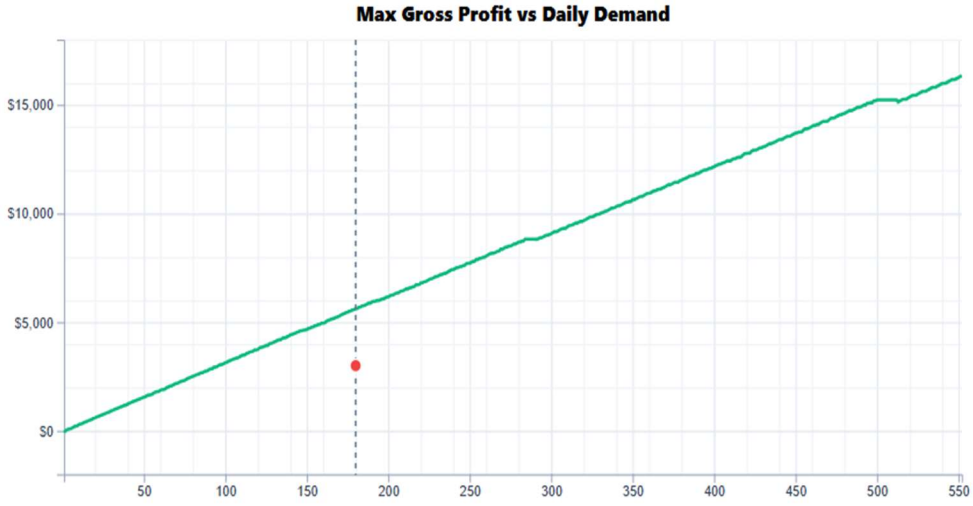
- **Problem**: With the logic finally correct, previous UI features—like tooltips and visual markers for the current scenario—had been lost during the extensive debugging.
- **Solution**: In the final phase, we carefully re-integrated the desired UI features. This included:
    - Drawing vertical lines from the axis to the data curve.
    - Logic was added to calculate the x and y coordinates for the "requested demand" (black line) and "actual throughput" (red line).
    - D3's d3.area() generator was used to create the red shaded region for unmet demand.
    - Separate tooltip div elements were created for each chart (profit-tooltip-profit and profit-tooltip-margin). Implementing independent, three-line tooltips for each chart to show the specific profit/margin and the configuration that achieved it.
    - The D3 mousemove event handler was updated to determine which chart the cursor was over and display the corresponding tooltip with the custom, four-line HTML content. The redundant crosshair text elements (badges) were removed.

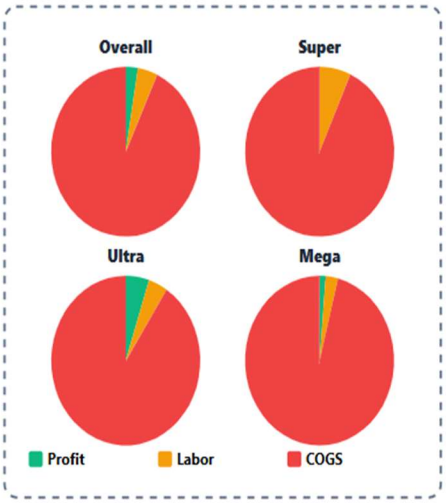
Iteration 4: Fixing Negative Value Handling in Charts

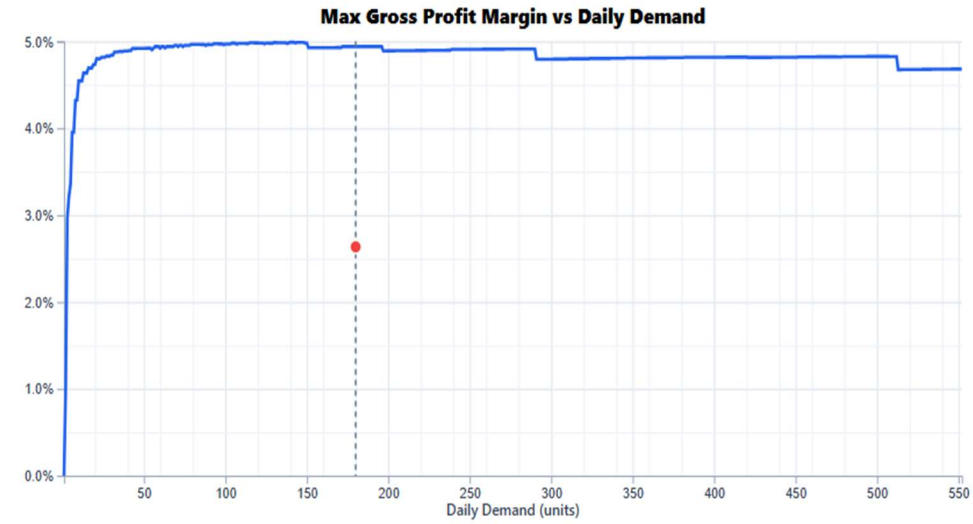
The development began with charts in the "Profit" tab not handling negative values correctly.

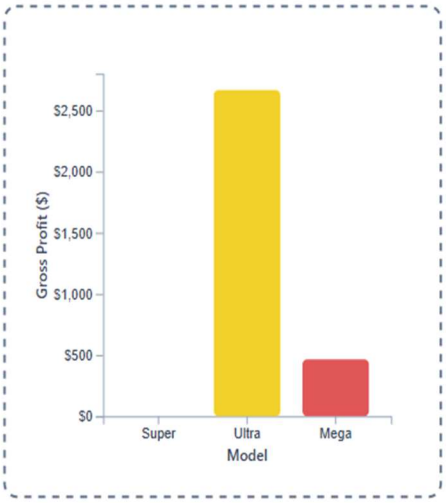
- **Problem 1: Initial Bug**: Negative profits in the bar chart caused rendering errors because the y-axis was fixed to start at zero. Pie charts were also misleadingly clamping losses to zero.

    - **Solution 1**: The code was modified to allow the bar chart's y-axis to dynamically adjust to negative domains. The pie chart logic was updated to display a "Loss" slice when profit was negative, providing a more accurate financial picture.
- **Problem 2: Regression and Scope Creep**: In fixing the first issue, a new one was introduced. The vertical bars indicating the gap between optimal and current performance were misaligned for negative values. The user also noted that the profit margin chart didn't account for the *current* configuration's performance, causing it to go out of bounds.
    - **Solution 2**: The logic for the vertical bars was corrected to anchor them properly regardless of the axis domain. The chart's y-axis domain calculation was expanded to always include the current configuration's performance, ensuring it never went off-chart.









Iteration 5: Optimizing Calculation Speed

- **Problem**: The user asked to evaluate the calculateOptimalProfitData function for efficiency improvements. The existing function used a brute-force method, checking every single combination of employees and operating hours, which was slow.
- **Solution**: The function was re-engineered to be significantly more efficient. The user provided a structure of non-feasible demand limits based on workstations to mitigate unnecessary loops, as well as providing the critical detail that once a valid hour combination is found, higher hours will always be less profitable. Instead of checking all hours, it first calculates the **theoretical minimum hours** needed for a given demand and configuration. It then starts its search from that point and uses a break; statement as soon

as a valid, profitable configuration is found. This was a key insight, as increasing hours beyond the minimum required only increases costs and reduces profit.

Iteration 6: Enhancing the Investment Tab

The final phase focused on improving the "Investment" tab's functionality and user interface.

- **Problem: Interconnected Statistical Inputs**: The user requested a more sophisticated way to model demand scenarios, asking for STD, CV, and CI fields that were all mathematically linked to the P10/P50/P90 values based on a Gaussian distribution. As a replacement for the previously computationally intense and abstract B2B funnel.
- **Challenge & Solution**: This was a complex logic request. The solution involved:
    1. Writing an updateProbabilisticValues function that could recalculate all statistical values based on which one was just changed by the user (the "driver").
    2. Using a switch statement to handle the different calculation paths (e.g., if CV is changed, calculate STD; if P90 is changed, back-calculate STD).
    3. Making the P50 (mean) value a read-only display, as it's directly driven by the main dailyDemand input. The user later refined this to be a plain text display.

Implementing Consistent Color Theme

Date: *October 6th, 2025 {Dhruv}*

Overview and Motivation

This session introduced a unified color system and a complete expansion of the Schedule tab in the Factory Flow to Fortune application. The primary objectives were to enhance the application's visual consistency and to make the scheduling visualization more informative, interpretable, and interactive. By applying a consistent color palette across all components and introducing new analytical elements to the Schedule tab, the dashboard achieved both greater visual cohesion and functional depth.

Related Work

The redesign was influenced by industrial analytics dashboards and project management systems, such as Gantt-based scheduling interfaces, which emphasize the use of color and layout hierarchy to improve information legibility. The approach also drew inspiration from visual design systems like Material Design, which stress clarity, contrast, and consistency in color usage to guide user attention and improve data interpretation.

Questions

- How can a limited three-color palette effectively distinguish between interactive, structural, and data-driven elements?
- What visual and interactive additions will make the Schedule tab more informative and intuitive to navigate?
- How can efficiency and idle time be communicated in a way that is both immediate and visually integrated into the existing layout?

Data

No changes were made to the underlying manufacturing data sources. The focus of this session was on visual presentation and interactive control of existing scheduling metrics, such as efficiency, idle time, and production throughput.

Exploratory Data Analysis

Testing revealed that, while the Schedule tab effectively displayed timing and workstation activity, it lacked visual clarity and key analytical elements. Users had to rely heavily on tooltips and lacked a concise way to filter or compare data. Additionally, the absence of color consistency across different components created a fragmented user experience. These findings led to the development of a unified color system and the addition of interactive and contextual elements to the scheduling interface.

Design Evolution

1. **Unified Color Scheme Integration**
    - A three-color system was applied across the interface to create a coherent design language:
        - **Teal (#009688):** Used for progress indicators, timeline motion, and active visualization elements.
        - **Amber (#FFB300):** Applied to key controls, timers, and numeric highlights to draw attention to active or changing data.
        - **Blue-Gray (#37474F):** Used for structural components such as headers, labels, and backgrounds, providing neutral contrast and visual balance.
    - These colors were defined as global CSS variables and integrated into both static and dynamic interface elements, ensuring consistency between the sidebars, chart components, and overlays.
2. **Product Counters and Filters**
    - Added individual counters for each product line (Super, Ultra, and Mega), providing real-time updates on completed units.
    - Implemented clickable visibility checkboxes next to each counter, allowing users to toggle the display of corresponding products on the Gantt chart.
    - Active selections are highlighted with teal to indicate which product lines are currently visible.
3. **Critical Path and Total Time Indicators**
    - Added a dynamic Critical Path display showing both total schedule duration and the critical path length.
    - Highlighted in amber and red to draw immediate attention to time-sensitive data.
    - Updated interactively during simulation playback to reflect real-time scheduling conditions.
4. **Efficiency and Idle Time Labels**
    - Introduced per-workstation efficiency and idle time metrics displayed directly alongside each workstation row.
    - Example: “Eff: 85.4% | Idle: 12.3m”
    - The placement of these labels provides contextual insight without requiring hover or secondary actions.
5. **Utilization Bars**
    - Added horizontal utilization bars beneath each workstation label to visualize relative efficiency.

- Bars fill proportionally to the workstation's utilization rate, with teal indicating normal operation and red highlighting bottlenecks.
- These visual indicators make workstation performance immediately recognizable.

6. **Legend and Visual Key**
   - Created a comprehensive legend positioned in the bottom-right corner of the visualization.
   - The legend identifies color encodings for product types (Super, Ultra, Mega), critical paths, bottlenecks, timeline markers, and utilization bars.
   - Its inclusion improves usability and comprehension, especially for new users.
7. **Control Panel and Interface Refinements**
   - Enhanced the playback control panel with consistent styling using the unified color scheme.
   - Controls now include play/pause, reset, critical path toggle, zoom in/out, and filter reset buttons.
   - Visual feedback is provided through color changes that clearly indicate active states.
   - Reorganized layout and spacing improved alignment and made controls easier to access and interpret during simulation.

Implementation

- **Files Updated:** script.js (Schedule tab logic and D3 rendering) and style.css (color variables and layout rules).
- **Key Functions Modified:**
  - drawScheduleVisualization() expanded to include new counters, checkboxes, labels, utilization bars, legend, and critical path indicators.
  - updateTaskVisibility() updated to respond to filter checkbox states and critical path toggles.
- **Technical Adjustments:**
  - Leveraged D3.js data binding for smooth updates and efficient rendering.
  - Ensured new components update dynamically during simulation playback without performance degradation.

Evaluation

- **Visual Consistency:** The new color palette created a unified, professional appearance across all dashboard components.
- **Analytical Clarity:** Additional metrics (efficiency, idle time, utilization) provide actionable insights directly in context.
- **User Control:** Product filters, critical path toggles, and zoom controls improve interactivity and flexibility for analysis.
- **Comprehension:** The legend and color hierarchy make the visualization more self-explanatory and accessible to new users.
- **Performance:** Despite added features, the Schedule tab maintains smooth transitions and responsive playback.

Session Outcomes

This session transformed the Schedule tab into a comprehensive analytical interface that combines data, control, and clarity. The integration of a cohesive color system and the addition of new visualization and control elements created a dashboard that is both aesthetically unified and functionally rich. Users can now interpret production data, identify bottlenecks, and explore schedule efficiency through a visually consistent and intuitive interface.

Efficiency Panel Overview Additions and SVG Spacing

Date: *October 7th, 2025 {Joel}*

Summary of Iterative Development: The Efficiency Panel
Our collaboration transformed the "Efficiency Panel" from a basic concept into a polished, data-rich, and interactive dashboard component. This process perfectly mirrored a real-world agile development cycle, characterized by incremental improvements and responsive feedback loops.

Iteration 1: The Initial Build & Scaffolding

- **Problem:** The initial request was to replace the existing line efficiency summary with a new three-column layout containing three distinct D3.js visualizations: a box-and-whisker plot, the existing pie chart, and a histogram.
    - **Solution:** The first version of the code established this fundamental structure. It was a functional scaffold—the columns were in place and the charts rendered with basic data, but they lacked professional polish. This is a common and effective starting point, prioritizing structure over aesthetics.

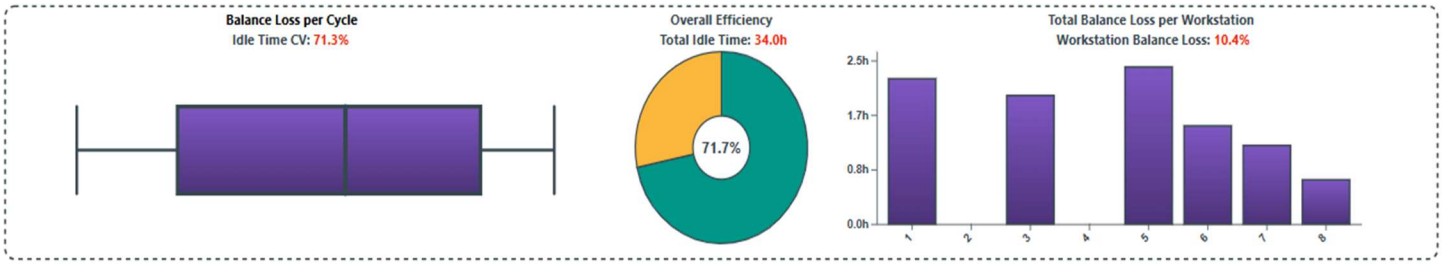
Iteration 2: Iterative Refinement & Problem Solving

This phase involved a series of feedback loops where problems were identified and systematically solved.

1. **Aesthetic and Layout Polish:**
    - **Problem:** The initial charts were described as "bland," "shrunk down," and "poorly aligned." They lacked axes, proper scaling, and visual appeal. The bar chart's axis labels, in particular, were not fitting within their designated space.
    - **Solution:** This was addressed over several iterations.
        - **Axes & Legibility:** D3 axes were added, and their font sizes and rotation were adjusted to improve readability.
        - **Layout & Scaling:** The SVG transform attributes and margin calculations were repeatedly refined to correctly center the charts and make them utilize their available space effectively. The overall summary was expanded by 10% to prevent cramping.
        - **Visual Appeal:** Borders were added to bars, the box plot was redesigned with a modern gradient, thicker lines for emphasis, and a contrasting "jitter" plot to show the underlying data distribution.
2. **Data Accuracy and Logical Corrections:**
    - **Problem:** A critical issue was identified where the visualizations were not displaying the correct data. The bar chart was not using the same idle time calculation as the individual workstation clocks. A deeper, more significant bug was then found: the workstation clocks themselves were calculating "balance delay" (idle time relative to the bottleneck) rather than the *total* operator idle time for the day.
    - **Solution:** The user's feedback was crucial here. The data binding for the bar chart was corrected to use the dailyIdleTime metric. More importantly, the core formula for the workstation clocks was

rewritten to subtract the total productive time from the total available operator hours, resulting in a logically correct and much more meaningful visualization.

3. **Feature Evolution and User Experience (UX) Improvements:**
    - **Problem:** As the feature took shape, its design evolved. The initial histogram was deemed less effective than a direct comparison. The box plot's static axis was seen as less user-friendly than an interactive element.
    - **Solution:**
        - The histogram was replaced with a bar chart to show a direct, named comparison of idle time across workstations.
        - A horizontal line indicating the minimum idle time was added to the bar chart, providing an immediate performance benchmark.
        - The box plot's axis was removed in favor of an interactive tooltip, cleaning up the UI and providing richer data on demand.
        - The static "Line Efficiency Summary" title was replaced with dynamic KPI labels for "Total Idle Time" and "Idle-Time CV," putting the most critical data front and center.



Iteration 3: Dynamic Scaling and Positioning

- **Challenge:** The most persistent technical challenge was the precise positioning and scaling of D3.js elements within a responsive container. This is a classic D3 difficulty that required several rounds of adjustments to the margin conventions and transform attributes to get right.
- **Manual Adjustments:** To modify the existing code was impractical with an LLM, and involved instead going through the code and identifying any cases of adding or setting a scalar value, and instead setting it as a ratio of the SVG container. For Circles, the spacing needs to use a Min/Max function. This process involved testing across several different monitors and resolutions.

Sidebar and Color Theme Refinement

Date: *October 7th, 2025 {Dhruv}*

Overview and Motivation

This session focused on unifying the visual identity and improving the structural stability of the Factory Flow to Fortune application following a series of major merges. The primary goals were to reintroduce and expand the application's color scheme, refine sidebar interactions for clarity and usability, and reimplement previously completed features that had been disrupted during team integration. Additional emphasis was placed on code organization, error handling, and consistent behavior across visualization modules to ensure long-term maintainability.

Related Work

This work built directly upon earlier development sessions that introduced animated efficiency visualizations, an enhanced overview tab, and animated metrics within the right sidebar. With new design updates from teammates recently merged, this session ensured visual and functional consistency across the codebase while reapplying key UI and animation systems developed in previous iterations.

Questions

- How can a four-color system maintain clarity and harmony across complex, data-driven panels?
- What refinements to sidebar interactivity can improve usability and align with the refined visual language?
- How can previously implemented visualization logic be safely reintegrated following team changes without introducing new conflicts?
- What structural improvements can prevent rendering duplication and performance regressions?

Data

No data models were changed during this update. The work primarily involved structural and stylistic revisions to existing visualizations and interaction elements, focusing on user experience and application stability.

Exploratory Data Analysis

During testing after the merge, inconsistencies were observed in the schedule and efficiency panels, where visual elements (such as overlays and tick marks) failed to match the updated palette or loaded asynchronously. Similarly, input fields in the right sidebar appeared misaligned and lacked uniform visual feedback. These findings guided a comprehensive visual and behavioral overhaul that included restoring broken rendering flows and reapplying the established animation and styling conventions.

Design Evolution

1. Color Scheme Expansion and Reintegration

- The original three-color palette was reintroduced and expanded with an additional **secondary accent color** to create stronger differentiation and improve overall harmony.
    - **Teal (#009688):** The primary operational color used for task elements, buttons, and key interface highlights.
    - **Amber (#FFB300):** The performance and efficiency accent color used for cycle and productivity indicators.
    - **Purple (#7E57C2):** Newly introduced secondary accent color, providing visual richness and contrast in charts, legends, and supporting elements.
    - **Blue-Gray (#37474F):** Neutral tone used primarily for text, lines, and structural framing, providing visual balance.
- The new color structure was applied consistently across the **Schedule, Layout, Efficiency**, and **Overview** panels, establishing visual coherence and intuitive color mapping.
- Palette assignments were adjusted in the D3 chart rendering logic to ensure smooth gradients and color transitions within the new expanded scheme.

2. Right Sidebar Refinement

- Input textboxes were fully restyled for a cleaner, more professional appearance consistent with the rest of the interface.
- Padding, border-radius, and font size were adjusted for uniformity and improved readability.

- Teal and amber accents were added to active and hover states for better interactivity and alignment with the application's design system.
- Enhanced middle-drag sensitivity and smoother incremental changes were implemented to allow fine-tuned parameter control without lag or overshoot.
- Refined formatting for numeric input values ensured precision display and eliminated visual jitter during rapid updates.

3. Reimplementation of Integrated Features

- Restored all previously built modules (Efficiency, Schedule, Layout, and Precedence) following merge disruptions to ensure consistent functionality.
- Verified animation and event flow between panels to prevent redundant or conflicting render calls.
- Reconfirmed synchronization between data metrics (e.g., efficiency, idle time, throughput) and their respective animated sidebar displays.
- Ensured that switching between visualization tabs preserved state without triggering reinitialization errors or missing components.
- Revalidated tooltip visibility, overlay alignment, and clock animation behavior across all efficiency visualizations.

4. Codebase Refactoring and Stability Enhancements

- **Function Reorganization:** Reordered and modularized visualization functions for clarity and logical execution sequencing.
- **Initialization Improvements:** Consolidated setup logic in main() and setupEventListeners() to ensure variables initialize only once, eliminating duplicate rendering and event handling.
- **Error Handling:** Added multiple try...catch statements across key visualization updates to gracefully handle missing elements and avoid UI crashes during rapid user interactions.
- **Metric Baseline Stability:** Reinforced the reset logic for configuration data to maintain deterministic simulation behavior when user inputs are modified.
- **Animation Consistency:** Unified the animation timing for right sidebar metrics, eliminating flicker and ensuring synchronized numeric transitions.
- **State Management:** Improved cross-tab variable persistence for animationState, precedenceChartDrawn, and invalidPrecedenceNodes to ensure reliable transitions and visual consistency.
- **Code Clarity:** Cleaned redundant comments, standardized naming conventions, and streamlined internal references for easier debugging and collaboration.

Implementation

- **Files Modified:**
    - script.js: Major refactor integrating color, sidebar, and rendering logic.
    - style.css: Updated variable definitions and sidebar element styling.
- **Key Updates:**

- Rebuilt color variables in the stylesheet and synchronized them with D3-rendered elements.
- Updated numeric input handling logic for middle-drag and scroll-based adjustments.
- Refined layout logic in tab-specific render functions for consistent state control.
- Reinforced synchronization between tab rendering and sidebar metric updates.

Evaluation

- **Visual Clarity:** The four-color system enhances separation between operational and analytical elements, improving readability and immediate comprehension.
- **User Experience:** The sidebar improvements make parameter tuning more intuitive and consistent, especially under rapid updates.
- **Performance:** Refactoring and streamlined animation logic preserved smooth 60fps rendering performance.
- **Stability:** Improved state management eliminated prior duplication and overlapping render issues.
- **Maintainability:** The reorganized code structure simplifies debugging and supports scalable future development.

Session Outcomes

This session successfully combined visual refinement with architectural reinforcement. The expanded color palette introduced visual depth and improved contrast, while sidebar enhancements elevated usability and polish. Reintegrating and validating all visualization modules ensured that the system retained its full analytical and interactive capability following collaborative merges. Collectively, these changes strengthened both the user experience and the technical foundation of the Factory Flow to Fortune application.

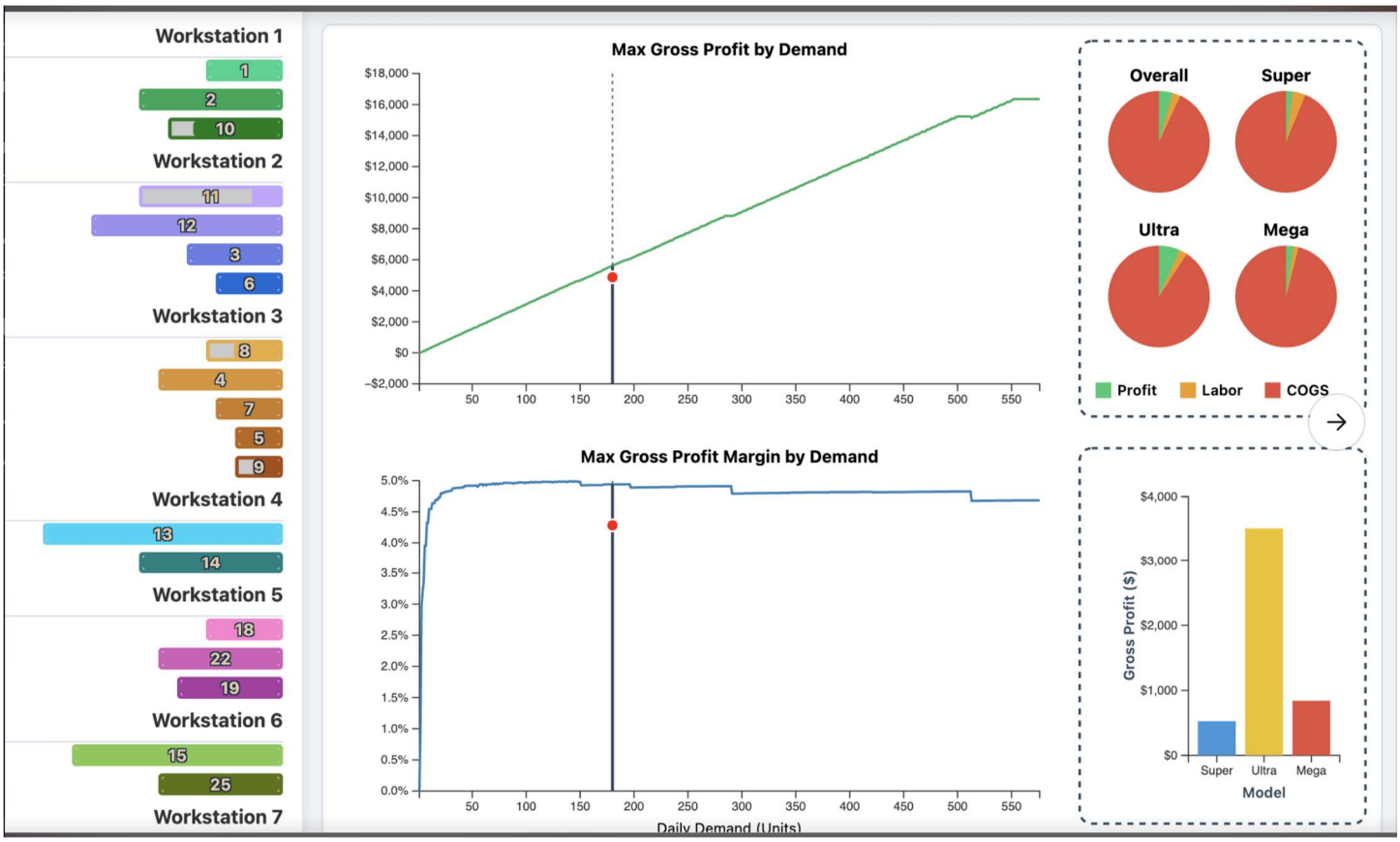
Profit Tab Refinement

Date: *October 7-8th, 2025 {Sumaiya}*

Current state of profit tab visualization was functional but lacked clarity, consistency, and readability for professional analytical use. The tab's backend logic was structured around calculating optimal daily profit and profit margin by simulating all combinations of demand, operating hours, and employee counts, with only legal configurations considered for each iteration. These results were aggregated and displayed as simple line charts—one for profit and one for margin—using D3 rendering, with raw values plotted directly against demand.

However, this version suffered from notable UI and usability issues:

Visual context was limited due to the absence of background gridlines in either chart, making it challenging for users to interpret trends or micro-shifts in profitability and margin data. Axis labels and titles offered basic guidance, but there was no high-contrast or visually appealing styling to facilitate quick analysis, especially under conditions like high-resolution or low-light screens. Tooltips, though present, were inconsistent and only revealed raw values without indicating which worker/hour configuration achieved those outcomes. Another notable drawback was the cramped and disjointed breakdown panel, which displayed per-model pies and bar charts. These charts suffered from mismatched color schemes and inconsistent legends, offering little structural navigation between the different sections. Altogether, the lack of gridlines, muted interactivity, uneven tooltip architecture, and a cluttered breakdown panel meant the tab delivered results but failed to communicate operational insights intuitively or elegantly. The overall experience, while technically sound, did not deliver a cohesive or refined analytical workflow for understanding profitability trends in the assembly line context.



I addressed these issues by implementing -

- Dual grid layers (major + minor) now appear in both charts, offering a frame of reference for both macro and micro trends. The major grid handles broad intervals, while the minor grid provides finer detail.
- The grid system is rendered through the helper drawAxesWithGrid(), which draws separate minor and major layers for both x and y axes.
- To maintain cleanliness, labels are only rendered on the main axes; both major and minor gridlines use tickFormat("") to avoid duplication.
- Axes and titles use increased contrast to improve readability on modern high-DPI screens.
- The profit chart (green) and margin chart (blue) share consistent styling conventions, using rounded line joins, subtle drop shadows, and inline labels

A live crosshair system was added that tracks the cursor on both charts simultaneously:

- Vertical and horizontal lines move with the pointer.
- Real-time value badges display the exact demand (x), profit (y_1), and margin (y_2) at the hovered point.
- This feature enables instantaneous reading without guessing from axis scales—ideal for comparative operational decisions.

Previously, tooltips existed per chart with inconsistent formatting. Now, all components—profit lines, margin lines, and the breakdown bar and pie sections—share a unified tooltip template:

- Each tooltip shows Demand (x), Optimal Profit, Profit Margin (%), and the best configuration (workers × hours).
- Bottom bar tooltips highlight profit and margin percentages in sync with the main charts.
- Tooltips now include a debounced mouse tracking system to prevent flickering and are consistently dark-themed with minimal offsets.

The redesign emphasizes clarity through restrained contrast:

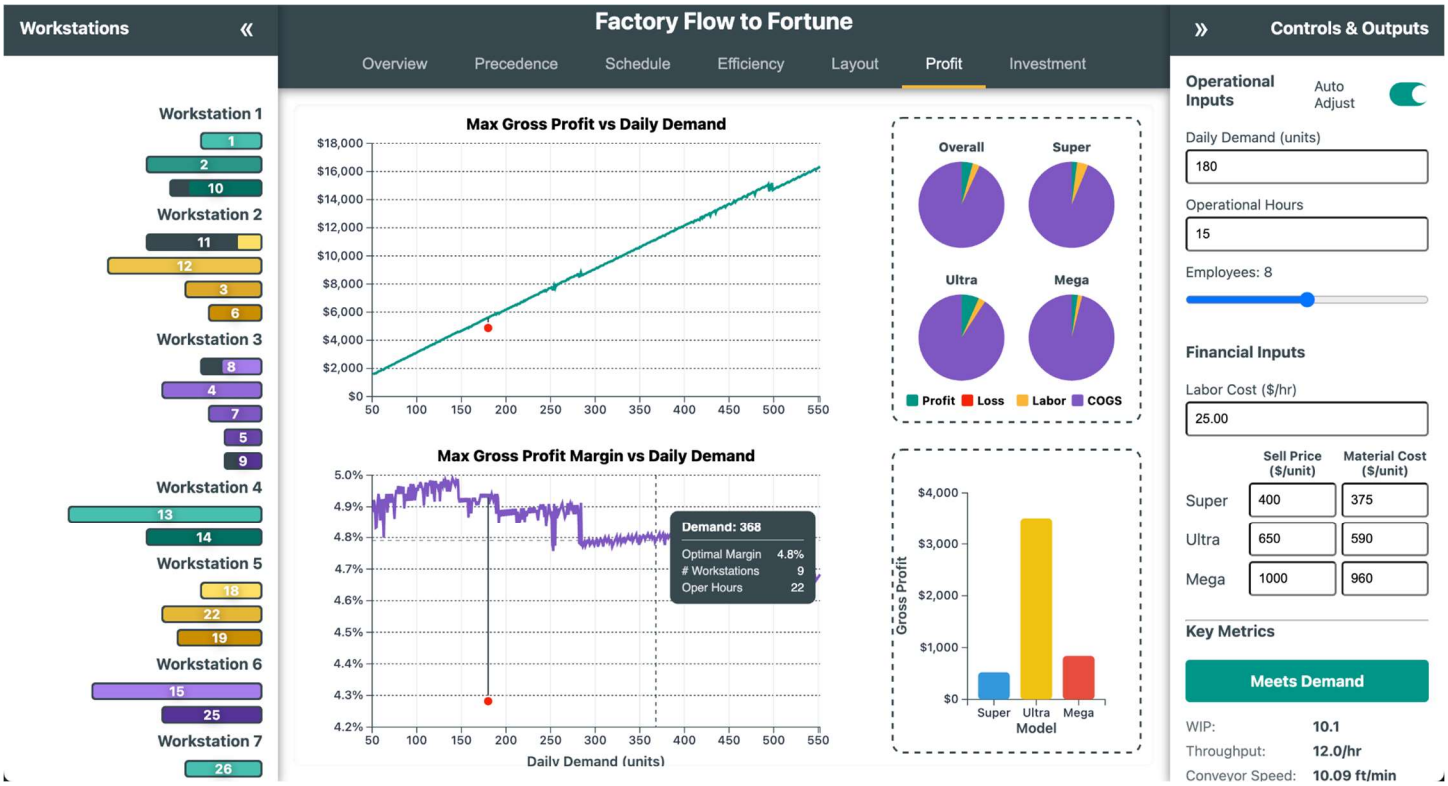
- The profit line is a vivid green (#10B981).
- The margin line is a crisp blue (#2563EB).

- The lost-profit area (the gap between requested demand and achievable throughput) remains but uses a softened gradient for context rather than distraction.

Each chart maintains:

- Zero baselines to prevent visual distortion.
- Rounded markers that illuminate on hover.
- Balanced spacing with meaningful negative space for betterinterpretation.

The breakdown panel—displayed on the right—was expanded by around 32% of total width and visually decoupled from the charts. I included rounded corners, a harmonized legend, and proportional pie sectors give a cleaner, structured interface. Each pie chart section (Profit, Labor, COGS) now uses distinct hues consistent across all visualization layers (green, orange, red).The bottom bar chart now features rounded bars, smoothed animations, and a matching tooltip style.



Tooltip and Animation Unification

Date: *October 8th, 2025 {Dhruv}*

Overview and Motivation

This development session focused on elevating the overall user experience and improving maintainability within the Factory Flow to Fortune application. The primary objectives were to unify the visual design of tooltips, standardize layout behavior across the Overview tab, enhance input control interactivity for smoother usability, and optimize animation performance using modern browser APIs. Collectively, these improvements created a more cohesive, efficient, and polished product experience.

Related Work

This session builds on the prior rounds of visual refinement and architectural stabilization, particularly those that introduced standardized color systems, improved sidebar interactions, and modularized visualization rendering. The current work addressed cross-cutting UI consistency and performance challenges that arose as the application's complexity increased.

Questions

- How can tooltip styles and layouts be unified across multiple visualizations while maintaining scalability?
- What layout adjustments would bring professional visual balance to the Overview page?
- How can interactive input controls provide instant feedback without degrading responsiveness?
- How can animation performance be improved when the user navigates away from the browser tab?

Data

No new data sources or datasets were added. This session focused entirely on interface and interaction behavior within existing application structures.

Design Evolution

1. Unified Tooltip Design System

Problem: Tooltip designs were inconsistent across modules, with different inline styles and separate CSS classes (.pert-tooltip, .profit-tooltip, etc.), leading to a disjointed user experience and difficult maintenance.

Solution: All tooltip styles were centralized into a unified design system within the CSS.

- **Consolidated CSS:** A single .d3-tooltip class was created to define consistent padding, background, shadow, and border-radius across all tooltips.
- **Structured Content:** Helper classes such as .tooltip-header and .tooltip-row were introduced to provide a clear, uniform hierarchy for tooltip titles and key-value data rows.
- **JavaScript Refactoring:** The createTooltip() function in script.js was refactored to use these standardized classes, eliminating inline styling and ensuring a single, reusable tooltip structure for all visualizations.

Outcome: All tooltips now share a professional, cohesive design. Future adjustments can be made entirely in CSS, greatly improving maintainability and reducing visual fragmentation across the application.

2. Standardized Overview Page Layout

Problem: The Overview tab displayed uneven card heights and inconsistent text sizing, reducing visual polish and readability.

Solution: The grid system and typography were standardized for a balanced and uniform presentation.

- **Uniform Card Height:** By removing height: fit-content and adding grid-auto-rows: 1fr to the .overview-container grid, all cards now expand to match the height of the tallest card in each row.
- **Consistent Typography:** The “Optimization Tips” section had a smaller font size due to an isolated rule. The .tip-content class was updated to match the standard 0.75rem used throughout the Overview tab.

Outcome: The Overview grid now maintains perfect visual alignment, creating a cleaner, more professional presentation of project information.

3. Enhanced Input Control Interactivity

Problem: The Ctrl+Click shortcut for resetting input fields to their default values did not commit changes instantly—requiring users to click away or press Enter to trigger updates.

Solution: The event-handling system was restructured for immediate feedback.

- **Scoped Event Handling:** The reset logic was moved from enableMiddleDragNumberInput() into attachCommitBehavior(), giving the event direct access to the onCommit callback.
- **Immediate Recalculation:** Upon Ctrl+Click, the application now calls commitInput(input, onCommit) directly, instantly updating all dependent visualizations.

Outcome: Input resets now occur immediately, delivering a responsive and intuitive user experience aligned with high-quality interactive design standards.

4. Animation Optimization with the Page Visibility API

Problem: The complex D3 animations used in the Schedule and Layout visualizations continued running even when the browser tab was inactive, consuming unnecessary system resources.

Solution: The Page Visibility API was implemented to intelligently pause and resume animations based on tab visibility.

- **Visibility Listener:** A new handleVisibilityChange() function monitors document.hidden.
- **Stateful Pausing:** When the tab is inactive, each active animation's isPaused flag is set, halting frame updates.
- **Seamless Resumption:** Upon returning, the isPaused flag is cleared, and the animation clock (lastFrameTime) is reset to the current time to ensure smooth continuation without time jumps.

Outcome: The application now conserves system resources efficiently, pausing complex animations when inactive and resuming them seamlessly—improving performance, battery life, and overall browser compatibility.

Implementation

- **Files Modified:** style.css, script.js
- **Key Additions and Changes:**
    - Introduced unified .d3-tooltip class with structured helper classes (.tooltip-header, .tooltip-row).
    - Refactored createTooltip() and related functions to eliminate inline styling.
    - Adjusted grid structure and text scaling within .overview-container.
    - Reorganized event handling for input field reset logic to allow direct commitment.
    - Added visibility state management and handleVisibilityChange() for animation loops.

Evaluation

- **Visual Consistency:** All tooltips and Overview cards now adhere to a unified design language.
- **Usability:** Input controls provide immediate feedback, reducing user friction.
- **Performance:** Background animation processing reduced significantly due to visibility-based pausing.
- **Maintainability:** Centralizing tooltip and layout logic simplified future styling and code adjustments.
- **Professional Polish:** The UI now demonstrates a higher standard of precision, alignment, and responsiveness across all tabs.

Session Outcomes

This session successfully consolidated several independent refinements into a cohesive and impactful update. Through visual unification, performance optimization, and improved interactivity, the Factory Flow to Fortune application achieved a new level of polish and technical maturity. The work completed here not only enhances immediate usability but also strengthens the maintainability and scalability of the system for future development.

Code Cleanup, Refactoring, and Optimizing

Date: *October 8th, 2025 {Joel}*

The HTML and CSS was refactored, with the HTML for the Overview and Investment tabs were moved into their own pages. While refactoring redundant CSS, a set of helper functions were used to eliminate redundant code within the structure. This process did little to modify or change the visuals, and therefore, will not be meaningfully expanded on here; it was simply intended to help improve the modularity of the code, and enable more consistent formatting as we update further UI changes.

Date: *October 9th, 2025 {Joel}*

The optimizations of the profit tab were applied to the investment tab, which has some similar, but still meaningful differences in how it calculates values. Certain elements of the backend were further tested and adjusted. Similarly, this had minimal impact on the UI itself. However, it did make it so that the tab no longer needed a message to indicate it was calculating. It was modified to dynamically recalculate with changes in variables which were previously being limited in their impact.

Date: *October 10th, 2025 {Joel}*

Refactoring of the JS tabs was implemented, where any functions used exclusively by a tab were shifted into self-contained IIFEs and then moved out of the main code. As LLM usage is no longer optimal, the time was taken to add clarifying comments to the code for more precise code editing to be viable. A meeting was held to review the current state of the code, acting as our internal milestone review, to be shown in the next segment.

Mouse UI Refinements

Date: *October 9th, 2025 {Dhruv}*

Overview and Motivation

This session focused on improving responsiveness, input precision, and animated visual feedback across the **Schedule**, **Efficiency**, and **Layout** tabs of the Factory Flow to Fortune application. The goal was to enhance usability, particularly on high-DPI and small-screen devices, while creating smoother and more intuitive interactions for zooming, adjusting simulation speed, and monitoring real-time production metrics.
The refinements addressed clunky scrolling behavior, inconsistent slider precision, under-emphasized production feedback, and limited motion coherence in visual cues such as clocks and gauges.

Related Work

These improvements build upon earlier interface stabilization and performance optimization work, including prior color system integration and sidebar interactivity refinements. This session continued the focus on polish and precision—aligning dynamic visualization behaviors with the system's overall aesthetic and functional standards.

Questions

- How can zoom and scroll interactions be made more predictable and fluid during schedule exploration?

- What adjustments will make sliders accurate and responsive across various resolutions and input types?
- How can production feedback be more visually engaging while maintaining performance and clarity?
- How can animation systems be refined to prevent flicker and improve visual continuity?

Data

No new datasets were introduced. All changes were interaction and animation refinements applied to the existing D3 visualizations and UI control systems.

Design Evolution

1. Schedule Tab Scrolling and Navigation

Problem: The schedule panel's zoom and pan behavior felt clunky and unbounded, making it difficult to explore the timeline smoothly.
Solution: Implemented **D3 zoom** with bounded scale and constrained transform logic to ensure smooth, contained navigation within the schedule chart.

- Added a zoom handler to manage scale extent and translation limits for consistent scroll boundaries.
- Bound the zoom behavior directly to the SVG group, ensuring natural, physics-like panning with dampened transitions.
  Outcome: Scrolling and zooming now feel precise and fluid, allowing predictable timeline exploration without overshooting or jitter.

2. Speed Slider Refinement (Responsive and High-DPI Fixes)

Problem: On high-DPI and small screens, speed slider clicks often jumped to the maximum value or registered imprecisely due to coordinate scaling issues.
Solution: Rebuilt slider logic with fully responsive sizing, local coordinate mapping, and domain inversion to match user expectations.

- Increased the effective interaction area for easier grab and click on smaller viewports.
- Reversed the slider domain from [minVal, maxVal] to [maxVal, minVal], ensuring intuitive "faster at top, slower at bottom" behavior.
- Calculated pointer position using native.clientY - rect.top to accurately determine local coordinates, removing reliance on inconsistent event.x values.
- Added wheel-based fine-tuning for incremental speed adjustment.
  Outcome: Slider interaction is now consistent and precise across resolutions, with smooth transitions for drag, click, and wheel inputs.

3. Product Counter Integration

Problem: Users lacked immediate visibility into current production counts during simulation playback.
Solution: Added live, on-screen counters for **Super**, **Ultra**, and **Mega** product lines, dynamically updating with simulation progress.

- Integrated counters into the schedule panel UI and bound them to the animation loop.
  Outcome: Production metrics are now surfaced prominently, improving situational awareness during simulation.

4. Efficiency Tab Clock Animation

Problem: Idle and efficiency clocks lacked visual smoothness and temporal clarity, making time representation feel static.
Solution: Introduced animated, tick-marked clocks with continuously updating hour and minute hands using **D3 attrTween** and **interpolation** for natural motion.

- Added tick marks and clamped angle transitions to prevent flicker during efficiency updates.
- Synchronized workstation idle clocks and overall summary clocks with real-time simulation time.
  Outcome: The Efficiency tab now communicates temporal performance through fluid, engaging clock animations that enhance interpretability.

5. Summary Text Alignment and Theming

Problem: The summary section had inconsistent positioning and theming across different resolutions.
Solution: Recentered and rescaled the summary group for proportional layout on all viewports.

- Adjusted text positioning, spacing, and border alignment for a balanced layout.
  Outcome: The summary visuals now align cohesively with the dashboard's color and typographic hierarchy.

6. Layout Tab Speed Slider and Gauge Animation

Problem: The Layout tab's speed slider suffered from the same precision issues as the Schedule slider, and speed feedback lacked visual clarity.
Solution: Implemented a fully interactive vertical speed slider paired with an animated speedometer gauge.

- Used local Y-coordinate mapping for accurate pointer interaction.
- Inverted the scale domain and clamped values for controlled range behavior.
- Introduced a persistent animationState.speedo.currentAngle to maintain the gauge's state across redraws.
- Implemented a smooth **attrTween** rotation transition for the speedometer needle, mapping conveyor speed to gauge angle.
  Outcome: The speedometer now provides both visual and numeric feedback, with motion continuity and clear responsiveness during user interaction.

Implementation

- **Files Modified:** script.js
- **Key Updates:**
  - D3 zoom logic for bounded schedule scrolling and smooth panning.
  - Fully responsive, high-precision speed sliders in Schedule and Layout tabs.
  - Real-time product counters integrated into the simulation update loop.
  - Animated clocks in Efficiency tab with attrTween interpolation.
  - Repositioned and themed summary section text.
  - Speedometer animation with persistent state and numeric readout for Layout tab.

Evaluation

- **Responsiveness:** Scrolling and slider interactions now perform smoothly and predictably across all input modes.
- **Precision:** Refined coordinate mapping and clamping logic provide consistent slider feedback across resolutions.
- **User Feedback:** Animated gauges, counters, and clocks create a more immersive, informative visualization experience.
- **Visual Consistency:** Summary section and animation color themes align with the established palette.
- **Performance:** Animation refinements maintain stable frame rates with no detectable latency on modern browsers.

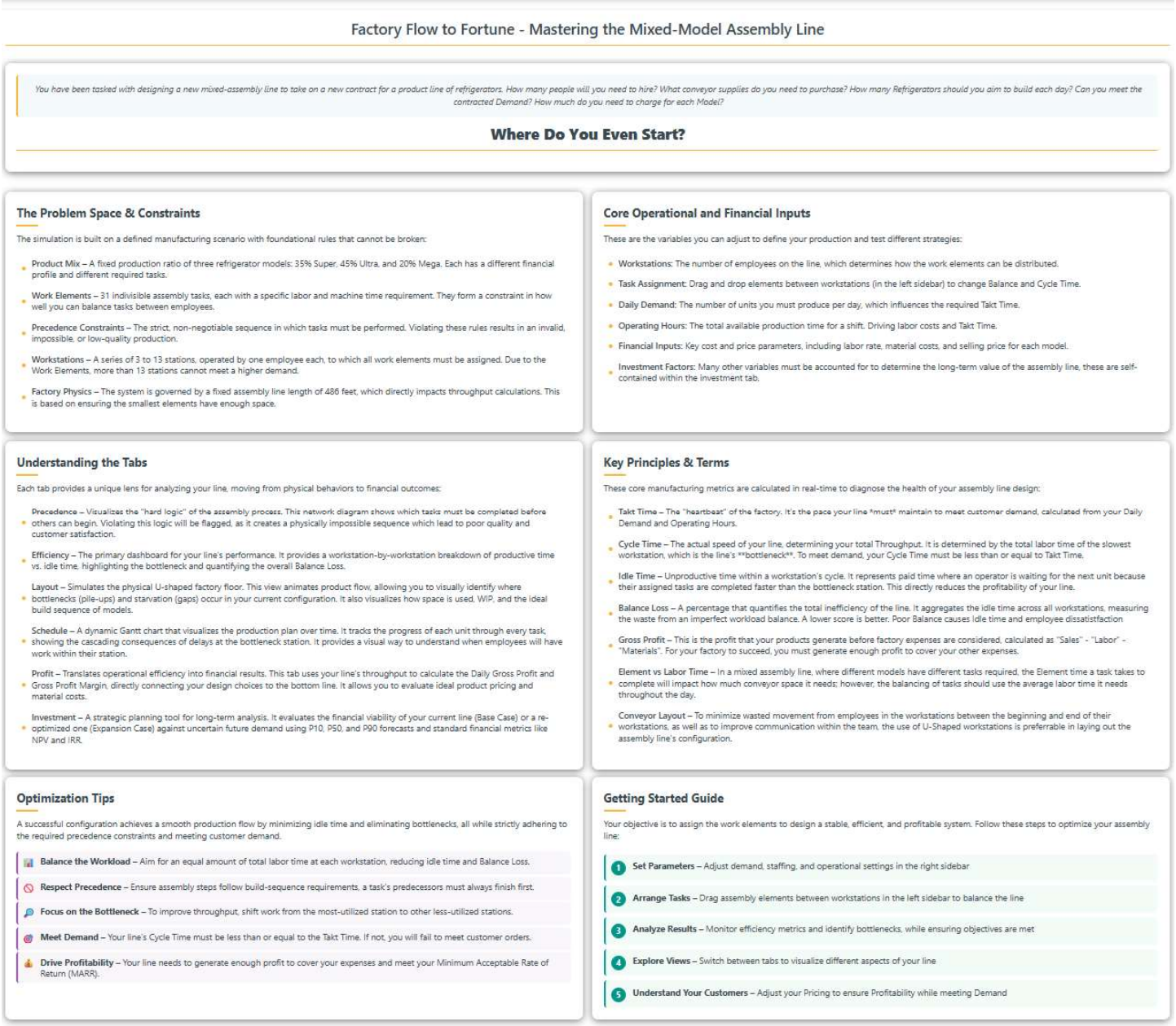
Session Outcomes

This session successfully unified and enhanced interaction responsiveness and visual feedback across multiple tabs. The improved scroll handling, responsive sliders, live counters, and animated efficiency clocks together create a more dynamic, tactile, and polished user experience. These refinements significantly elevate both the functional precision and the aesthetic quality of the Factory Flow to Fortune interface, setting a strong foundation for future accessibility and adaptive-layout enhancements.

Overview Tab and UI Adjustments

Date: *October 11th, 2025 {Joel}*

The base template for the Overview Tab created by Dhruv was modified to become a realistic explanation of the problem space, self-imposed constraints, operational and financial inputs, a description of the other 6 tabs, an exploration of key principles of factory physics, with a set of tips on how to optimize an assembly line, and a getting started guide to cover the basic sequencing someone should use to understand the webpage. Little was visually changed, other than adjusting the emoticons used in the optimization tips. I instead focused on the text descriptions themselves.



I then went through our meeting minutes from the 10th and resolved the notes assigned to myself:

- Change the Scrollbar colors to match the grey accent color.
- Change the color scheme of the Investment tab to match the other tabs.
- Fix the partial demand values showing in the tooltips.
- Add tooltips to describe all the investment tab variables
- Adjust the formatting of the text in the Investment tab to be more uniform and legible.
- Flag negative values in the scorecard.
- Make it so that changing CI values will modify the P10 and P90 values, rather than just showing.
- Prevent P10 from being greater than other demand levels, and P90 being less than.
- Add an Auto-Adjust switch to the right sidebar to be able to turn on or off the hierarchical logic.
- Add a rounded line around the elements in the left sidebar.
- Remove the small grey corners which show in some of the element bars.

Profit Tab Improvements

Date: *October 11-12th, 2025 {Sumaiya}*

Several significant modifications have been made to the profit tab code from the previous iteration to the latest version, focusing on improved usability, visual integrity, and more accurate data presentation[file:11].

- X-Axis Label Visibility

  The x-axis label “Daily Demand (units)” was not fitting within the SVG viewport in earlier versions, often causing it to be cut off or not visible, especially on certain screen sizes. This issue was fixed by increasing the chart's bottom margin from 40 to 56 pixels. Additionally, the label’s position was changed to be dynamically set at $$ y = \text{chartHeight} + (\text{margin.bottom} - 12) $$, ensuring universal visibility regardless of layout or viewport[file:11].

- Legend and Breakdowns: COGS to Material

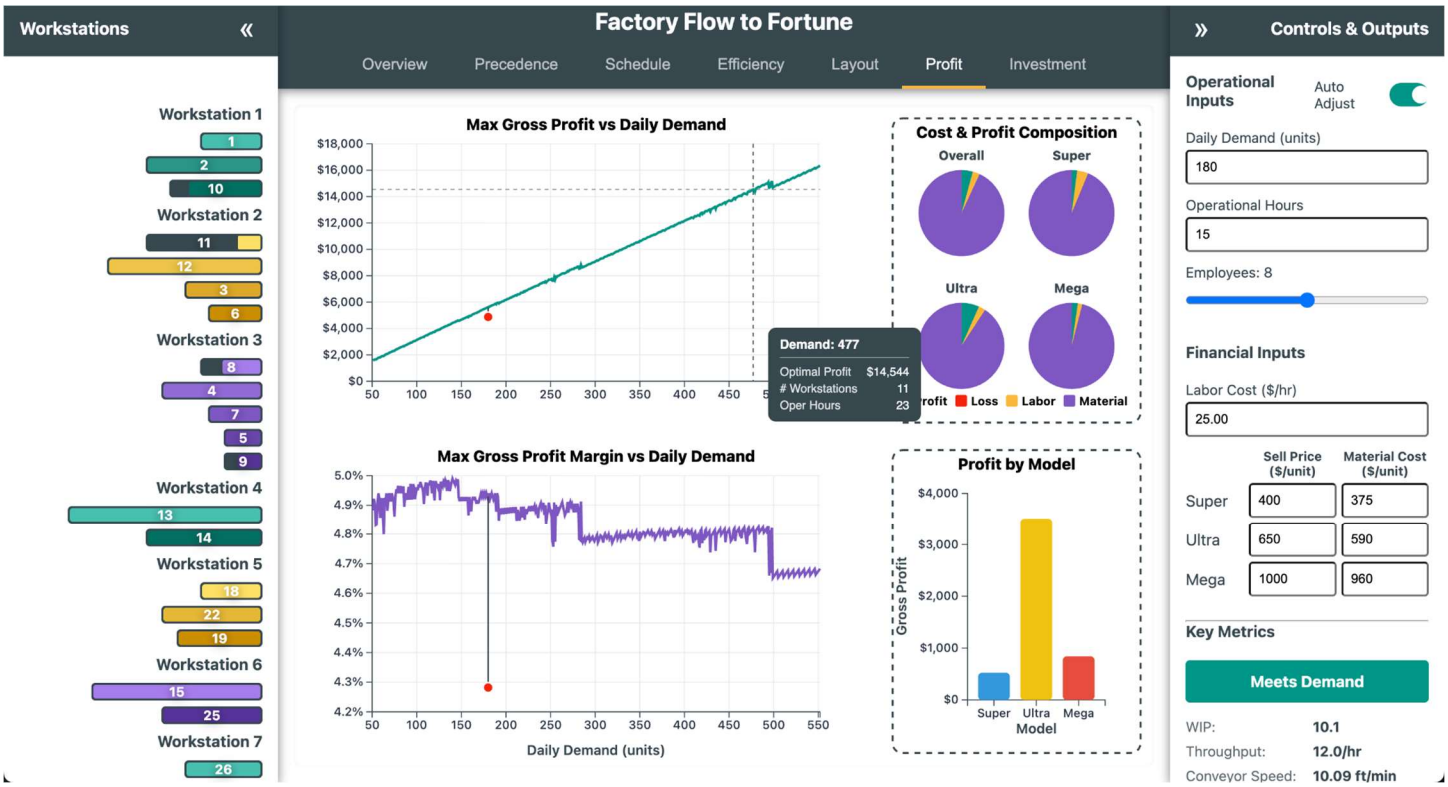
  The legend previously left extra padding on both sides, restricting horizontal room and sometimes resulting in cramped or overlapped entries, especially when labels were longer. The updated code now aligns the legend to the inner border of the right panel by starting at $$ x = \text{pad} / 2 $$ and using a total width of $$ \text{breakdownWidth} - \text{pad} $$, allowing the “Material” category (formerly “COGS”) to be added seamlessly without overflow. Furthermore, the legend layout no longer uses fixed columns; instead, each label’s width is measured and legend items are wrapped to a new line when necessary, preventing visual overflow or overlap outside the bordered region[file:11].

- Grid Line and Crosshair Contrast

  Previously, the x-axis displayed dotted grid lines that could visually conflict or overlap with the live crosshair indicator, making it hard for users to distinguish cursor position from chart references. The code now renders the x-axis grid with the “grid-major” class, resulting in a muted line style so the crosshair remains visually distinct and easily trackable on the plot[file:11].

- Legend Title and Layout Improvements

  A dedicated title for the legend was introduced, which brings greater clarity and helps users quickly interpret the breakdown panel. The legend’s elements have been repositioned so that all items are laid out in a flowing, wrapped manner with improved spacing, making the legend more readable and visually integrated within the overall chart design[file:11].



Now, the updated Profit tab delivers a cleaner, more readable, and user-friendly experience through a series of thoughtful visual and structural refinements. The x-axis label issue was resolved by increasing the bottom margin and repositioning the “Daily Demand (units)” label to fit neatly within the viewport. Gridlines were softened to a muted tone so the crosshair now stands out clearly, and the right-side legend was expanded, wrapped, and renamed—replacing “COGS” with the clearer term “Material.” Additional titles were added for both the legend and subpanels (“Cost & Profit Composition” and “Profit by Model”), making the layout easier to interpret. Tooltips are now unified across all charts, presenting consistent information such as demand, profit, margin, and configuration, while the loss and zero baselines are handled gracefully in pies and bars. Together, these changes improve clarity, readability, and analytical precision, turning the Profit tab into a polished, insight-focused visualization rather than just a functional chart display.

SME Interviews

As follows are summaries of a series of feedback interviews from a variety of Subject Matter Experts who could review how effective the visualizations were within a given context, and how they could be further improved.

Randy Bremner: *(Manufacturing Senior Staff Engineer)*
Date: *October 13th, 2025 {Joel}*

- **Precedence Tab:** This tab can feel overwhelming, although there aren’t many great ways to communicate precedence as a visual. The use of a tree is good for showing precedence, but it takes a while to process what you are looking at. The use of this as a diagnostic tool is effective, due to the highlighting feature. This pop-out feature is what makes the tab useful rather than just a novelty.
- **Schedule Tab:** This tab is an interesting visual, but it’s just showing an ideal situation. In a real production setting, this wouldn’t be particularly helpful, as you need to see how to adapt to the unexpected, not the expected. Even if you have set demand ratios for how tasks are allocated, it would be good to show how scheduling should be done if supply chain is interrupted for one of your models, or to account for machine outages. This tab is good to visualize the base behavior, but as a decision tool, you need to include how you should react to the unexpected.
- **Efficiency Tab:** This tab is good to show the balance between workstations and idle to productive time for them; and this is the sort of tool that a floor manager would have the most use for. However, it is important to note that idle time is essential for smooth production, you should allocate time for employees to be idle, or else they are more prone to make mistakes due to feeling rushed, being exhausted, or simply having no room for error. Even in an assembly line, variance exists, and you need to use either a Kanban system or deliberately run idle time across stations to ensure proper quality. This lacks attention to how quality and efficiency balance each other with different costs.
- **Layout:** This tab is extremely satisfying, especially with the choice of how to structure the line algorithmically, and how the conveyor speed and spacing calculations are being done. The decision to make everything a U-line, including the whole line, as a nested u-line is an optimal decision, and seeing it being done here so effectively is really awesome (He spent 15 minutes talking about how much he liked the structure) The use of an algorithm to sequence out the mixed build to keep production smooth is a great touch, as not using proper sequencing for mixed-lines will lead to a drift of work within stations, where eventually some will fall behind. It would be good to allow a change to this sequencing to account for how to optimally do the sub-optimal. Sometimes your customer might want you to surge a particular model, so it is important to be able to recalculate what your actual capacity for a new ratio is, and how to realign your sequence to it.
- **Profit:** This tab is interesting for how it shows the optimal build for each demand level, although this is going to have its limits due to being based on just gross profit. It does not account for many of the other variables which govern profitability. How are you accounting for re-work?
- **Investment:** This is probably the best tab from a decision standpoint. It lets you consider how you can change demand and pricing, which are intimately tied together, with your operation to make that determination of how you should expand, or if you should build in the first place. It would be really neat if it had some microeconomic elements which would let you follow a supply-demand curve, to further determine not only your ideal configuration, but also your ideal combination of supply and pricing to get the most profitable demand. This is probably outside the scope of this sort of project but creating an

investment tool like this really helps communication with executives and taking further into account the economic factors would really make this tool stand out.

- **Recommendation:** If there was something I feel this really needs, it is an awareness of how Quality yields will impact scheduling, cost, and what level of idleness buffer you need. While you cannot make something like this truly representative of real-life, any manufacturing system needs to account for cost and schedule impacts of Quality, both in Scrap and Rework, and their impacts from yield.
- **Summary:** This is an incredible mix of views to give a diverse view of how a conveyor line behaves. It seems like it would be a good tool for education, but something like this shouldn't be used in an actual manufacturing environment. The reason for this is that all dashboards and visualizations should avoid being general and instead focus on a singular stakeholder and what they need to drive their decisions. You shouldn't make a tool which simultaneously shows what your floor manager, factory manager, and executive leadership need. Rather, the main reason that dashboards like this fail is that they don't consider what the end-user actually needs or will use. Sometimes it is just as simple as how bars are stacked. This collection in a dashboard is great as an educational tool, but a manufacturing dashboard needs to be more selective in what it shows—as it is important to only share what is needed or useful to a given level.

Neil Zimmerman: *(Manager Programs 2)*
Date: *October 2nd, 2025 {Joel}*

Initial Conversation: Neil was enthusiastic about the way that factory physics was animated intuitively, and he liked the current state of the UI. His recommendation was that while the current tabs help explore the concept of factory physics, it doesn't really give the information that a manager would want to use to determine if building the line was beneficial. "If you do a 'current' vs 'proposed' model for any kinds of improvements, you can look at the time-value of money and rate of return on continuing your operation vs pursuing your proposed modifications. It would also be good to consider asset depreciation in assessing the investment. By adding economic analysis, you can see how long the current setup is good for, and at what point you should expand to meet a new demand. You can create a Demand/Sales Funnel Scenario which assesses P10, P50, and P90 demand projections, then consider what equipment and resource expansions would be necessary. It also informs the question of, do I get salvage value from a workstation, or do I put in the resources to drive an increase in demand? Being able to show the dynamic ability to zoom into efficiency metrics, and with that zoom out to capital/strategic business planning would show a mastery of the problem-space."

Date: *October 13th, 2025 {Joel}*

Review of the website tabs in the current release:

- **Overview –** this tab is great, I have no recommended changes, it succinctly describes the problem space in a way that represents the subject well.
- **Precedence –** it would be good to set window boundaries, whenever I move a node, it often leads to the graph shifting off the screen. The drag and drop feature is really cool and interactive, but it shouldn't compromise on legibility of the precedence requirements.
- **Schedule –** this tab is very intuitive, a good view of how product will move through the line. An adjustment to the layout to reduce overlap with the lower stations would be good.
- **Efficiency –** this tab is intuitive and meaningful for evaluating balance and idle time; no recommended changes to the tab, it is effective at what it needs to communicate.
- **Layout –** this tab is absolutely amazing, not only is it engaging, but the automatic restructuring of the line is a top-notch feature. Adding some sort of note to show that grey fill means the model isn't being worked on at that element would be good. Everything else is superb. This and schedule are my favorite tabs; the animation really helps bring a dynamic subject to life.
- **Profit –** The view of looking for optimal gross profit is interesting and offers a good 'intro' level of understanding to the economics of the line; but gross profit isn't particularly meaningful compared to net-profit. I personally would have preferred a P&L Sankey to show how the 'value' of the product is diminished through expenses for each model. It is understandable not to want to integrate a full P&L, but it would make this far more meaningful for an actual business decision.

- **Investment –** the graph is legible, and the overall interface is asking the right questions for determining if an assembly line is worth investing in. It is impressive how quickly it is performing the analysis; it will need quite a bit of time to verify that the outputs are correct due to how complex the calculations are. When you take Engineering Economic Analysis, you should come back to this tab to further develop and improve it.

Pedro Huebner: *(Director of Systems Engineering – University of Utah)*
Date: *October 16th, 2025 {Joel}*

It was Pedro's summer course on Design of Production and Service Systems which formed the base problem space for this project. He doesn't have much experience with the Economic aspect of this project, but he has studied and taught principles of Industrial and Systems Engineering for years. He intends to provide both a technical review and to also give feedback on how effective the website would be as a supplemental teaching aid in teaching the basics of factory physics.

- **Overview –** This is actually a great summary of the problem space, and the way this website addresses it. It would be good to more clearly specify that tooltips will exist to explain certain terminologies, how the auto-adjust works, and how elements can be dragged around between workstations to change the balance are all features which would be good to understand at the beginning, but you are expected to just find them. The summary of factory physics is great, the description of what the site can do needs more development. The page looks weird on his laptop though, so it may need some adjustment for the font sizing and spacing to be resolution ambivalent. A link to explain the backend may be helpful, for those who want to understand the inner mechanics.
- **Precedence –** The drag and drop functionality is fun; but it would be nice from a task allocation perspective to include the ability to automatically shift the nodes into a Kilbridge and Wester diagram; basically, the tasks are assigned to columns based on how far into the precedence tree they are. This is used for some heuristics for line-balancing and is something that is good to make sure tasks are allocated to favor how early in their sequence they are. The highlighting of precedence violations is a great feature, but it seems to have a glitch where if you have precedence violated, the 'flag' for breaking precedence is being rechecked if elements are automatically rearranged (such as changing employee count), you must physically readjust them to reset the flag.
- **Schedule –** The bottom left is getting obscured by the product selection filter, it should be shifted to a less critical area of the screen and given a white background like the legend. The bottleneck workstation may make more sense if the red background is given to the whole workstation, since it doesn't cover the whole section for the workstation, it isn't intuitive that it impacts the full station. Clicking on the schedule will shift it forward very abruptly, and the behavior isn't immediately clear. It might help to give it a fast animation, so you can visibly see the abrupt movement forward rather than the screen just flashing forward. It would be nice if you had an export button which would give you a list which showed the build sequence—as this is important information for someone who is going to be making the schedule, and a list is going to be more readable from a planning perspective. All in all, this is a good use of discrete event simulation.
- **Efficiency –** This tab is really a great tool for finding how you should balance your line to be productive, and the whole thing just wouldn't work without it. It would be good to flag the Bottleneck workstations, such as highlighting red the axis labels for zero values in the balance per workstation and/or adding a highlight, similar to the precedence diagram, around the bottleneck(s) idle-time pie graph. If the total idle time is over 12 hours, it makes sense to highlight the clocks to distinguish them, but the behavior isn't immediately obvious; perhaps make the Meets Demand button go red and say, "Too Much Idle Time"? There is also a graphical glitch where the Balance Loss diagram is stacking up at the end if the employee count changes quickly. (this is timing issue for the axis assignments)
- **Layout –** It would be good to add a functionality to be able to skip to the end of the simulation, so that you can see the full finished goods sequence without needing to wait through the animation. This tab is absolutely incredible, the way it dynamically readjusts and allocates space based on the element assignments. The integration of the MSSA algorithm for the build sequences is a great touch to making this a strong representation of how the line would behave. The dynamic adjustments of the spacing and conveyor speed are well-done; however, the color sequencing of the speedometer is confusing without some sort of legend. What do the colors actually mean?

- **Profit –** this tab is intuitive, but it would be further improved by clarifying the red dot as being the 'current configuration', and the red area representing demand which cannot be met. These features are great, but go unexplained, so one of the most helpful parts of the chart has to be 'figured out'.
- **Investment –** this is really impressive and really shows a deep understanding of how to apply the subject material. It is surprising how fast it is able to perform these calculations. He isn't an expert on these, but he looks forward to how these can be further improved once I take Economic Analysis this Summer with Professor Easton. He wanted to know where I learned this, and I explained that since I had a decade of management experience, combined with some self-study, these were subjects I reasonably understood. He is interested to see how I can further refine this when I take Engineering Economic Analysis.
- **Location –** this tab is not developed beyond a basic framework, it is simply meant to show the intent and behavior, as this is something Pedro has experience with. He recommended making sure that the 'target' location in the continuous form will show the geo coordinates of the new factory and also simplify how FTL and LTL shipments are communicated. He wanted and explanation of how the calculations were being performed, to which I explained that it is performing a Cost-Weighted Algorithmic Median search in which cost is being computed with a scaling circuitry adjustment to great circle distances, combined with a cube-out of 54 units to then find an optimal combination of FTL and LTL shipment strategies to meet the specific demand schedule. He is impressed by how well both continuous and discrete analysis are performed here, as I will be taking Operations Research next semester, he is interested to see how I can further refine my algorithm after taking that class. He suggested allowing the adjustment of PPI, rather than using my current fixed value, would allow pricing to be adjustable overtime. You could also add a holding cost into the calculations, from which you could show a shipment schedule, aligning it to the factory production, and also showing the cost of holding excess WIP. To help allow for the shipping schedule to match up, you could add a calendar input to select when shipments for a customer/distribution hub should start. This tab and the Investment tab synced up is a nice touch and it really gives that macro-decision-making perspective that would help inform concepts of Capacity Planning.
- **Sidebars –** outside the individual tabs, the biggest concern was the use of color for the tabs. Keeping such a restrictive color pallet definitely helps make the theme more cohesive, but this comes at the cost of making unrelated things look related. There were too many times when he needed to pause and confirm what color meant. He doesn't really have the solution, but we might be trying too much to keep everything similar, to the point that it is jeopardizing the importance of distinguishing things. We should either expand our colors for the various elements/workstations or find another way to distinguish them. The question of what the best throughput is to show, operational or total, isn't one with a singular answer; it may be good to show both or distinguish how long a first unit takes to build. I explained the intent to modify schedule to show these better, and he said that clarifying the form of throughput show in the sidebar vs. schedule would be worth considering. Ultimately, both are valid ways to articulate throughput.
- **Overall –** Pedro said that this tool is absolutely mind-blowing, he has never seen something of this caliber for the subject before—and it really showcases the value that Data Visualizations has for a subject which is so abstract and filled with so many variables and concepts. They haven't had very many students in the MEM program choose to pursue the data analytics route, so while courses like Data Visualizations are listed as approved electives, they haven't really had students who have leveraged these CS principles to support their course material before Last semester, I had given one of the most incredible papers he had read about line-structuring, and developed innovative algorithmic approaches in a way that no student had previously. This tool takes it to a whole new level, showing just how powerful CS can be for Industrial/Systems Engineering; to be able to solve and explore the design space in a powerful way. He asked a variety of questions about the coding structure and reflected on how he wished he had learned more coding for things like this. He is very impressed with the depth and comprehensiveness this website gives to subjects of Factory Physics, Capacity Planning, Economic Analysis, and Operations Research. He asked if, when the tool is fully completed, can he use it to support his course on Production and Service Design, as the concepts of the course are abstract in a way which many students struggle to understand. He feels like this tool would greatly supplement his course. He is utterly blown away at just how much this site can do, and how well it is put together and designed from just a few weeks of development.

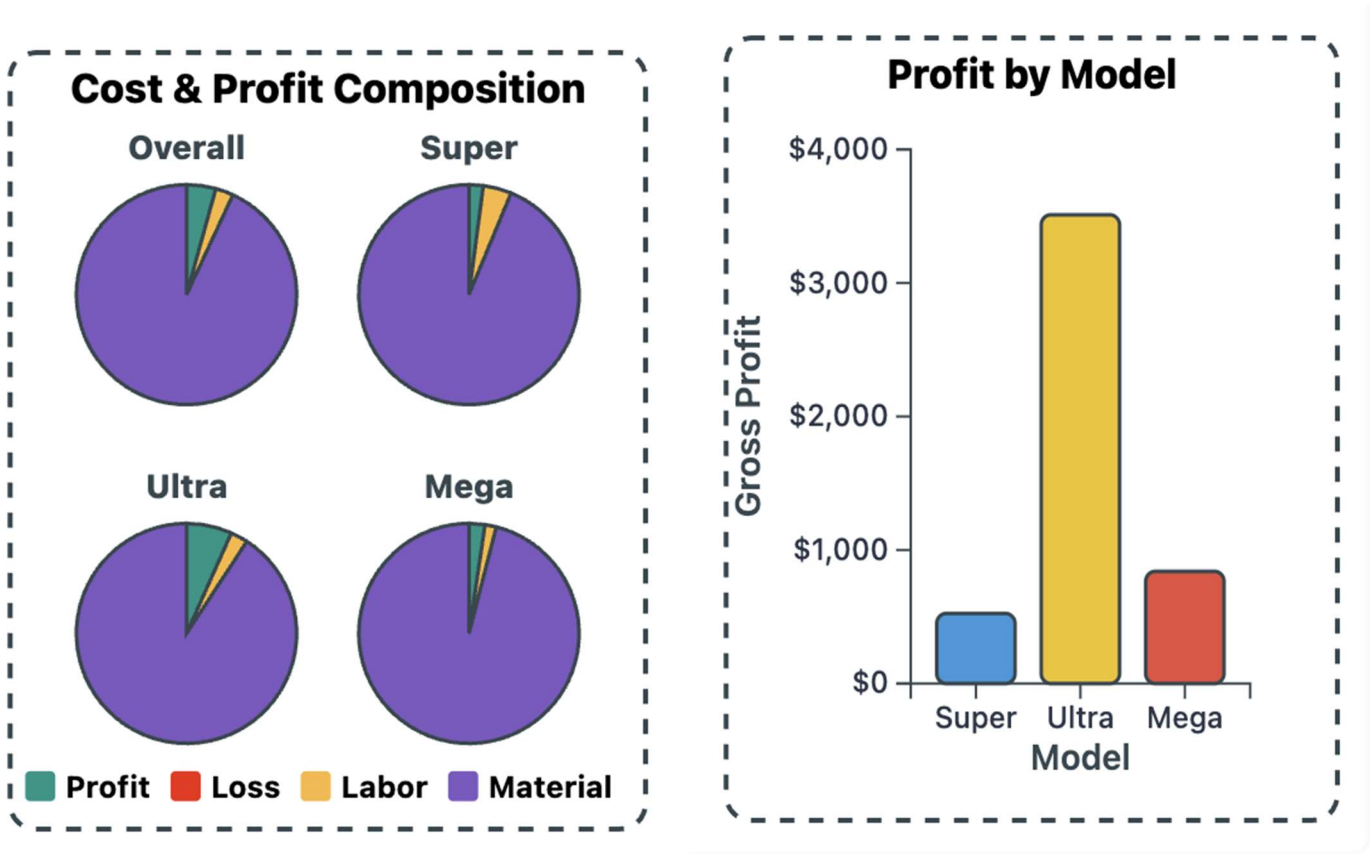
Profit Legend Modifications

Date: *October 13-14th, 2025 {Sumaiya}*

- Pie charts (Cost & Profit Composition): I added a subtle outline to every slice using our accent color with a 1.5px stroke. It's the same treatment that we have used on the Efficiency tab's "Work/Idle" pies, so the visual language is consistent across tabs and the slices read cleanly against light or busy backgrounds.

- Bar chart (Profit by Model): I gave each bar the same accent outline at 1.5px. This adds separation between bars and the dashed panel border, improves contrast for accessibility, and matches the look of the Efficiency tab's idle-time bars.

We used key shapes (pies, bars, box plot) already with a thin accent stroke—about 1.5–1.8px—to provide definition without stealing attention from the fills. I mirrored that pattern here to keep a unified design system: edges are crisp, colors remain the hero, and the entire dashboard feels coherent end-to-end. This approach keeps the charts clear and cohesive, providing visual contrast without overpowering the fill colors, and maintains design consistency across all tabs. Added nicer spacing between rows of the legend.



Zoom in Precedence

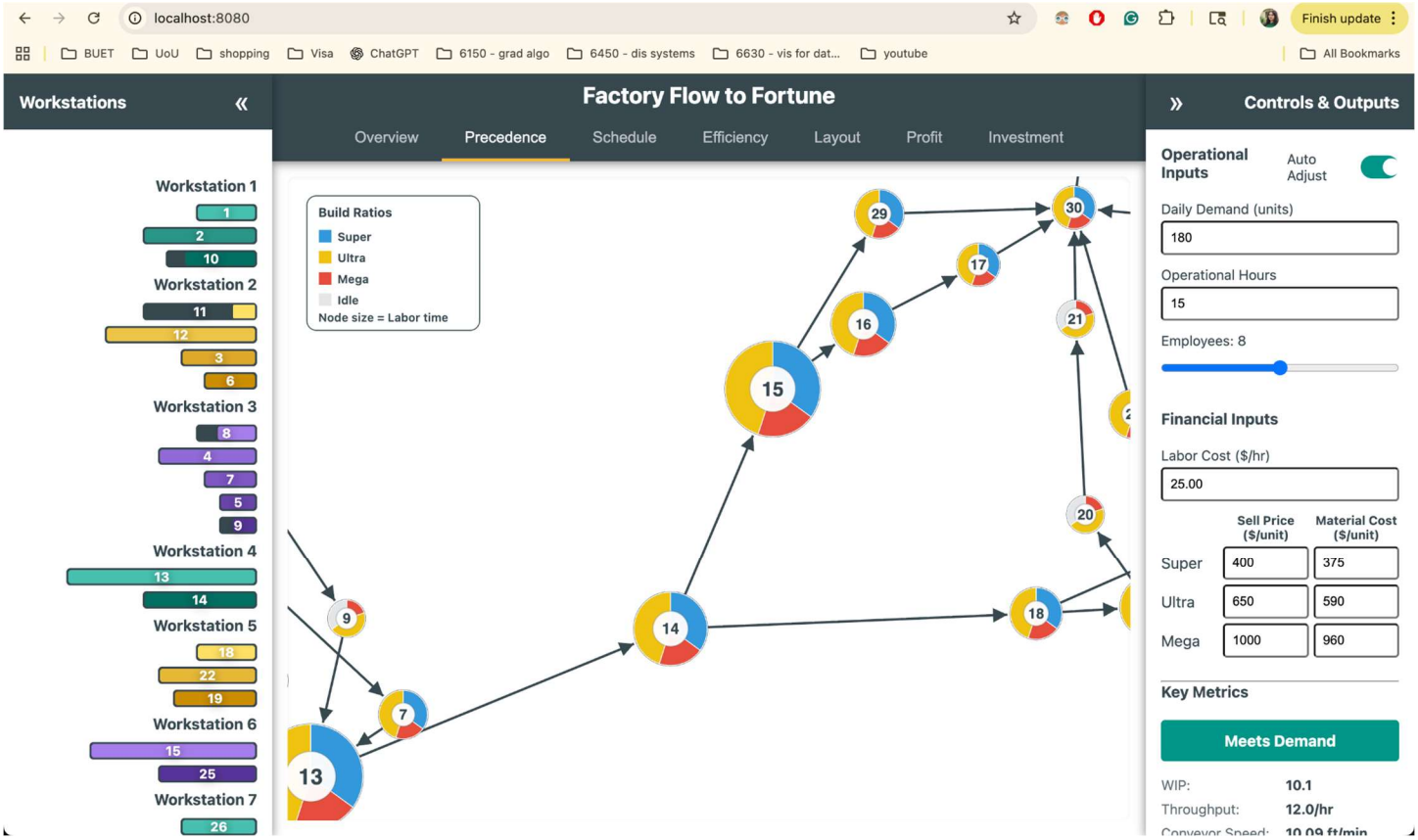
Date: *October 15-16th, 2025 {Sumaiya}*

Now when we first load the precedence tab it looks like the screenshot attached in the following section. It has several issues. First of all, the default zoom out option is not working and maximum amount of nodes are out of the screen and the node relations are not clear from this picture. In the older version, zoom was applied directly to the SVG without separation between the viewport and interaction surface.

Zooming & Panning

- Dedicated zoom catcher behind the graph
    - Added a transparent rect (zoomPane) underneath all visual elements that captures zoom and pan gestures, ensuring smooth and consistent behavior even in empty spaces.
    - Now users can zoom/pan even when they're not directly on a node or link, which makes navigation feel much more forgiving.
- Explicit zoom limits + a sensible default view

    - Zoom is clamped between 10% and 800%([0.1, 8]) of scale. This limits infinite zooming in or out. The main group (mainGroup) receives transformations through D3's zoom handler to maintain proper node positioning inside the coordinate space.
    - On load, the chart auto-zooms out (default scale ≈ 0.6) and recenters, so the layout starts comfortably framed instead of filling the entire viewport. Now it is automatically scaled down and centered in the view. This improves the first-view experience.
- ViewBox set to the SVG size
    - The viewBox attribute tied to the actual container's width and height, which ensures proportional rendering at different screen sizes.
    - This makes zooming consistent and keeps the coordinate system stable across different screen sizes.



I introduced a boundary-aware drag system, ensuring the nodes remain visible and constrained within the chart container. I used padding constants (CLAMP_PAD) and node radii to calculate valid coordinate ranges, effectively creating a "safe zone" inside chart edges. Nodes cannot be dragged outside this boundary, keeping the graph visually contained. The drag functions interpret mouse positions through the current zoom transform, allowing accurate dragging regardless of the current zoom or pan offset. This ensures node movement feels consistent whether zoomed in or out. The drag functions now temporarily fix node positions (fx and fy) during motion and release them when dragging ends, balancing fluid interaction and D3's force simulation dynamics. Together, these refinements turn free-floating elements into controllable, bounded objects, improving both usability and visual organization.

Legend layout & placement

- Legend moved to the bottom-right corner
    - Instead of living at top-left, the legend now anchors to the lower-right with a small margin.
    - Now it stays out of the way of the main graph.

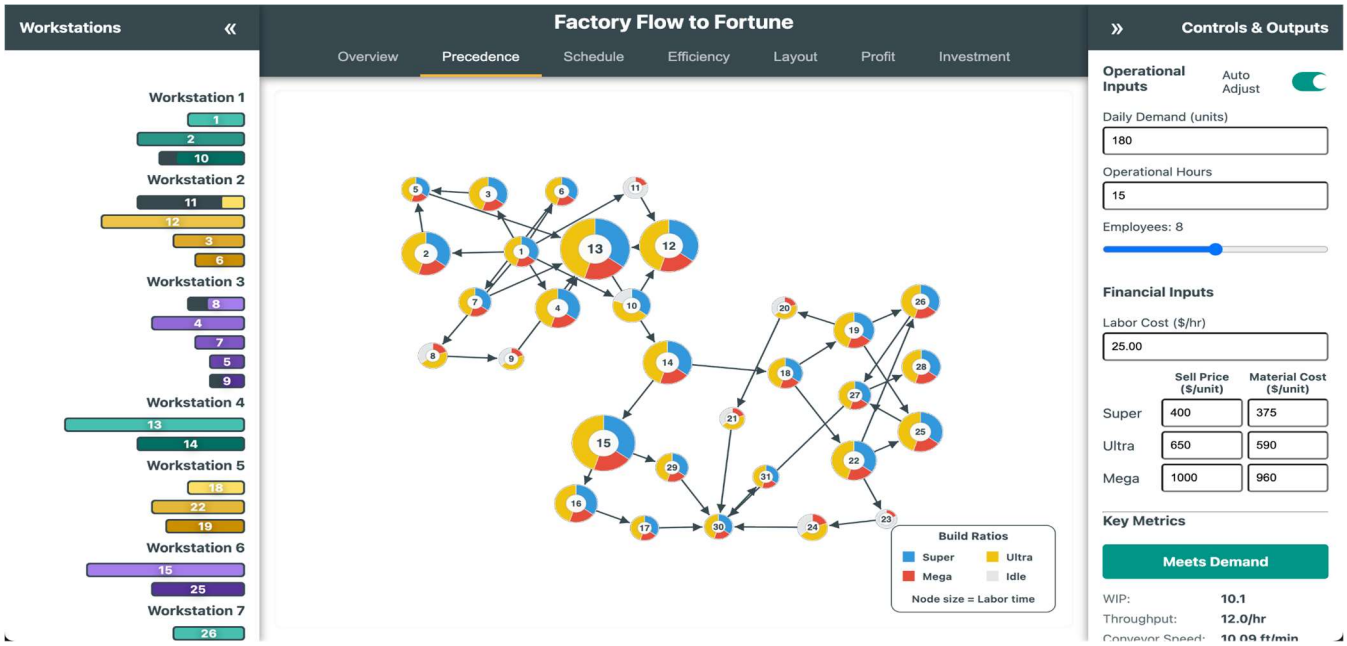
- Centered headings and captions
    - “Build Ratios” (title) and “Node size = Labor time” (caption) are center-aligned inside the legend, improving readability and symmetry.
    - Compact 2×2 grid for color keys

Instead of a vertical list, items appear in a 2×2 grid, visually balancing color swatches and labels in two columns (“Super”, “Ultra” on the top row; “Mega”, “Idle” below). The color labels (Super, Ultra, Mega, Idle) are displayed in a 2 by 2 grid, giving a tighter, more balanced block. The inter-item spacing is tuned to reduce empty space (“too much pacing”), so the card looks denser and more polished.

Dragging behavior & boundaries

- Drag works correctly at any zoom level
    - During drag, pointer positions are converted from screen space to graph space.
    - Nodes track the cursor precisely even when zoomed or panned—no “drift” or jumpiness.
- Hard boundary clamping
    - Nodes are clamped to stay inside visible chart area on each simulation tick and during dragging.
    - Now nodes won’t slip off screen, and you can’t drag them out of bounds.

Now, the navigation is smoother (zoom catcher, default zoom out, bounded zoom). The legend is cleaner (bottom-right, centered texts, 2×2 keys, tighter spacing). Drag & drop is reliable at any zoom and keeps nodes inside. Links and nodes read cleaner (arrows end at circle edges; labels remain bold and legible).



Actioning Meeting Notes

Date: *October 16th, 2025 {Dhruv}*

Overview and Motivation

This session focused on implementing a series of targeted UI and visual adjustments across the **Schedule**, **Efficiency**, and **Layout** tabs, based on collaborative feedback from the most recent team meeting. The primary goals were to improve visual clarity, reinforce thematic consistency across tabs, and enhance the interpretability of production data through subtle color and layout refinements.

Collectively, these changes refined the overall user experience—making the application more readable, balanced, and aligned with the team's visual direction.

Related Work

The updates build upon prior color system integration, responsive layout adjustments, and animation refinements. They extend those improvements with color tuning for emphasis, positional adjustments for better spatial hierarchy, and visual polishing to align the Factory Flow to Fortune dashboard with professional manufacturing analytics standards.

Questions

- How can product-related data be made more visually distinct without overwhelming the layout?
- What adjustments ensure that charts and animations remain legible and properly aligned across screen resolutions?
- How can interface components reflect hierarchy and interaction logic through color and position while maintaining overall design consistency?

Data

No modifications were made to data handling or computational logic. All updates focused on presentation, layout positioning, and visual encoding to better communicate existing data structures.

Design Evolution

1. Schedule Tab Refinements

Problem: The Schedule tab needed improved hierarchy and clearer product differentiation, particularly around the product counters, timeline controls, and model visualization.

Solutions:

- **Product Counter Emphasis:** Added a **semi-transparent background** behind the product counter group, improving contrast and visibility against the chart background.
- **Counter and Filter Repositioning:** Moved the **product counter and checkboxes** to the **left of the legend** for a more balanced layout that follows the natural visual reading order.
- **Stacked Timeline Controls:** Reorganized the playback and timeline control buttons into a **vertical stack**, rather than a single horizontal row, improving usability and preserving space at smaller viewport widths.
- **Timeline Completion Logic:** Fixed a logic issue causing the product counters to lag one update behind. The counter now accurately registers the **last finished product** when the simulation completes.
- **Cycle and Process Time Metrics:** Added two new production performance indicators:
    - **Cycle Time** – the average time between consecutive models completing the line.
    - **Process Time** – the total time required for a specific product type to move through all workstations.
      These metrics provide deeper insight into production rhythm and throughput efficiency.

Outcome: The Schedule tab now communicates production timing and performance more intuitively, with improved alignment, visibility, and contextual metrics.

2. Overall Color Adjustments

Problem: The colors assigned to product types (Super, Ultra, Mega) lacked enough visual separation, and their saturation levels made them blend into the visualization background.

Solution: The product colors were adjusted to **lighter, pastel variations**, maintaining hue identity but reducing saturation.

- The more subdued palette makes each model type easier to distinguish without visually dominating the chart.
- This adjustment also improves contrast with text and overlay elements, especially against the neutral backgrounds used throughout the interface.

Outcome: The dashboard now exhibits a more cohesive and professional color balance, improving both aesthetics and interpretability.

3. Efficiency Tab Refinements

Problem: The Efficiency tab contained several minor issues affecting readability and design harmony, including color imbalance in the bar chart, overlapping text at smaller screen sizes, and minor inconsistencies in labeling and clock visuals.

Solutions:

- **Median Bar Highlight:** Changed the **median bar color** to a more distinctive tone, improving visual emphasis on central efficiency trends.
- **Bar Chart Correction:** Restored the **previous bar chart implementation** from an earlier version, removing the collapsing/rising bar animation to improve clarity and reduce distraction during updates.
- **Text Alignment:** Adjusted the positioning of text below the clocks, shifting it **slightly downward** to prevent overlap with clock graphics on lower resolutions.
- **Label Consistency:** Corrected text capitalization from “idle” to “Idle” for uniformity across labels and metrics.
- **Clock Color Updates:**
    - Clock face now uses **--accent** for its border.
    - Clock tick marks use **--idle-color** for improved contrast.
    - Clock hands now use **--secondary1** (the violet secondary color, #7E57C2) to align with the overall color hierarchy.

Outcome: The Efficiency tab now feels more balanced and readable, with improved emphasis on key values and consistent adherence to the updated color system.

4. Layout Tab Refinements

Problem: The Layout tab’s speedometer design required improved readability and visual harmony with the updated global color palette.

Solution:

- **Speedometer Color Update:** Swapped the color assignments for --primary and --secondary1, using violet (#7E57C2) as the new **primary highlight** for the gauge needle and teal (#009688) for supporting frame elements.
- **Gradient Enhancement:** Introduced a **gradient fill** to the speedometer, providing a subtle depth effect and better dynamic feedback as the needle moves.

Outcome: The Layout tab now features a more visually engaging speedometer that integrates smoothly with the broader color palette, conveying speed changes with greater visual clarity and aesthetic refinement.

Implementation

- **Files Modified:** script.js, style.css
- **Key Updates:**
    - Schedule tab layout repositioning for counters, controls, and legend.
    - Added background transparency and completion logic for product counters.
    - Implemented new metrics: cycle time and process time.
    - Updated product color variables to pastel tones.
    - Adjusted median bar, clock visuals, and typography in Efficiency tab.
    - Updated gradient and color assignments for Layout tab's speedometer.

Evaluation

- **Visual Clarity:** Pastel colors and improved element placement enhance readability and distinguish key data points.
- **User Experience:** Timeline and control adjustments streamline navigation and highlight meaningful production metrics.
- **Consistency:** Color and typography updates align the interface across all tabs for a unified look and feel.
- **Performance Insight:** The addition of cycle and process time metrics gives users deeper visibility into production efficiency.
- **Aesthetic Cohesion:** The gradient-based Layout tab and refined Efficiency visuals provide a polished, modern aesthetic consistent with the system's evolving design standards.

Session Outcomes

This session delivered a suite of refinements that collectively improved both the usability and visual appeal of the Factory Flow to Fortune application. Through pastel color tuning, improved layout hierarchy, clearer metric visualization, and the addition of contextual production insights, the interface now presents data with greater balance, precision, and clarity. These updates reflect a successful collaboration between design and development efforts, ensuring the system continues to evolve toward a professional, cohesive user experience.

Precedence Node Anchoring and Force Adjustments

Date: *October 17-18th, 2025 {Sumaiya}*

1. Arrow Lengths Have a Higher Minimum - Arrow (link) lengths are controlled by:

.force("link", d3.forceLink(links).id(d => d.id).distance(40))

- Here's the mechanism: The .distance(40) sets a minimum distance between nodes connected by arrows.
- To make arrows longer, you increase this value (try 60 or higher).
- This ensures the arrows won't be too short and crammed, making the graph readable.

That's a fixed target distance that's larger than D3's default, so links naturally try to space nodes farther apart. In the simulation tick I trimmed arrow endpoints to the circle edge (targetRadius = (d.target.r || 12) + 3), so arrows never "shrink" into nodes; they always look like they have a bit of length. So now the arrows stabilize with a longer visual minimum because the layout wants ~40px, and the endpoint trimming preserves a visible segment even when nodes move closer.

2. PERT Chart Resizing for Different Resolutions – The code responds to dynamic container sizes:

- The chart gets the latest container width/height and sets the SVG's viewBox accordingly.
- This ensures resizing works, as all positions and scaling are based on width and height.
- The layout and zoom logic also use these values, so the chart fits the available space.

I made the SVG responsive—the browser can scale the SVG to any CSS size without distorting coordinates. All positions, forces, and clamping use those measured width/height values, so node layout respects the real canvas size. I applied a zoom transform (with d3.zoom) and set a **default scale** (DEFAULT_ZOOM = 0.7) centered via a translate+scale. That keeps things readable on large/small canvases. Now, The chart sizes itself to the current container, and the viewBox + zoom combo maintains consistent proportions across resolutions.

3. Anchoring Nodes

This code specifically does not anchor first/last nodes with fixed coordinates, but it allows to do so by setting d.fx and d.fy for those nodes, and clamps their values if needed. Typically, to anchor nodes, I could set the fx and fy to desired (x, y). The clamping ensures that fixed nodes stay within the SVG bounds.Now, Nodes are not hard-anchored to fixed positions. The simulation uses link/charge/center/collide forces and viewport clamping, but there's no explicit pinning of "first" or "last" nodes. Dragging pins an individual node at its dropped spot (d.fx = d.x; d.fy = d.y in dragended), which effectively anchors nodes user manually place.Now,it is possible to anchor any node by dragging it.

4. Page Borders Repel More

- Border repulsion is handled by clamping positions every simulation tick:
- This means nodes cannot go outside the visible area of the SVG; they "bounce" or stay in bounds.
- Adjusting CLAMP_PAD (making it larger) increases the border area that nodes cannot enter, hence making border repulsion stronger.

A collision force keeps nodes from overlapping: d3.forceCollide().radius(d => (d.r || 50) + 8). On every tick, clampped node positions to the viewport (clampToViewport) using the node radius and a padding (CLAMP_PAD), and clampped fixed positions too (d.fx, d.fy). During dragging, I also clamp the dragged coordinates. Nodes get "pushed back" at the boundaries and never drift off-screen—both during the simulation and while dragging.

5. Legend Layout: 2x2 with Centered Title

- Legend is rendered as a 2x2 grid, with the title centered:
- Each row and column is calculated and rendered, resulting in a 2x2 layout.

The legend title is positioned at the center (horizontal middle).

It centers the title by placing it at centerX = legendWidth/2 with text-anchor="middle".
It positions the legend in the bottom-right with a margin, independent of node layout.

Schedule Templating Adjustments

Date: *October 17th, 2025 {Dhruv}*

Overview and Motivation

This session focused on refining the spatial layout and visual layering of the **Schedule visualization**, specifically addressing issues with the timeline alignment and the visibility of overlapping elements below it. The main objectives were to **lower the timeline** for better visual balance and to implement a **clipping mask** solution that restricts chart rendering to the main visualization area while preserving the visibility of key interface components such as the timeline and time markers.
These adjustments resolved lingering compositional issues and improved both the visual polish and clarity of the application's layout.

Related Work

This update builds upon earlier layout refinements from previous sessions that introduced the product counters, legends, and improved positioning of timeline controls within the Schedule tab. The clipping mask discussion originated in the team's prior debugging conversation, where improper layering and uncontrolled overflow from the left sidebar elements were visually interfering with the main chart area.

Questions

- How can the schedule's timeline be repositioned to achieve better visual hierarchy and alignment with the rest of the interface?
- What method can be used to clip or mask elements extending below the timeline without covering essential chart features like the time axis or markers?
- How can the sidebar and timeline transitions remain smooth when animated clipping is applied?

Data

No new data was introduced or modified in this session. All work centered on SVG layering, mask logic, and visual structure within the **Schedule tab**.

Exploratory Analysis

Before this session, the Schedule timeline appeared too close to other elements, creating visual tension and clutter near the top of the chart. Moreover, graphical elements from the Gantt chart occasionally extended beyond the intended viewing boundary, overlapping with both the timeline and the left sidebar content.
The initial attempts to use a static rectangular overlay resulted in unwanted occlusion—particularly over timeline markers. During the debugging conversation, the challenge was clarified: the goal was **to cover only the content below the timeline**, leaving the timeline and markers untouched.

Design Evolution

1. Timeline Position Adjustment

Problem: The timeline was positioned too high in the chart area, causing tight spacing between the header and the start of the Gantt visualization.
Solution:

- Lowered the timeline slightly to establish clearer separation between the timeline and the title elements.
- Adjusted SVG group translations and scaling to ensure proportional spacing between the timeline, Gantt rows, and the summary region below.
  Outcome: The timeline now aligns visually with other major interface elements, improving balance and readability.

2. Clipping Mask Implementation for Main Visualization

Problem: Chart elements were rendering outside their intended boundaries, visually colliding with sidebar components and timeline elements.
Solution: Implemented an **SVG clipping mask** on the main visualization area to restrict rendering to the visible bounds of the chart.

- Created a clipping path applied to all **Gantt chart SVG elements** that dynamically matches the viewport of the main-content region.
- The clip is animated in synchronization with the timeline's update transitions, ensuring seamless entry and exit as the chart renders.
- This ensures that only elements within the designated visualization area are visible, maintaining a clean edge beneath the timeline.
  Outcome: All schedule elements now render within a controlled viewport, eliminating visual overflow and ensuring a professional, polished layout.

3. Sidebar Coverage with Animated White Rectangle

Problem: The left sidebar occasionally appeared through the clipping region due to separate SVG layering.
Solution: Introduced a **white rectangle overlay** positioned above the left sidebar region but below the timeline layer.

- The rectangle covers the left sidebar edge and the region immediately beneath the timeline.
- It is animated to transition smoothly with the rest of the visualization, maintaining alignment during zoom and pan events.
- This approach provides the visual effect of a continuous boundary without altering the structure of sidebar components.
  Outcome: The visual continuity between the main-content area and the left sidebar is fully restored. The white rectangle effectively "masks" undesired sidebar bleed while preserving animation flow and performance.

Implementation

- **Files Modified:** script.js (Schedule tab rendering logic) and style.css (minor layering adjustments).
- **Core Additions:**
  - Introduced dynamic SVG clipping mask applied to the main-content visualization group.
  - Added animated white rectangle overlay to the left boundary for sidebar coverage.
  - Adjusted timeline translation and scaling for improved vertical balance.
  - Verified that both the clipping mask and overlay animations synchronize with the chart's render loop and pan/zoom interactions.

Evaluation

- **Visual Clarity:** The schedule visualization now displays with clear separation between the timeline and chart elements, free of overlapping or extraneous content.
- **Professional Finish:** The clipping mask provides a clean edge, reinforcing the structural boundaries of the dashboard.
- **Responsiveness:** Both the mask and the sidebar overlay animate fluidly during chart updates and viewport adjustments.
- **Layout Balance:** The lowered timeline and refined spacing create a more readable and visually stable presentation.
- **Maintainability:** The mask and overlay logic are modular and easily adjustable if future layout changes require resizing or repositioning.

Session Outcomes

This session successfully resolved the timeline overlap and sidebar clipping issues that had persisted through earlier iterations. By combining a **precisely applied SVG clipping path** for the main visualization with an **animated white overlay** for the sidebar boundary, the application achieved both functional cleanliness and aesthetic cohesion.
The Schedule visualization now maintains visual integrity during zooming, panning, and animated updates, completing one of the key UI refinement goals identified in the team's previous discussions.

Simulation Synchronization

Date: *October 18th, 2025 {Dhruv}*

Overview and Motivation

This session focused on improving the **synchronization, usability, and visual consistency** of the Factory Flow to Fortune interface, particularly refining the simulation experience across the **Layout** and **Schedule** tabs.
The main objectives were to correct discrepancies in time calculations between visualizations, enhance control over simulation playback, and standardize legend presentation across all tabs.
These updates improve both the technical precision of the application and the clarity of communication in how users interpret visual cues during simulations.

Related Work

This build extends previous sessions that focused on animation responsiveness, layout clipping, and control interactivity. The refinements made here ensure that each visualization component — from the moving products in the Layout tab to the Gantt chart in the Schedule — now behaves predictably and shares a unified design system.

Questions

- How can simulation playback be made more user-friendly while maintaining visual clarity and temporal accuracy?
- What adjustments are needed to synchronize time calculations across tabs that share overlapping simulation logic?

- How can legends, colors, and symbols be standardized across visualizations for consistent user interpretation?

Data

No new datasets were introduced. Updates centered on **refining simulation logic**, **layout scaling behavior**, and **UI consistency** across tabs.

Design Evolution

1. Layout Tab Enhancements

Problem: The Layout tab's internal time tracking did not fully align with the Schedule tab's, leading to mismatched completion and progress behavior. Additionally, UI scaling and legend clarity required refinement.

Solutions:

- **Time Synchronization:** Recalibrated the Layout tab's internal time calculation to match the Schedule tab's simulation logic. Product flow, animation timing, and finished goods completion are now fully synchronized across both tabs.
- **Responsive Element Scaling:** Adjusted SVG element sizing logic to dynamically resize based on window resolution. This ensures that all layout elements, including assembly lines, bins, and product icons, remain proportionally consistent on various display sizes.
- **Legend Stability Fix:** Resolved a bug where **product icons in the legend** were inadvertently deleted when the **reset button under the clock** was pressed. The legend now persists correctly throughout all animation states.
- **Skip-to-End Functionality:** Introduced a new **"Skip to End" button** that allows users to instantly view the finished goods bin without waiting for the full simulation to play out. This feature streamlines analysis and is especially useful for quick demonstrations.
- **Legend Enhancement:** Added a **gray fill indicator** in the legend to represent elements not used by a given model. This improvement clarifies assembly step applicability for each model type.
- **Icon Update:** Replaced the textual "Grid" label in the legend with a **square icon**, improving visual coherence and aligning with other symbolic legend elements.

Outcome: The Layout tab now operates with precise timing alignment, improved scalability, and a more informative legend. The addition of a skip control enhances simulation interactivity, while bug fixes ensure reliability during resets and restarts.

2. Schedule Tab Refinements

Problem: The bottleneck indicator and timeline navigation behaviors required better visual communication. The build sequence also needed exportability for analysis outside the application.

Solutions:

- **Expanded Bottleneck Highlighting:** Extended the red bottleneck overlay to cover the **entire workstation width** instead of just a single task bar. This adjustment ensures that users can clearly see which workstation is acting as the limiting factor for production throughput.
- **Smooth Timeline Navigation:** Replaced the **instant jump** behavior during timeline click navigation with a **fast transition animation**. This subtle temporal easing makes it visually clear that the user is "moving forward" in time, rather than triggering an abrupt view change.

- **Build Sequence Export:** Added a new **Export button** that generates a **comprehensive build sequence list**, detailing the order of models and their entry/exit times on the line. This feature enables further analysis of production order efficiency and task sequencing outside the visualization environment.

Outcome: The Schedule tab now delivers a smoother, more intuitive navigation experience and enables data export for external review. The extended bottleneck visualization provides a stronger, more immediate understanding of line constraints.

3. Global Updates – Legend Standardization

Problem: Legends across different tabs had diverging visual styles and structures, creating inconsistency in color mapping, spacing, and typography.

Solution:

- Implemented a **standardized legend CSS system** that defines shared styling variables and layout rules across all visualizations.
- Unified padding, font size, alignment, and symbol spacing under a single set of CSS rules.
- Ensured consistent use of color tokens for product types, idle/active indicators, and model-specific markings across all tabs.

Outcome: Legends now share a cohesive visual language across the application, improving readability and reinforcing design consistency.

Implementation

- **Files Modified:** script.js, style.css
- **Key Updates:**
    - Layout tab: time synchronization with Schedule, responsive scaling, legend fix, skip-to-end button, gray fill indicator, and icon replacement.
    - Schedule tab: extended bottleneck coloring, animated timeline transitions, and build sequence export functionality.
    - Global: standardized legend styling in CSS to unify all visual components.

Evaluation

- **Performance Alignment:** The Layout and Schedule tabs now share synchronized timing and consistent simulation pacing.
- **Usability:** The Skip-to-End and animated navigation features make exploring results more intuitive and efficient.
- **Clarity:** Expanded bottleneck visualization and updated legends reinforce production insights at a glance.
- **Consistency:** Unified legends and visual scaling provide a cohesive user experience across different modules.
- **Reliability:** The legend deletion bug fix and stable reset logic ensure predictable behavior across repeated simulations.

Session Outcomes

This session achieved a critical milestone in harmonizing the simulation experience across the application. By synchronizing timing, refining animations, and standardizing legends, the interface now offers a **more predictable, accessible, and visually consistent** experience.
Users can navigate, analyze, and interpret simulations with improved confidence, aided by clear bottleneck cues, a responsive layout, and streamlined export capabilities. These updates represent another step toward a fully integrated, professional-grade visualization system for manufacturing analysis.

Implementing Pop-Out Features

Date: *October 19th, 2025 {Dhruv}*

Overview and Motivation

This session focused on improving **visual feedback, bottleneck identification, and UI consistency** across multiple tabs.
The updates aimed to make performance issues more apparent at a glance, ensure proper scaling across different screen resolutions, and refine the behavior of input controls for smoother interactivity.
Collectively, these enhancements improved the clarity, usability, and aesthetic consistency of the Factory Flow to Fortune application.

Related Work

These refinements build upon the bottleneck highlighting and animation improvements introduced in earlier sessions for the **Schedule** and **Efficiency** tabs. The new visual indicators extend that work by providing dynamic emphasis and contextual error states to guide users' interpretation of production data.

Questions

- How can bottleneck workstations be made immediately recognizable without requiring users to interpret efficiency metrics manually?
- What visual and behavioral cues can help distinguish between normal inefficiency and excessive idle time?
- How can input resets and layout responsiveness be made more consistent across all views?

Data

No changes were made to data structures or computations. All updates focused on **UI responsiveness**, **visual feedback**, and **logic-driven error signaling** based on existing performance metrics.

Design Evolution

1. Efficiency Tab Enhancements

Problem: Users had difficulty visually identifying the workstation acting as the bottleneck, especially when multiple stations had minimal idle time. Additionally, blinking clock animations for idle time occasionally caused confusion without clear context.

Solutions:

- **Flashing Bottleneck Highlight:** Implemented a **pulsing highlight animation** for bottleneck workstations—mirroring the dynamic emphasis style used in the Precedence tab.

    - The highlight flashes around the workstation with the **lowest idle time**, drawing immediate attention to the production constraint.
    - The effect is subtle yet visually effective, avoiding distraction while maintaining clear focus.
- **Idle Time Error Indicator:** Introduced a new condition in which the **"Meets Demand" status** displays an **error state** if any workstation exceeds **12 hours of idle time**.
    - This feature provides direct visual feedback to clarify the meaning of blinking clocks or extreme inefficiency.
    - The button now shifts color and label dynamically to "Excessive Idle Time," reinforcing the system's feedback loop.

Outcome:
The Efficiency tab now clearly communicates both underperformance and overcapacity through visual cues. Users can instantly identify bottlenecks and understand the reason behind alerting behaviors, improving diagnostic efficiency and interpretability.

2. Schedule Tab Refinements

Problem: Only the **first** detected bottleneck workstation was being highlighted, leaving additional bottleneck stations unmarked when multiple existed.

Solution:

- Updated the highlighting logic to ensure **all bottleneck stations**—those sharing the minimum idle time threshold—are simultaneously marked with the red overlay.
- The adjustment ensures parity between **Schedule** and **Efficiency** tabs, aligning bottleneck recognition across the interface.

Outcome:
The Schedule visualization now accurately reflects all active bottlenecks, providing a more reliable and complete representation of production constraints.

3. Right Sidebar Fixes

Problem: The **Ctrl+Click reset feature** for input fields occasionally reverted to outdated or incorrect values, particularly after configuration changes.

Solution:

- Corrected the **default value mapping** for all numerical inputs to ensure each field resets to the appropriate baseline value (e.g., demand, operating hours, labor cost).
- Updated the reset handler logic to call the standardized default dictionary directly, ensuring future input additions automatically inherit proper default behavior.

Outcome:
Input fields now reset predictably with **Ctrl+Click**, providing a more stable and user-friendly interaction pattern during configuration adjustments.

4. Global Interface and Scaling Improvements

Problem: The visual layouts of the **Efficiency** and **Schedule** tabs did not scale consistently across screen resolutions, occasionally causing clipping, overlap, or excessive spacing. The background also lacked a cohesive visual identity across tabs.

Solutions:

- **Resolution-Aware Scaling:** Adjusted SVG viewbox parameters and D3 layout calculations to ensure that visual elements rescale smoothly on varying display sizes and aspect ratios.
    - Text, icons, and visual clusters now maintain proportional positioning without distortion.
- **Unified Background Color:** Introduced a soft background tone (#F6FFFF) across the entire application.
    - The new color provides a subtle contrast to charts and interactive panels while maintaining a clean, data-focused aesthetic.
    - It visually ties the application together across all tabs, replacing the plain white background with a cohesive, modern palette.

Outcome:
The interface now scales gracefully across resolutions and has a unified background tone that enhances readability and aesthetic cohesion.

Implementation

- **Files Modified:** script.js, style.css
- **Core Updates:**
    - Efficiency tab: flashing bottleneck highlights; idle-time error feedback logic.
    - Schedule tab: multiple bottleneck highlight fix.
    - Right sidebar: corrected default input values for Ctrl+Click reset.
    - Global: responsive scaling for Efficiency and Schedule tabs; added background color #F6FFFF.

Evaluation

- **Visual Communication:** The flashing bottleneck highlights and idle-time alerts create immediate, intuitive awareness of production constraints.
- **Functional Accuracy:** All bottleneck stations are now consistently marked across visualizations.
- **Interactivity:** The improved input reset behavior reinforces predictable control feedback.
- **Aesthetic Consistency:** Unified scaling and background tone create a clean, professional interface that maintains its layout integrity on any screen size.
- **Usability:** These refinements collectively improve diagnostic efficiency, making the dashboard easier to interpret and operate under different viewing conditions.

Session Outcomes

This session achieved important advancements in **bottleneck visualization**, **error signaling**, and **interface uniformity**. The Efficiency and Schedule tabs now present synchronized and context-rich production feedback, while sidebar controls and layout responsiveness contribute to a more stable, user-friendly system.

Through careful refinement of color, motion, and logic, the Factory Flow to Fortune interface continues to evolve toward a highly intuitive and visually cohesive operational dashboard.

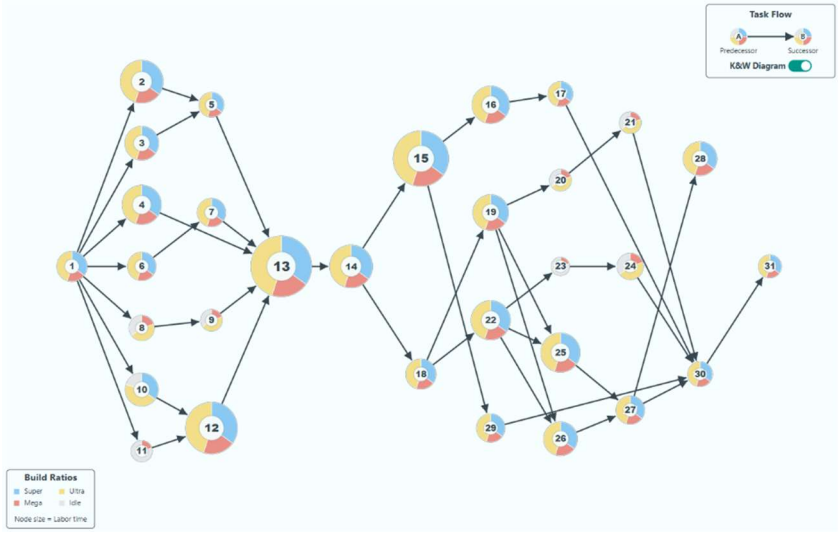
Interview Feedback Implementations

Date: *October 20th, 2025 {Joel}*

Iteration 1: Initial Feature Implementation & Refinement

The session began with straightforward feature requests:

1. **Layout Tab:** Adjusting the order of drawn SVG elements to ensure assembly line components were rendered in the correct visual stack.
2. **Precedence Tab:** Relocating the legend, adjusting the default zoom, and implementing a major new feature: a toggle switch to transform the force-directed graph into a static, column-based Kilbridge & Wester (K&W) diagram.



Iteration 2: Deep Iterative Debugging & State Management

The introduction of the K&W diagram toggle revealed significant underlying bugs related to state management and animation timing within the D3.js library. Our process became a focused debugging loop:

- **The Bug:** The initial implementation caused severe graphical glitches. When the switch was toggled, the nodes (circles) would snap to incorrect positions while the links (lines) animated correctly, creating a disconnected and broken visual. The reversion back to the physics-based layout was also jarring and abrupt.
- **Failed Attempts:** My initial attempts to fix this by simply stopping the simulation or using competing physics forces failed repeatedly. These approaches did not properly manage the conflict between the live physics data (d.x, d.y) and the target animation coordinates, leading to race conditions and unpredictable behavior.
- **The Breakthrough:** The key insight, guided by your detailed solution, was to **decouple the animation from the live simulation.** The final, successful solution involved a "snapshot and tween" method:
    1. **Stop** the physics simulation.
    2. **Snapshot** the exact starting coordinates of every element.
    3. **Animate** each element from its captured start point to its pre-calculated end point using a custom attrTween function.
    4. For the reversion, this process was reversed: animate back to the "snapshot" position first, and *only then* restart the physics simulation.

Iteration 3: Logic Correction and UI Polish

With the core animation stable, we moved to refining the application's main logic and user interface:

- **Auto-Adjust Logic:** You identified a critical flaw in the "Auto Adjust" feature's calculations, which was failing to account for essential variables like line throughput time. The logic was overhauled to mirror the application's primary, correct calculation method.
- **State & UI Bugs:** We fixed several related bugs, including stale error flags that persisted incorrectly after new data was loaded and a faulty input stepper that reset values on click.

- **UI/UX Refinement:** The final stage involved polishing the UI based on your feedback. We created a more intuitive and visually consistent legend, fixed overflowing tooltips with smarter positioning logic, and ensured all interactive elements were laid out responsively.

Sidebar Collapse and Expansion Mechanics

Date: *October 20th, 2025 {Dhruv}*

Overview and Motivation

This session addressed a long-standing issue with the **sidebar expansion and collapse mechanism**, which had been disrupting visualization rendering and responsiveness across several tabs. When the **right sidebar** was toggled, the main content area resized visually, but the underlying SVG elements did not always adjust to fit the new dimensions.
The problem manifested inconsistently—some tabs redrew correctly after a delay, while others failed to respond entirely. The goal for this session was to **ensure all SVG-based visualizations dynamically and smoothly resize** during sidebar transitions, creating a cohesive and fluid user experience.

Related Work

This fix builds upon earlier work on **UI responsiveness** and **viewport scaling**, which improved the interface's adaptability to different resolutions. However, the sidebar toggle behavior remained an exception, requiring deeper synchronization between CSS transitions, D3 redraw logic, and DOM resize events.

Questions

- How can the application synchronize sidebar CSS transitions with dynamic SVG resizing for smooth visual adjustments?
- How can redraws be handled efficiently without clearing live content (e.g., foreignObjects or ongoing simulations)?
- What strategies can ensure consistent behavior across all visualization types—foreignObject-based (Overview), force-directed (Precedence), and animated (Efficiency)?

Design Evolution

1. Sidebar Resizing Problem

Problem: When the right or left sidebar expanded or collapsed, the SVG visualizations inside #main-content did not properly resize.

- In some tabs, resizing was delayed until after the CSS transition.
- In others (e.g., Precedence, Efficiency), the layout failed to adjust at all, resulting in misaligned or cropped visuals.
- Frequent redrawing triggered white flashes or abrupt clearing of foreignObject content (especially in the Overview tab).

Objective: Create a **smooth, non-destructive resize process** that updates all active visualizations seamlessly during sidebar transitions.

2. Smooth Resize Function Implementation (script.js)

Solution: Developed a new smoothResize() function that orchestrates real-time resizing over the sidebar's **300ms CSS transition** window.

This function uses requestAnimationFrame to repeatedly update the active visualization as the layout changes, maintaining fluid visual transitions without white flashes or flickering.

Key Features:

- **Transition Synchronization:** Duration matches the CSS transition time (300ms), ensuring consistent motion.
- **ForeignObject Preservation:** For text-based or mixed-content tabs (Overview, Investment), resizing is handled by adjusting foreignObject dimensions rather than redrawing, preventing content loss.
- **Precedence Tab Integration:** Added a dedicated resize() method to the Precedence tab, allowing the D3 force simulation to re-center nodes and legends dynamically during sidebar transitions.
- **Generalized Rendering Logic:** For all other tabs (Schedule, Efficiency, Layout), the system calls renderActiveTab() for live redraws, ensuring every visualization correctly fits the resized main-content container.

Outcome: The sidebar now resizes smoothly across all tabs. Visualizations update continuously during the animation, eliminating flickering, content loss, or lag between layout states.

3. Efficiency Tab Adjustments

Problem: On resize, the Efficiency tab's workstation group elements were redrawn inconsistently due to incomplete D3 data joins and untracked element positions.

Solution:

- Refactored workstation data binding using **keyed joins** (d => d.id) to ensure proper element tracking during updates.
- Introduced **separate pie and clock subgroups** created only on new elements, preventing redundant redraws during resize.
- Implemented **conditional animation** logic—animated transitions run only on data recalculations, while resizes reposition instantly to maintain performance.
- Added dynamic text sizing for workstation headings, scaling with available space for consistent readability.

Outcome: The Efficiency tab now resizes fluidly without stutter or misalignment. Elements scale proportionally, maintaining their spatial relationships and readability across viewport changes.

4. Precedence Tab Enhancements

Problem: The Precedence tab's D3 simulation used static canvas dimensions, causing layout distortion when the sidebar changed width.

Solution:

- Introduced persistent references for both the svg and simulation objects.
- Implemented a dedicated resize() method to recalculate the SVG's **viewBox**, zoom pane dimensions, and force-center position dynamically.
- Re-centered the node network smoothly and re-positioned the legend based on new viewport bounds (translate(width - 150 - 20, height - 100 - 20)).

Outcome: The Precedence visualization now dynamically re-centers and scales with every sidebar transition, maintaining node spacing and alignment.

5. Sidebar Event Refactor

Problem: The old sidebar toggle handlers relied on transitionend events for redrawing. This approach was brittle and caused inconsistent update timing, as multiple overlapping events could trigger redraw loops or missed updates.

Solution:

- Replaced the event-based redraw system with **direct smoothResize() calls** immediately after toggling.
- Simplified event handlers for both left and right sidebars, improving code readability and reliability.

Outcome: Sidebar toggling is now immediate, synchronized, and fully deterministic—ensuring consistent resizing behavior across all interface states.

Implementation

- **Files Modified:**
    - script.js — Replaced transitionend listeners with smoothResize(); integrated resize handling for all tabs.
    - **Efficiency tab** — Refactored D3 workstation data joins and resize logic for consistent redraws.
    - **Precedence tab** — Added persistent SVG/simulation references and implemented resize() method for dynamic centering.

Core Updates:

- Smooth, frame-based sidebar resize handler with live redrawing.
- Unified behavior across Overview, Precedence, Efficiency, and Schedule tabs.
- Simplified event architecture for improved maintainability.

Evaluation

- **Responsiveness:** Visualizations now resize seamlessly during sidebar animations, with no delay or flicker.
- **Visual Consistency:** Elements scale and reposition uniformly across all tabs.
- **Performance:** Smooth transitions achieved without additional layout thrashing or heavy redraw cycles.
- **Stability:** The removal of transitionend listeners prevents event race conditions and ensures reliable synchronization between CSS and JS.
- **Maintainability:** Centralized resize logic under a single, reusable function simplifies future layout modifications.

Session Outcomes

This session resolved one of the most persistent and complex interface synchronization issues in the project. The introduction of a **frame-synced resize system** successfully unified the behavior of all visualizations during sidebar transitions. The refactored Efficiency and Precedence tabs now dynamically adapt to the available viewport, while other visualizations preserve full fidelity without flashing or lag.

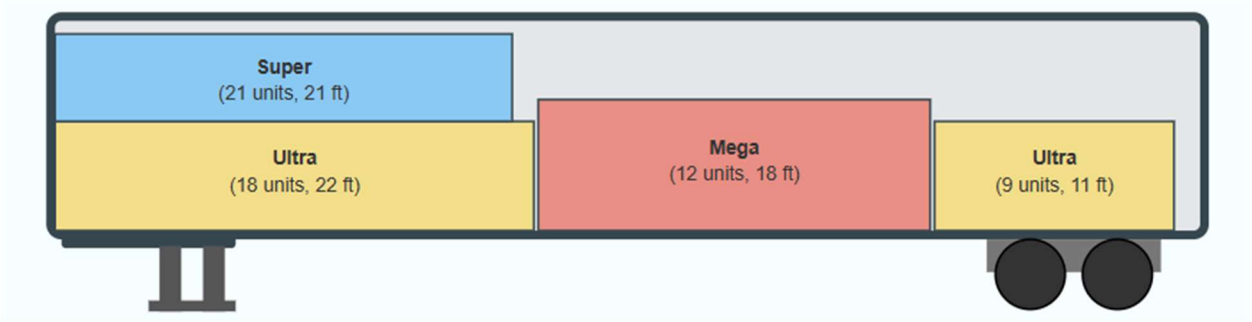
The result is a smoother, more cohesive user experience that reinforces the application's professional design quality and technical polish.

Backend Page Development

Date: *October 21st, 2025 {Joel}*

Activity: Backend page built and placed into overview tab.

This is just an HTML Page which details the problem space, and how the backend calculates everything, most of it is just text, except it does use a table for elements, LaTex for formulas, and it also has a few diagrams drawn, such as the trailer packing.



Milestone Bug Fixes

Date: *October 21st, 2025 {Dhruv}*

Overview and Motivation

This session was primarily dedicated to **bug fixing, behavioral refinement, and interface consistency** across multiple tabs.
Many of the issues addressed were subtle but impactful—ranging from unintended animation restarts to incorrect UI states after sidebar transitions or user interactions.
Additionally, this session marked the **introduction of the new Location tab**, expanding the application's functionality to handle geographic selections and batch city management.

Together, these updates significantly improved the **stability, usability, and extensibility** of the platform.

Related Work

These refinements build directly upon the previous session's sidebar resizing improvements and the recent animation logic updates.
The fixes ensure that the dynamic resizing, data reactivity, and animation timing systems now work together reliably across all tabs, preparing the foundation for the new Location module.

Questions

- How can animations remain visually consistent during layout transitions without resetting progress?
- What mechanisms should control animation pause/resume states to avoid unwanted restarts?
- How can sidebars better synchronize state changes (dragging, scrolling, resizing) across all visualizations?

- What new interface elements are required to support multi-selection functionality in the upcoming Location tab?

Design Evolution

1. Global Animation and Rendering Refinements

Problem:
When the sidebar expanded or collapsed, certain visualizations—especially the Efficiency tab's bar chart—fully redrew, causing bars to reset to zero and reanimate unnecessarily.
Additionally, animations across several tabs (Schedule, Layout, and Efficiency) resumed automatically after being manually paused, leading to inconsistent user control.

Solutions:

- **Bar Chart Stability:** Modified the redraw logic so that when the sidebar is toggled, the bar chart **retains its current animation state** and does not reinitialize from zero. This was achieved by caching the existing bar data and only updating layout transforms rather than re-running transitions.
- **Animation Resume Fix:** Implemented a global check to ensure that **manually paused simulations remain paused** even after tab switches, sidebar interactions, or window resizes. The animation state (isRunning flag) now persists correctly and is respected by all visualization loops.

Outcome:
Animations across all tabs now maintain continuity and respect user interactions. Visualizations remain stable during layout changes, eliminating jarring replays or unintended motion resets.

2. Precedence Tab Updates

Problem:
The **precedence error indicator** in the right sidebar was not updating correctly when a precedence rule was violated, and the main arrow visualization lacked a clear visual cue when errors occurred.

Solutions:

- **Error Indicator Synchronization:** Fixed the sidebar indicator logic to dynamically reflect when a **precedence error is detected** in the workstation order. The display now accurately signals the current validation status.
- **Blinking Arrow Alert:** Added a subtle **blinking animation to the arrow** in the main Precedence diagram when a precedence violation occurs, mirroring the behavior of other alert states (such as the bottleneck highlight).

Outcome:
Users are now provided with clear, consistent feedback when precedence constraints are broken, improving both visual clarity and diagnostic awareness.

3. Schedule Tab Refinements

Problem:
The Schedule tab had several small but disruptive issues affecting usability and layout integrity.

- The first column displayed "Unique ID" instead of a sequential build number.
- The masking rectangle intended to hide elements below the timeline persisted across all tabs instead of being limited to Schedule only.

- The existing timeline controls were functional but visually dense and not easily discoverable.

Solutions:

- **Sequence Numbering:** Replaced the “Unique ID” column with **“Sequence #”**, clarifying the order of model builds.
- **Conditional Masking:** Restricted the masking rectangle’s visibility to the Schedule tab only. It now appears and disappears dynamically based on the active tab state.
- **Controls Panel Addition:** Added a **collapsible control panel or info button** containing all timeline and animation controls, previously positioned at the bottom. This improves layout organization and accessibility.

Outcome:
The Schedule tab now presents production sequence data in a clearer format and provides a more user-friendly control experience, with interface elements that better match user expectations.

4. Right Sidebar Input Corrections

Problem:
Numeric textboxes in the right sidebar occasionally reset to their **minimum values** when using arrow controls, rather than incrementing/decrementing relative to the current input.

Solution:
Corrected the input event logic to properly reference the **current value** before applying adjustments.
Now, arrow keys and stepper buttons modify values incrementally as intended, maintaining expected behavior for all inputs.

Outcome:
The input system now behaves predictably, ensuring accurate and stable data entry without unintended resets.

5. Left Sidebar Adjustments

Problems:

1. After a **precedence failure**, drag-and-drop functionality for rearranging workstation elements was disabled entirely.
2. The **sidebar content** was being visually cut off at the bottom due to the buffer space created by the clipping rectangle.

Solutions:

- **Restored Dragging on Precedence Failure:** Updated the event conditions to allow continued **drag-and-drop interaction** even when precedence errors are present, enabling users to fix issues directly through reordering.
- **Removed Mask and Adjusted Scrolling:** Eliminated the redundant clipping rectangle that was causing cutoff issues. Implemented a **global scroll extension mechanism** instead of restricting it to the Schedule tab only, allowing all tabs to benefit from smoother, consistent sidebar scrolling.

Outcome:
The sidebar now displays all elements properly and maintains full interactivity, regardless of validation errors. Users can seamlessly correct workstation order issues without losing control of the interface.

6. Layout Tab Refinements

Problem:
The Layout tab's **legend and animation controls** were misaligned after resolution changes, particularly on smaller screens.

Solution:
Repositioned and anchored the legend and controls to scale proportionally with the viewport. Adjusted coordinate calculations to ensure consistent placement regardless of resolution or window resizing.

Outcome:
The Layout visualization now scales correctly, preserving visual balance and accessibility across different display sizes.

7. Location Tab – Initial Integration

Overview:
This session also marked the early introduction of the **Location tab**, expanding the application's functionality beyond factory visualization to geographic management.

Features Added:

- **Select All Checkbox:** Allows users to instantly select all cities for inclusion.
- **Individual City Checkboxes:** Enables **multi-city selection** for granular control.
- **Remove All Cities Button:** Provides a one-click option to clear all selected cities.

Outcome:
The Location tab introduces a scalable interface for future geographic data integration. It demonstrates the system's growing versatility and user-centered approach to handling broader operational contexts.

Implementation

- **Files Modified:** script.js, style.css, and tab-specific rendering modules.
- **Key Changes:**
    - Global animation persistence and redraw fixes.
    - Precedence tab error indicator and blinking arrow alerts.
    - Schedule tab layout, mask visibility, and control panel addition.
    - Sidebar input logic and scroll system improvements.
    - Layout tab responsive positioning.
    - New Location tab multi-selection interface and bulk control functionality.

Evaluation

- **Stability:** Fixed multiple cross-tab behavioral bugs that previously caused redraws, value resets, and layout persistence issues.
- **Usability:** Improved control flow and feedback mechanisms, especially within the Schedule and Precedence tabs.
- **Interactivity:** Dragging, scrolling, and sidebar input responsiveness now behave consistently across all tabs.

- **Scalability:** The addition of the Location tab signals the platform's transition toward multi-domain functionality.
- **Performance:** Animation caching and pause-state preservation reduce unnecessary re-renders and enhance fluidity.

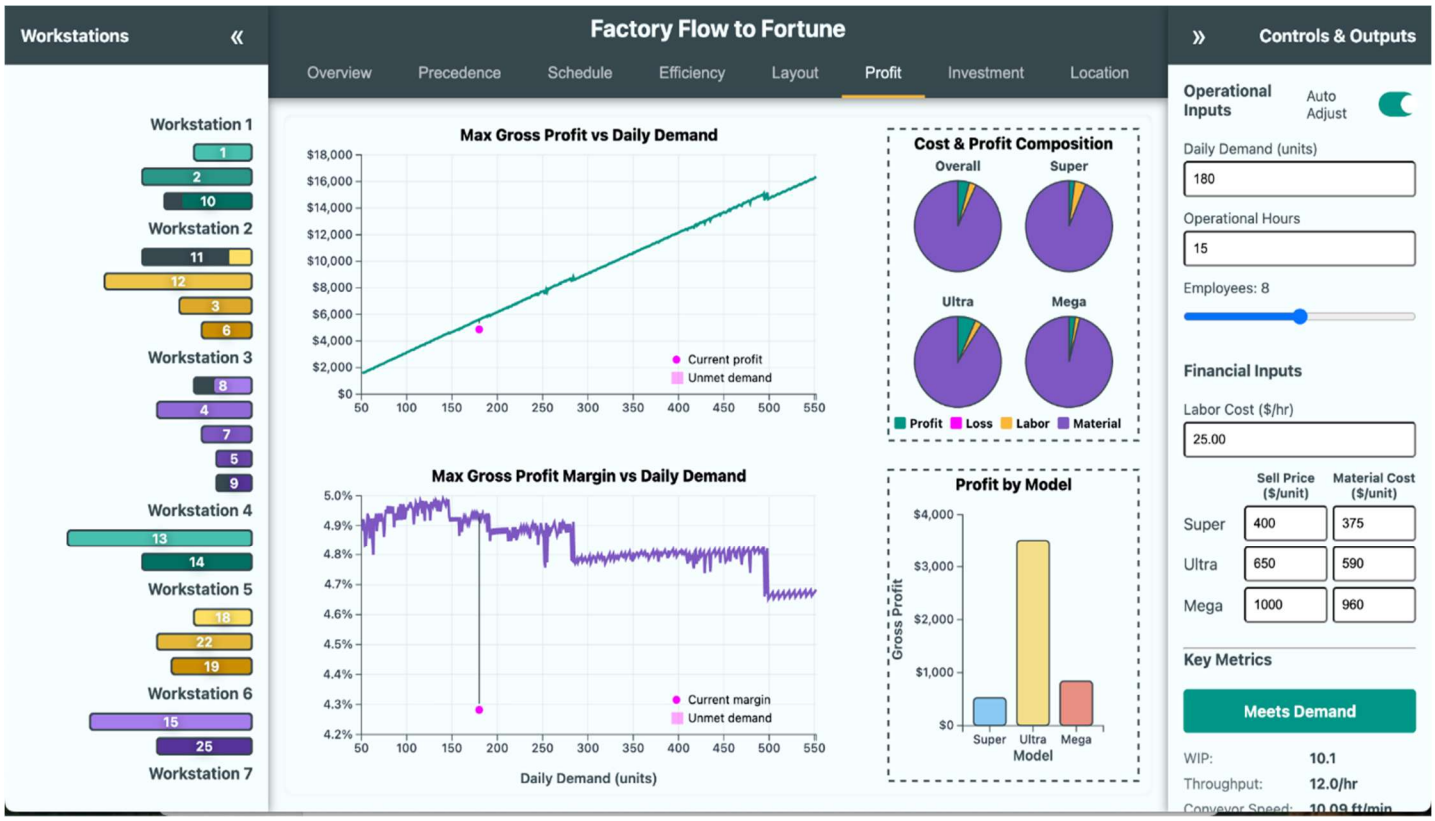
Session Outcomes

This session was a major **stabilization milestone**.
By systematically resolving persistent bugs and polishing the interaction flow across every sidebar and visualization, the application now performs with greater **consistency, clarity, and reliability**.
The introduction of the **Location tab** expands the platform's scope, laying the groundwork for future modules that integrate spatial or logistical dimensions.
With these refinements, Factory Flow to Fortune continues to evolve into a robust, multi-functional visualization system, capable of delivering both analytical precision and an intuitive user experience.

Profit Tab Modifications

Date: *October 22nd, 2025 {Sumaiya}*

I added an explicit "Current Profit" tooltip on the red dot and labeled, in-chart mini legends for both charts ("Current profit/margin" and "Unmet demand"). This makes it obvious what the dot and red area mean without hunting for a help panel. The red polygon (lost-profit area) in both charts now has hover text that explains what it is and why it matters. On the profit chart, the hover tooltip now computes and shows Total Lost Profit (optimal at hovered demand – current config), so users can quantify the gap immediately.
I kept the gridlines and added robust crosshair guides on hover (vertical + horizontal), so it's easier to read exact values against the axes. I fixed the tooltip positioning to use clientX/clientY and set it to position: fixed, which keeps the tooltip stable and aligned with the cursor even if the page scrolls. I kept the multi-row, wrapped legend for the pies but refined the spacing logic (measuring text widths and adding targeted extra gaps). This prevents overlap and keeps the legend compact on narrow layouts. I used consistent color tokens (from CSS variables) for fills, strokes, and accents so the visuals align with the app theme.



K&W Functionality Added to Precedence

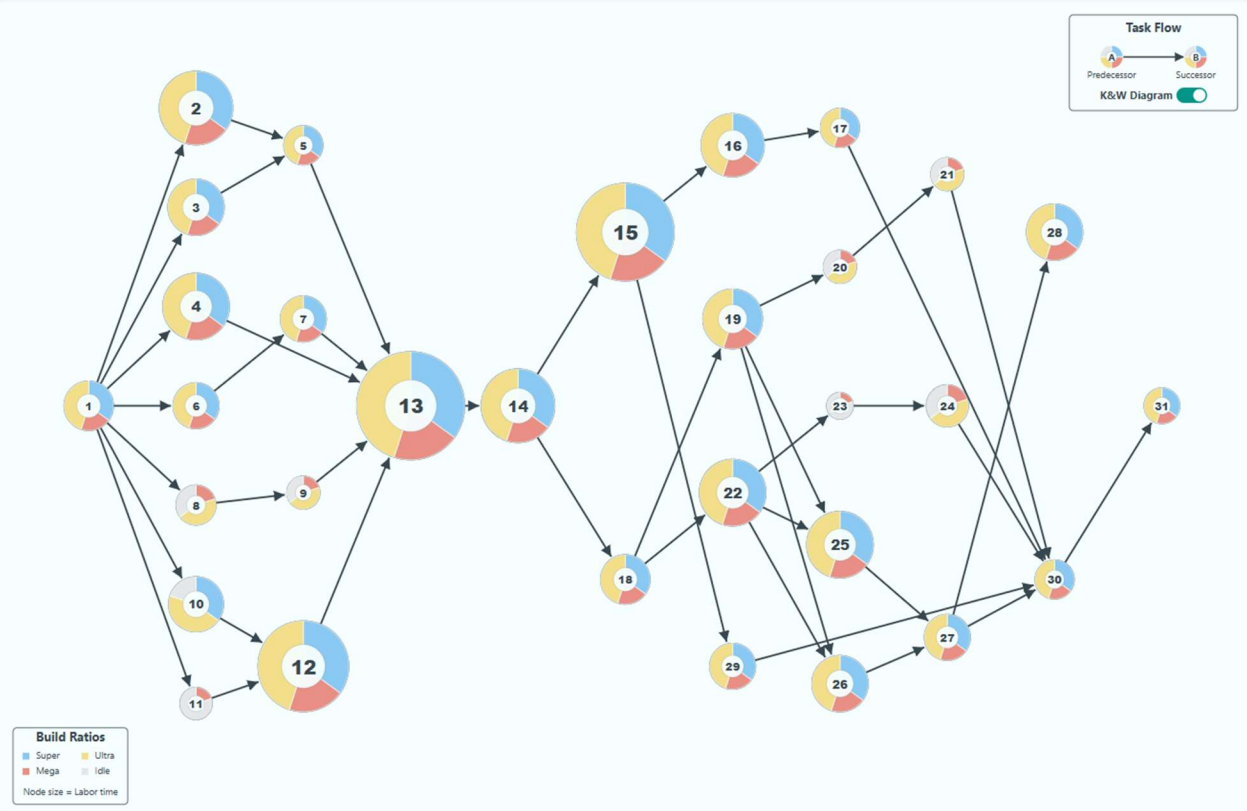
Date: *October 22nd, 2025 {Joel}*

Activity: Addressed precedence flagging glitches, dynamic precedence resizing, and adding the K&W view/legend.

Technical Implementation: Precedence Tab

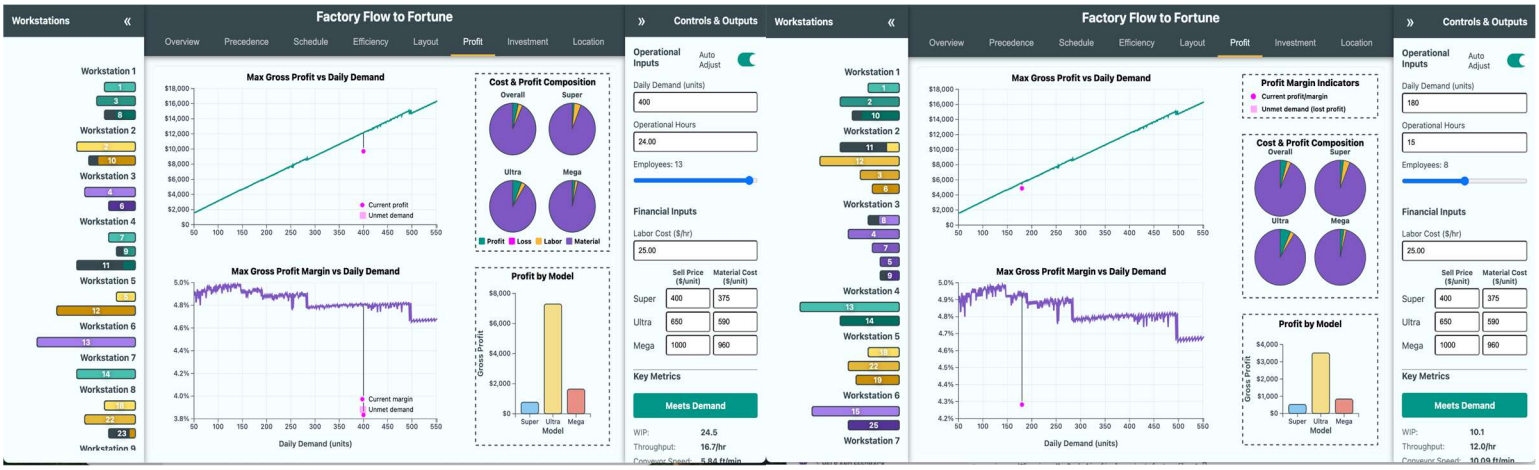
Objective: Enhance the D3.js PrecedenceTab module to ensure the graph remains responsive and restricts user interaction to the visible bounds.

- **Responsive SVG Resizing:**
    - Replaced static width/height attributes with the SVG viewBox attribute to preserve aspect ratio and coordinate systems during scaling.
    - Implemented a handleResize() function that dynamically recalculates the container dimensions (clientWidth, clientHeight).
    - Added a debounced window event listener to trigger handleResize(), which updates the SVG bounds and recenters the D3 physics simulation (d3.forceCenter).
- **Boundary Constrained Dragging:**
    - Updated the dragged event handler within the D3 drag behavior.
    - Added logic to "clamp" the node's position during movement. The logic checks the node's radius (d.r) against the current container width and height, forcing the fx and fy coordinates to stay strictly within the visible area.



Profit Tab Legend Fix

Date: *October 23rd, 2025 {Sumaiya}*



The legends now overlap with the charts. So I did the following for a solution to the above problem

- Moved the legend out of the chart area.

I pulled the “Current profit/margin / Unmet demand” legend into its own right-hand panel (“Profit Margin Indicators”) so it can’t overlap the profit/margin lines. This fixes the clutter you saw in the first screenshot.

- Made the right side a 3-section layout.

I split the right column into top (legend), middle (pies), bottom (bars) and drew borders/titles for each. That gives each element predictable space and keeps the visuals stable at different widths.

- Measured the legend before laying out the rest.

I measure the legend’s true height and then size the right column sections based on that measurement. This prevents surprises where labels push into the pies or bars.

- Tuned the pies and bars to the available space.

I compute the pie radius and bar chart height from the inner panel sizes, so they scale cleanly and never collide with titles or borders.

- Upgraded tooltips and guidance.

I kept tooltips fixed-position (so they stay near the cursor), added clear labels (e.g., “Total Lost Profit”) and crosshair guides. The red “lost-profit” area now explains itself on hover.

- Made “lost profit” on hover when there is unmet demand

I show Total Lost Profit at the hovered demand (optimal profit at that demand minus what the current setup would deliver at met demand). That turns the gap into a number users can act on.

- Kept visual language consistent.

I used the app’s CSS color tokens for fills/strokes, added panel titles, rounded bar corners, and zero lines—so everything looks part of one system.

- Preserved the main charts’ story.

The top chart still anchors on profit vs demand with a “current” marker; the bottom shows margin vs demand. The legend relocation just makes that story unobstructed.

This change was made to fix the overlapping issue where the legend interfered with the main plot, as seen in the first screenshot. By giving the legend its own space to the right, the charts remain clear, readable, and visually separated from explanatory elements. This improves the user experience by making both the graph and the legend more legible and accessible, preventing any important data from being hidden behind interface elements.

Profit Tab Baseline Fix for Bar Chart

Date: *October 24-25th, 2025 {Sumaiya}*

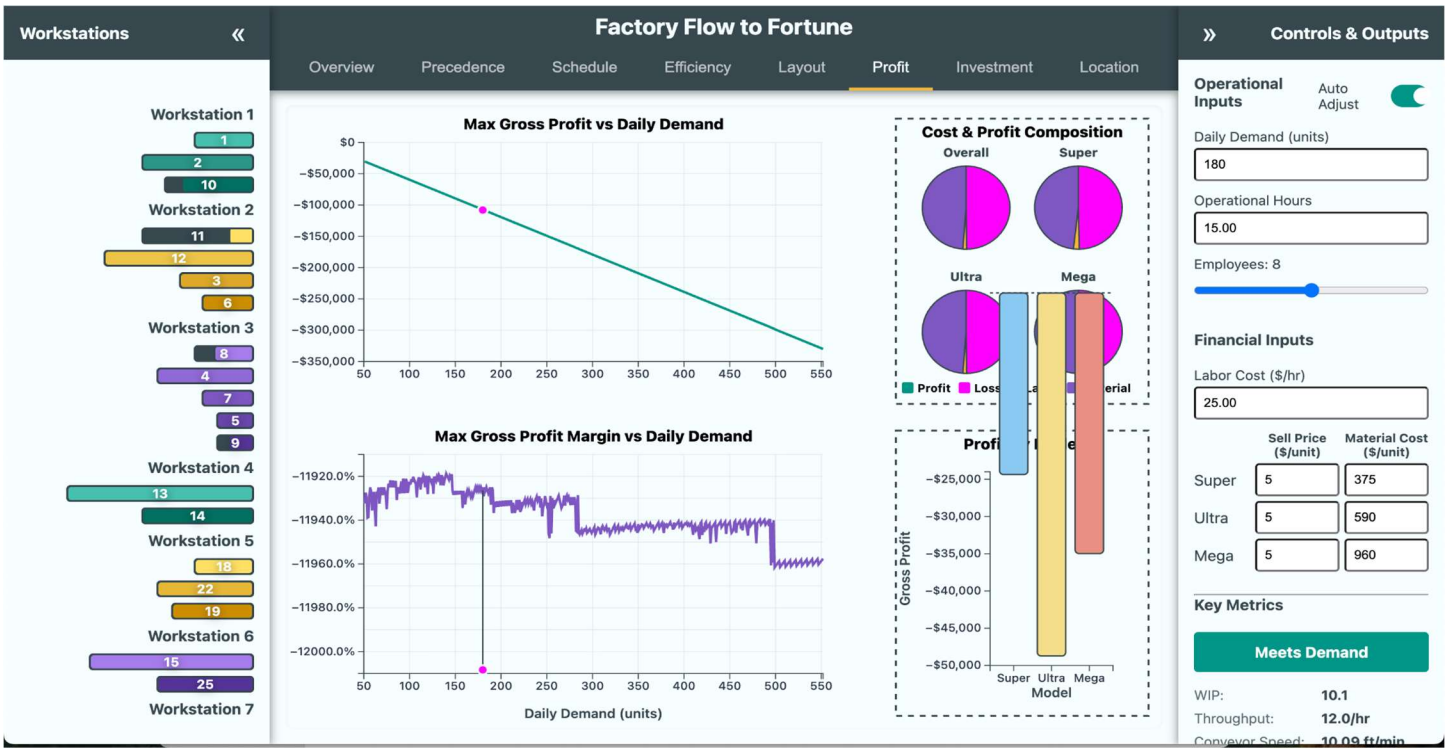
1. I added a small legend card on the right (“Profit Margin Indicators”) with two items: Magenta dot = Current profit/margin and Magenta shaded area = Unmet demand (lost profit)I also kept concise tooltips on the red dot and the red area that restate those meanings.This removes the ambiguity shown in the first screenshot and stops the legend from overlapping the charts.

2. I add “Total Lost Profit” to the tooltip. I calculate Total Lost Profit as
   (current demand × optimal profit at that demand) − (met demand × current-config profit),
   and I show it in the magenta-area tooltip on both charts, and in the general hover tooltip (only when unmet demand actually exists). It turns the gap into a dollar value users can act on.

3. Bars “explode” upward when all three models are losing money. I force the bar chart’s y-axis domain to always include $0, even if all profits are negative. I clamp the scale and clip the bars to their panel. This prevents runaway bars and locks everything inside its box (see the “before/after” screenshots shared).

4. $0 baseline leaks into the pie charts above. With the domain including zero, I draw a zero baseline that’s guaranteed to be inside the bar panel. I also added a clipPath around the bars area so nothing can bleed upward.
   Why: keeps the baseline and bars visually bounded; no more overlap into the pies.

In summary, To fix the legend and tooltip clarity, I introduced a dedicated legend panel that clearly labels the magenta dot as the current profit or margin and the magenta shaded area between lines as unmet demand (lost profit). This was done so users could immediately understand what the visuals represent without confusion. I also enhanced the tooltip to include a “total lost profit” row. This calculation now appears directly in the tooltip and uses the required formula: (current demand × optimal profit at that demand) minus (met demand × current configuration profit). This provides precise, actionable insight whenever unmet demand is present.



To address bar chart glitches when all models are unprofitable, I made sure the Y-axis for the bar chart always includes zero and clamps bars inside the plot area, preventing bars from leaking into or overlapping with other chart components—even if all values are negative. I also ensured that the $0 baseline line is always visible and stays inside the plot panel, so it won't spill upward into the pie chart section no matter how low the gross profit goes. This keeps the chart visually clean and interpretable.These changes were made to improve readability, make explanations accessible without needing external help, solve previous graphical and alignment issues, and ensure accurate context is provided for important metrics in loss scenarios.

Mixed Integer Linear Programming Helper Development

Date: *October 23rd – October 25th, 2025 {Joel}*

Activity: MILP modeling, building the worker function, and solving specific simulation bugs. This was a multi-day process of building up a Mixed Integer Linear Programming model which would take LP Strings as inputs. This was more complex than the rest of the site's backend put together, and the development of it was quite time-consuming, but did not really involve any visualizations—thus the descriptions of the development are kept brief, as despite being key for the functioning of the site, they lie outside the scope of the course—involving concepts of Operations Research & Integer Programming optimization.

Selection of MILP Modeling: This type of optimization problem is best solved using mixed-integer linear programming, which was structured using ***"Operations Research: Applications and Algorithms" by Wayne L. Winston.*** Initial attempts to run directly using JavaScript were disastrous, as it computationally would cause several minutes of lag. A heuristic was developed initially to help minimize computations, but even that was unacceptable. Ultimately, several different LP optimization APIs were tried, but HiGHS gave by far the best performance. HiGHS is published under an MIT license and developed by an optimization group from the University of Edinburgh. It takes an input of strings and processes them using web-assembly. This process Is managed using a web-worker, which allows the simulations to run without locking up the website. This is BY FAR the most computationally intense part of the website. The heuristics developed are still used to handle edge cases, and as a fallback if the web-assembly fails.

Technical Implementation: MILP & Web Worker

Objective: Transition to using the HiGHS optimization solver within a JavaScript Web Worker.

- **Solver API Migration (HiGHS):**
    - Migrated to the high-level highs-js library using .solve(lpString), which passes into a web-assembly worker.
    - **Initialization:** Implemented a robust getSolverInstance function to prevent race conditions, explicitly checking for the existence of the .solve method before returning the instance.
    - **Syntax Fix:** Resolved "Memory access out of bounds" errors by flattening the LP string generation (explicitly distributing signs: + 1 x_1 - 1 x_2 <= 0 instead of using parentheses).
- **Inventory Simulation Fix (Day 12 Bug):**
    - **Diagnosis:** The simulation failed on Day 12 due to floating-point rounding errors in Overtime logic. Math.floor() was truncating fractional shortfalls (e.g., 0.5 units), preventing the simulation from clearing tiny balances.
    - **Solution:** Removed Math.floor from maxExtraProduction and actualProductionToAdd. Updated the completion check to use an epsilon (if (actualProductionToAdd <= 0.01)) to handle floating-point precision.

Location Tab Integration of MILP Optimization

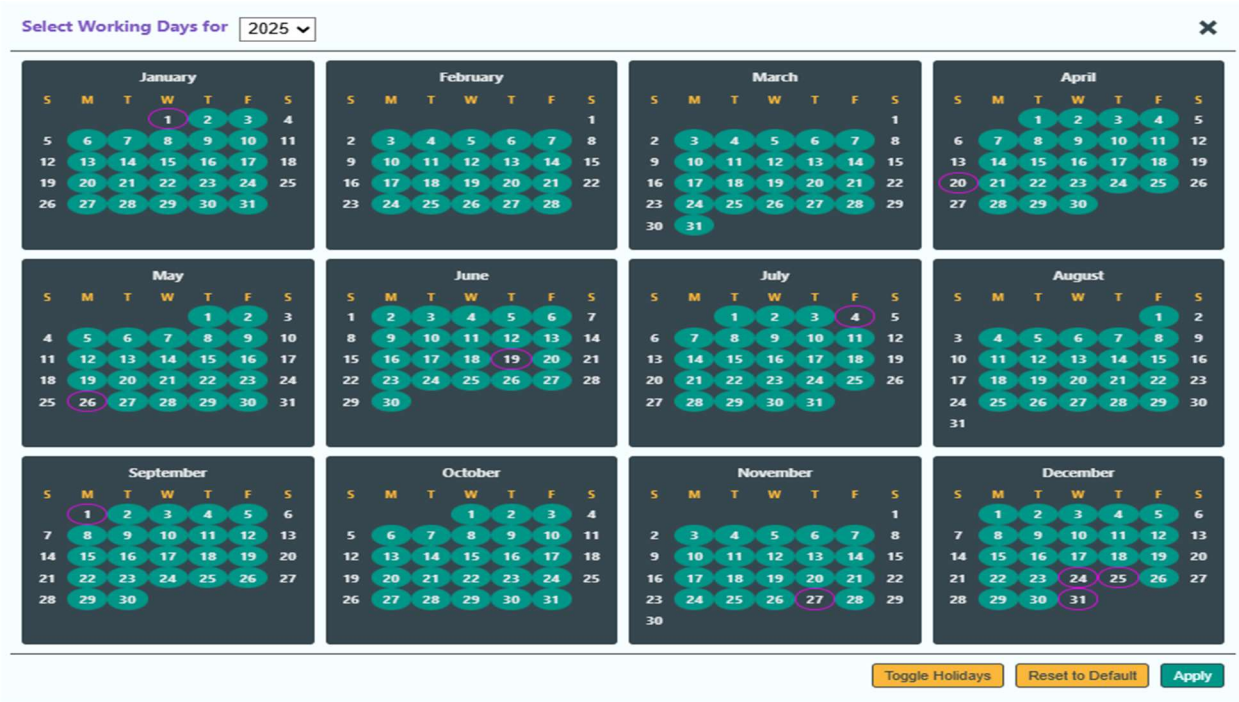
Date: *October 26th, 2025 {Joel}*

Activity: Enabled popup bar for shipment schedules and added working days calendar to the investment tab.

Technical Implementation: Investment & Location Logic

Objective: Enhance tabs to support granular production planning and inventory cost analysis based on specific calendar days.

- **Investment Tab (investment.js):**
    - **Interactive Calendar:** Replaced static "Working Days" input with a modal calendar (2025–2035). Implemented logic for holidays (New Year's, Christmas, calculated Easter) and user-toggled days.



- **State Management:** The specific list of selected dates is now stored in investmentState and exposed via data-working-days-list.

- **Location Tab (location.js):**
  - **Holding Cost Logic:** Implemented a sub-component calculator for the holding cost percentage based on Capital (MARR), Storage (distance), Service (tax), and Risk (frequency).
  - **Inventory Simulation:** Created a daily simulation loop where production only adds to inventory on days marked as "Working Days."
- **Worker Logic Update (simulation.worker.js):**
  - **Constraint Generation:** Modified the MILP constraints loop. It now checks if a specific day t is a working day.
  - **Logic:** If t is *not* a working day, the solver skips generating the constraint shipment_load - Z <= 0. This ensures shipments falling on weekends/holidays do not contribute to Peak Load (Z).

Investment Tab UI Improvements

Date: *October 26th, 2025 {Dhruv}*

Overview and Motivation

This session focused on improving **data interpretability, layout organization, and user guidance** across both sidebars and the **Investment tab**.
The primary goals were to make metric groupings more logical, align all tooltips under a consistent design system, and extend interactive functionality to financial components within the Investment view.
Collectively, these updates strengthen the application's analytical depth while improving clarity and user experience.

Related Work

This build builds directly upon the sidebar redesign and UI standardization efforts completed earlier this month. Where previous sessions concentrated on animation behavior and structural resizing, this round emphasized **information hierarchy, tooltip coherence, and input consistency**—ensuring that financial and operational insights are both accessible and comprehensible.

Questions

- How can the sidebar layout better communicate logical relationships among operational and financial metrics?
- What tooltip system provides consistent contextual help across all tabs and inputs?
- How can text box interactivity in the Investment tab be aligned with existing sidebar input logic for a unified experience?

Data

No changes were made to underlying data sources. All updates centered on **interface usability** and **metric categorization**, with minor logic enhancements to maintain consistent parameter behavior.

Design Evolution

1. Right Sidebar Refinements

Problem:
The right sidebar's organization and input behavior required refinement to improve usability and logical flow. The parameter tooltips were inconsistent with the rest of the interface, and certain financial metrics were grouped under generic sections, reducing clarity.

Solutions:

- **Operational Hours Adjustment:** Updated mouse movement sensitivity to modify operational hours in **0.25-hour increments** (instead of 0.1) for finer but practical adjustment control that matches the simulation's real-world time granularity.
- **Investment Metrics Integration:** Added **Investment tab metrics** to the right sidebar, providing direct visibility into key financial performance indicators alongside operational parameters.
- **Category Reorganization:**
    - Introduced a new **Financial Metrics** section to consolidate all profitability-related values.
    - Moved **Profit Margin** and **Gross Profit** under this section for better contextual alignment with related data.
- **Logical Reordering of Metrics:** Repositioned the **Demand Status Bar** above the **Key Metrics** section, creating a more intuitive hierarchy that aligns with the user's analytical flow—from demand evaluation to performance assessment.
- **Tooltip Implementation:** Added **tooltips to every parameter** in the sidebar. Each tooltip now briefly explains what the parameter represents and how it influences the simulation outcomes. These tooltips follow the same hover-triggered interaction model and design styling as other tabs, ensuring full visual and functional consistency.

Outcome:
The right sidebar now provides a more intuitive, logically organized, and self-explanatory interface. The inclusion of detailed tooltips allows users to better understand each parameter's significance, while the new Financial Metrics section presents economic indicators with greater clarity and professionalism.

2. Left Sidebar Enhancements

Problem:
Tooltips for workstation elements were visually inconsistent with those in the right sidebar and lacked descriptive detail about each workstation's physical attributes.

Solutions:

- **Tooltip Style Alignment:** Updated the left sidebar tooltips to match the global tooltip design—using the same typography, shadowing, and opacity settings introduced earlier for metric panels.
- **Enhanced Descriptive Content:** Added new descriptive information to each workstation's tooltip, including:
    - The **total conveyor length** associated with that workstation (in feet).
    - The **length of each individual element** contained within it.
      These details help users understand not only functional relationships but also spatial and mechanical proportions within the assembly line.

Outcome:
The left sidebar now delivers uniform visual behavior and richer informational context. The improved tooltips make workstation analysis more tangible, bridging operational data with physical layout insight.

3. Investment Tab Expansion

Problem:
The Investment tab previously displayed static financial indicators without interactive adjustment or explanatory support.

Solutions:

- **Interactive Input Parity:** Implemented text box functionality for **Economic Parameters** (mirroring the behavior of numeric inputs in the right sidebar).
    - Users can now manually enter or increment/decrement values for financial parameters such as discount rate, initial investment, and analysis period.
    - Inputs respond dynamically to user edits, immediately updating dependent calculations (e.g., NPV and IRR).
- **Financial Tooltips:** Added explanatory tooltips to major financial indicators:
    - **Net Present Value (NPV):** Describes how NPV represents the value of future cash flows discounted to present day.
    - **Internal Rate of Return (IRR):** Defines IRR as the discount rate at which NPV equals zero, indicating investment efficiency.
    - **Payback Period:** Explains how long it takes for cumulative returns to recover the initial investment.
      Each tooltip follows the same style and behavior established across other tabs, ensuring continuity and readability.

Outcome:
The Investment tab now supports both **interactive parameter exploration** and **self-explanatory guidance**. This creates a smoother transition between operational and financial analysis, empowering users to evaluate investment performance without requiring external references.

Implementation

- **Files Modified:** script.js, style.css, and Investment tab rendering logic.
- **Core Updates:**
    - Adjusted step size for operational hours to 0.25.
    - Added Financial Metrics section and reorganized sidebar hierarchy.
    - Added comprehensive tooltips for all sidebar parameters.
    - Unified tooltip styles across sidebars.
    - Extended interactive input logic to the Investment tab.
    - Added tooltips for key financial metrics (NPV, IRR, Payback Period).

Evaluation

- **Consistency:** All tooltips now share a unified design language and behavior across tabs.
- **Clarity:** Sidebar and Investment layouts follow a more intuitive information hierarchy.
- **Interactivity:** Economic parameters in the Investment tab are now fully adjustable and responsive.
- **Usability:** Fine-tuned step control for operational hours provides practical precision.
- **Accessibility:** Users can now understand all sidebar parameters and financial metrics without requiring documentation, supporting self-guided learning.

Session Outcomes

This session achieved a major **usability and informational clarity milestone**.
Through consistent tooltip integration, structured layout reorganization, and the introduction of interactive

financial inputs, the Factory Flow to Fortune interface now provides a **more educational, transparent, and analytical user experience**.
Users can navigate, adjust, and interpret both operational and financial performance parameters seamlessly—reflecting a mature, integrated design approach that unifies all sides of production and investment analysis.

Simulation Ribbon Refinement and Backend Update

Date: *October 27th, 2025 {Joel}*

Activity: Added PPI charts and expanded the backend page.

Technical Implementation: Daily Simulation & Visualization

Objective: Integrate the daily simulation loop and resolve scoping errors.

- **Location Tab - Daily Simulation:**
  - Implemented runDailyInventorySimulation to simulate 365 days.
  - **Logic:** Inventory accumulates via production; Shipments deplete available inventory. If demand fails, the system "looks back" to previous working days to apply overtime.
  - **Visualization:** Created a new modal and D3.js chart to visualize Weekly Shipments vs. Inventory Levels.
- **Bug Fixes & Scoping:**
  - **Scope Resolution:** Resolved errors where majorCities and loadCsvBaselineData were undefined by moving them to the correct scope.
  - **Shipment Logic:** Corrected an order-of-operations error where shipments were assessed before production was added, causing false shortages on Day 0.
  - **Map Rendering:** Separated static map initialization (geography) from dynamic element updates to prevent "flickering" on redraws.

Refactoring to handle sizing and tooltips uniformly

Date: *October 28th, 2025 {Joel}*

Activity: Implemented dynamic resizing refactoring of precedence, and then other tabs, and centralized tooltips.

Technical Implementation: Global Refactoring

Objective: Modularize the architecture, centralize shared logic, and implement a robust responsive system.

- **Central Infrastructure (script.js):**
  - **Resize Logic:** Implemented a ResizeObserver on #svg-container. This triggers a debounced handler that calls the specific .resize() or .draw() method of the active tab.
  - **Global Tooltips:** Moved createTooltip and positionTooltip to the global scope to standardize styling and positioning.
  - **State Management:** Merged taskData, configData, and currentMetrics into a single global state object to fix synchronization issues.
- **Precedence Tab Refactor:**
  - **State Promotion:** Moved critical variables (nodes, links, layout targets) out of the draw() closure to module-level scope.
  - **Animation Logic:** Implemented "Snapshot & Tween" logic for the K&W toggle. The simulation stops, captures coordinates, and interpolates to new targets to prevent nodes from "snapping" or arrows detaching.

- **Layout Tab Refactor:**
    - **Responsiveness:** Implemented logic to calculate the bounding box of the U-shaped line and scale it to fit the 80/20 split of the SVG.
    - **Finished Goods Bin:** Rewrote logic to dynamically calculate bin dimensions and center items based on available space.
- **Schedule Tab Refactor:**
    - **Sidebar Coupling:** Fixed the issue where "Workstation" labels scrolled away from the chart by splitting the SVG into a staticGroup and contentGroup with synchronized scroll events.

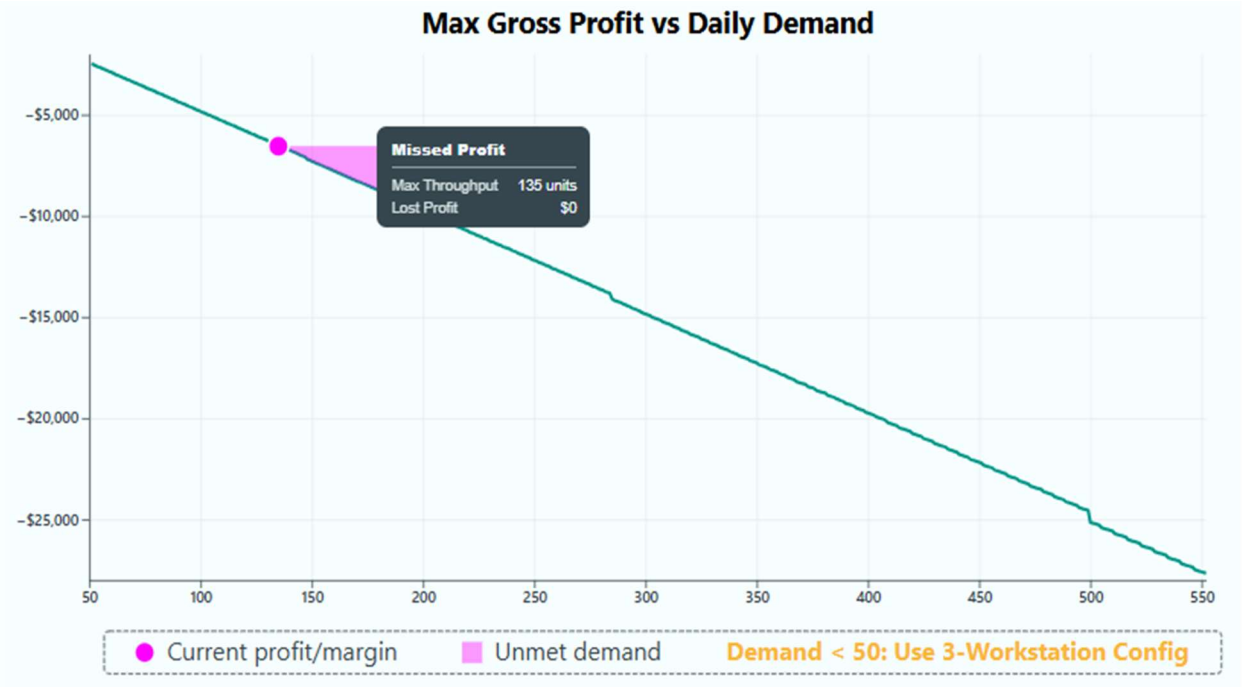
Restructuring Profit for Dynamic SVG Resizing

Date: *October 29th, 2025 {Joel}*

Activity: Fixed spacing and sizing in profit and modified tooltips for lost profit display.

Technical Implementation: Profit & UI Polish

- **Profit Tab (Profit.js):**
    - **"Lost Profit" Logic:** Updated calculation to define Lost Profit as Optimal - Current. Implemented a d3.area() generator to correctly fill the jagged area between the optimal curve and the current profit line.



    - **Responsiveness:** Added logic to constrain Pie Chart radius to prevent expansion on wide screens. Rewrote legend logic to center horizontally at the bottom.

- **Global Tooltips:**
    - **Stacking Context Fix:** Implemented a universal tooltip system that toggles display: none (in addition to opacity). This prevents hidden tooltips from blocking mouse events on underlying elements.
    - **Z-Index:** Applied global high z-index and absolute positioning to ensure tooltips float above all dashboard panels.

Further MILP Refinements and Brush Functionality

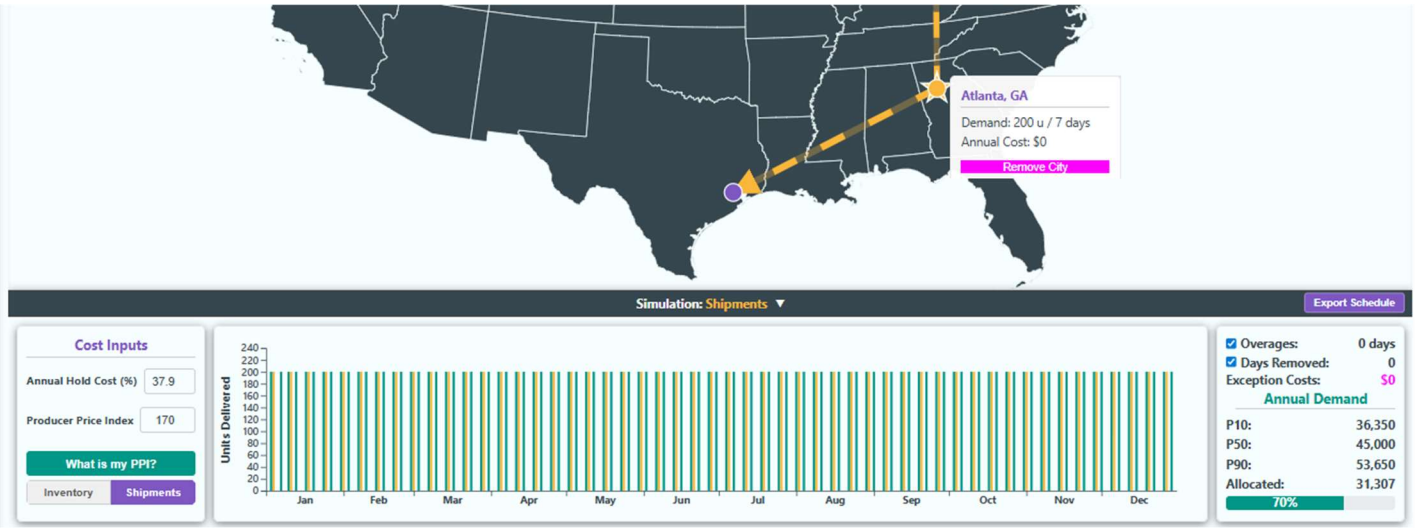
Date: *November 1st, 2025 {Joel}*

Activity: Refined MILP, fixed layout tab, added Brush to the date range in simulation. Modified the MILP to become more complex, and to use a smoothing heuristic after minimizing and smooth inventory levels.

Technical Implementation: Simulation Engine Overhaul

Objective: Shift logic from "Push" to "Pull" and implement specific Tiered logic.

- **Simulation Logic (simulation.worker.js):**
    - **Pull Model:** Transitioned to a "Build-to-Target" model.
    - **3-Tier Logic:**
        1. **Smoothing:** Build specific target to maintain even inventory.
        2. **Flex:** If shortfall, flex up to max capacity (no cost).
        3. **Reactive OT:** If Tier 2 fails, look backward to add Overtime.
    - **Heuristic Update:** For high-frequency shipments (>11 days), the worker now generates evenly spaced candidate start days and scores them based on collision potential before sending to the solver.
- **Location Tab UI (LocationTab.js):**
    - **Ribbon UI:** Replaced modals with a persistent, collapsible bottom ribbon.
    - **Chart Interaction:** Implemented D3 Brush functionality for selecting date ranges. A clicked city will highlight the shipments to that city in the shipping simulation tab.
    - **Soft Failure:** Implemented logic to check for "Demand Conflict" errors. If found, the previous chart is preserved, and a semi-transparent red overlay is drawn on top.



Quality Yield Simulation Development

Date: *November 2nd, 2025 {Dhruv}*

Overview and Motivation

This session combined **simulation logic advancement** with several **visual and functional refinements** across the Factory Flow to Fortune interface.
The key addition was a dynamic, behaviorally driven **Quality Yield Model**, which introduces real-world variability into production outcomes by linking operational stress to quality degradation.

Alongside this system-level improvement, multiple UI enhancements were implemented across the **Right Sidebar**, **Efficiency**, **Layout**, and **Location** tabs to improve animation fidelity, layout consistency, and interactivity.

Together, these updates mark a critical step toward making the platform's simulations both more **visually polished** and **realistically complex**.

Related Work

This session builds upon prior refinements to animation responsiveness, tab synchronization, and the sidebar interface. The new quality yield model extends the project's emphasis on operational realism—moving beyond static efficiency calculations toward dynamic, condition-dependent outcomes.

Questions

- How can production quality be realistically modeled as a function of operational stress while maintaining computational efficiency?
- How can interface animations be standardized across all visual modules without performance loss?
- What structural improvements can make metric display, configuration management, and map-based animations more intuitive and engaging?

Data

No new data sources were introduced; however, simulation data models were expanded to include **quality yield factors**, which modify throughput and profitability calculations based on stress and randomness parameters.

Design Evolution

1. Right Sidebar Refinements

Problems:

- Metric text in the right sidebar (except for investment tab metrics) wasn't animating correctly when values updated, breaking the consistency of visual feedback.
- There was no convenient way to save and compare configuration states for performance benchmarking.

Solutions:

- **Text Animation Fix:** Corrected the animation logic for metric transitions to ensure that all numeric values now **animate smoothly** when changing, rather than snapping instantly. This applies to throughput, profit, and other operational indicators, providing a consistent and engaging feedback experience.
- **Configuration Management:** Added a new **"Save Configuration" button** with an optional **"Compare" switch**.
    - Users can now save the current operational configuration, then toggle the compare mode to visualize both the current and saved parameter sets side-by-side.
    - This allows for quick benchmarking of changes to operational parameters (e.g., staffing or production hours) and their impact on key metrics.

Outcome:
The right sidebar now offers smoother data presentation and introduces comparative configuration analysis—a valuable tool for decision-making and optimization testing.

2. Efficiency Tab Adjustments

Problems:

- Text labels above the summary charts were misaligned on smaller screens due to inconsistent scaling.
- The code structure for text layout was inefficient, leading to readability issues and redundant transformations.
- The bar chart animation occasionally distorted bar shapes due to transition conflicts in the newer implementation.

Solutions:

- **Responsive Text Spacing:** Reworked layout scaling calculations so that chart titles and percentage labels adapt gracefully to smaller resolutions, maintaining proper alignment and spacing.
- **Code Optimization with <tspan>:** Refactored multi-line SVG text rendering to use <tspan> elements, simplifying code and improving readability and alignment consistency across screen sizes.
- **Restored Stable Animation:** Reverted the bar chart animation logic to a **previous stable version**, eliminating rendering artifacts and preserving smooth, reliable transitions during updates.

Outcome:
The Efficiency tab now displays metrics with professional polish across all resolutions. Animations remain visually consistent, while text formatting and scaling are robust and maintainable.

3. Layout Tab Refinement

Problem:
The assembly line visualization used inconsistent corner styling—some turns appeared with sharp edges while others were rounded—creating visual discontinuity.

Solution:
Standardized all **assembly line path corners to rounded edges**, unifying the design language and improving the perceived flow of the layout visualization.

Outcome:
The Layout tab now feels visually cohesive, with a smoother, more realistic representation of the assembly process.

4. Location Tab Animation Enhancements

Problems:

- The arrow animation connecting cities was static at first and only animated the dashed line portion, not the arrowhead itself.
- The dashed line rendered prematurely, appearing before the start and end markers were defined.

Solutions:

- **Fully Animated Arrowhead:** Implemented a complete arrow animation sequence where the **arrowhead first travels from the starting marker to the destination**, followed by the **dashed line animation** that trails behind it.
- **Conditional Animation Trigger:** Adjusted the drawing logic so that the dashed line is only created and animated **after both start and end markers are placed**, preventing early visual artifacts.

Outcome:
The Location tab now features a more dynamic, sequential animation that effectively communicates directionality and data flow between geographic points. This makes spatial interactions more intuitive and visually compelling.

5. Quality Yield Model Implementation

Problem:
The simulation previously assumed constant production quality, regardless of working conditions. This limited the model's realism and ability to simulate the trade-offs between speed, stress, and output quality.

Solution:
Introduced a **dynamic Quality Yield Model** that ties production quality to operational stress factors using an exponential decay function with random variation:

- **Core Equation:** Quality yield decreases exponentially as operational stress increases, where stress is influenced by variables such as staffing levels, working hours, and demand volume.
- **Variability Component:** Added stochastic (randomized) perturbations to simulate real-world unpredictability in human performance and manufacturing variation.
- **Dynamic Interaction:** Quality yield recalculates continuously during simulation playback, meaning that small changes in staffing or operational hours can immediately influence throughput and profitability.

Impact on System:

- Configurations with **fewer employees, longer hours, or higher demand** now result in visibly lower yields.
- Efficiency and profit calculations dynamically adjust to reflect real-world consequences of overutilization or understaffing.
- The simulation encourages users to manage parameters strategically, finding balanced configurations that optimize both output and quality.

Outcome:
The new model brings the platform's analytics closer to real-world manufacturing dynamics, adding depth to both the simulation and decision-making experience. It transforms operational stress from a static constraint into a **core gameplay mechanic and educational feedback system**.

Implementation

- **Files Modified:** script.js, style.css, and tab-specific render logic for Efficiency, Location, and Sidebar systems.
- **Core Additions and Changes:**
    - Fixed right sidebar metric animations.
    - Added "Save Configuration" button and "Compare" toggle for benchmarking.

- Corrected Efficiency tab text scaling, introduced <tspan> optimization, and reverted stable bar animation logic.
- Standardized assembly line corner styling in Layout tab.
- Implemented complete arrowhead and dashed-line animation sequencing in Location tab.
- Integrated Quality Yield Model linking stress factors to performance outcomes.

Evaluation

- **Functional Stability:** All previously inconsistent animations now operate smoothly and predictably.
- **Visual Cohesion:** Consistent rounded-corner design, proportional text spacing, and responsive layout scaling enhance polish.
- **Analytical Depth:** The Quality Yield Model introduces a new behavioral layer to the simulation, rewarding balanced management strategies.
- **Interactivity:** The right sidebar's new comparison tools improve iterative testing and configuration analysis.
- **User Engagement:** The Location tab's enhanced arrow animation provides a visually satisfying and informative spatial experience.

Session Outcomes

This session represents a major step forward in **both realism and interactivity**.
By combining advanced simulation logic with meticulous UI refinements, the Factory Flow to Fortune platform now offers a **holistic production management experience**—one that merges operational, financial, and quality dimensions under a unified, dynamic model.
The Quality Yield system introduces authentic variability, while interface updates across all tabs ensure a smooth, intuitive, and visually consistent user experience.

Quality Yield Visual Integration

Date: *November 3rd, 2025 {Dhruv}*

Overview and Motivation

This session focused on **stabilizing the Quality Yield implementation** introduced in the previous build and expanding its impact across multiple tabs through visual feedback.
The primary objectives were twofold:

1. **Resolve glitches** in the Investment, Location, and Profit tabs to ensure the quality yield calculations properly integrated with their respective performance metrics.
2. **Visually represent defective products** in both the Layout and Schedule tabs, introducing a layer of stochastic realism where defect patterns vary each simulation run, even under identical input parameters.

These updates mark the transition from a purely mathematical model of quality degradation to an **experiential simulation feature**—one that visually demonstrates the consequences of operational stress and randomness.

Related Work

This session builds upon the **Quality Yield Model** introduced on October 24, which linked production quality to operational stress through an exponential decay equation.

While the previous version successfully integrated quality yield into production and profit calculations, this session extended that logic visually and ensured consistent, stable propagation of its effects throughout the system.

Questions

- How can the platform visually communicate the impact of quality degradation without overwhelming the primary data visualizations?
- How can the defect generation logic remain stochastic yet reproducible within a single simulation run?
- How can dependent tabs (Investment, Location, Profit) respond consistently to changes in dynamic quality yield without creating desynchronization errors?

Data

No new data sources were introduced.
Quality yield and defect generation were integrated as extensions of existing production data structures, with additional randomization layers applied at runtime to simulate unpredictable manufacturing variation.

Design Evolution

1. Global Quality Yield Stabilization

Problem:
After implementing the Quality Yield Model, certain dependent tabs—particularly **Investment**, **Location**, and **Profit**—exhibited glitches such as incorrect recalculations, temporary data mismatches, or frozen metric updates when quality yield changed dynamically.

Solution:

- Refactored the update propagation logic to ensure that **quality yield values cascade properly** through all dependent modules.
- Adjusted asynchronous event handling so that financial and geographic visualizations update in lockstep with the main operational data model.
- Resolved rounding and update-order inconsistencies in the Profit and Investment calculations to ensure accurate reflection of quality-related losses.

Outcome:
All tabs now respond coherently to quality yield fluctuations, maintaining full synchronization between operational stress, production output, and financial performance.
The platform's analytical integrity was restored without compromising responsiveness or simulation speed.

2. Layout Tab – Visual Representation of Defective Products

Problem:
While the quality yield system mathematically reduced throughput and profit, there was no **visual feedback** showing how defects manifested in the assembly line. This limited user comprehension of how quality degradation impacts production visually.

Solution:

- Introduced a **defective product visualization system** within the Layout tab.

    - A small, randomized subset of finished products are now marked as **defective**, visually distinguished by altered color, pattern, or iconography.
    - Defect generation is **stochastic**—each simulation run produces a different defect pattern, even under identical operational parameters.
- The number of defective products directly corresponds to the current quality yield ratio, ensuring logical consistency between the numerical model and its visual outcome.
- Defects are distributed dynamically along the line, emphasizing that failures can occur at any stage of production.

Outcome:
The Layout tab now offers a tangible and intuitive way to observe how operational stress translates into production variability. The randomness of defect appearance reinforces the simulation's realism, making each run unique and more reflective of real-world uncertainty.

3. Schedule Tab – Consistent Defective Product Representation

Problem:
The Schedule tab's visualization previously treated all products as identical, offering no differentiation between successful and defective units. This created a disconnect between the detailed operational sequencing and the new defect model visualized in the Layout tab.

Solution:

- Implemented the same **defective product visualization system** used in the Layout tab.
    - Each defective product identified in the Layout tab is mirrored within the Schedule visualization, maintaining one-to-one consistency.
    - The mapping ensures that the same items marked defective in Layout are represented as such in Schedule—matching their positions and sequence in the build order.
- The underlying defect assignment remains **randomized per simulation run**, ensuring that even with unchanged parameters, users experience a fresh and realistic outcome.

Outcome:
The Schedule visualization now aligns perfectly with the Layout tab, presenting a synchronized depiction of both temporal and spatial aspects of production quality.
Users can trace when and where defective products occurred, providing a holistic view of quality control across the manufacturing timeline.

Implementation

- **Files Modified:** script.js, style.css, and visualization logic for Layout and Schedule tabs.
- **Core Updates:**
    - Fixed cascading quality yield integration bugs in Investment, Location, and Profit tabs.
    - Added stochastic defect generation logic to Layout tab.
    - Implemented synchronized defective product rendering in Schedule tab.
    - Ensured defect distribution and color coding remain visually distinct but non-intrusive.

Evaluation

- **Simulation Realism:** Randomized defect representation introduces believable variability and risk dynamics.
- **System Stability:** All dependent tabs now correctly interpret and update quality yield data without desynchronization.
- **User Insight:** The correlation between operational stress and visible defects creates an immediate, intuitive connection between management decisions and quality outcomes.
- **Cross-Tab Consistency:** Layout and Schedule visualizations remain synchronized in defect occurrence and timing.
- **Engagement:** The unpredictability of defect outcomes adds depth and replayability to simulations.

Session Outcomes

This session elevated the Factory Flow to Fortune simulation from a purely numerical quality model to a **visually demonstrative system** that integrates stochastic variation across operational, scheduling, and financial layers.
By stabilizing the Quality Yield framework and introducing visual representations of defective products, the platform now communicates the tangible effects of operational stress with unprecedented clarity.
Each simulation run feels more authentic, emphasizing the delicate balance between efficiency, output, and quality that defines real-world production environments.

Profit Tab Modifications

Date: *November 4-5th, 2025 {Sumaiya}*

1. Transformed “Cost & Profit Composition” area to a layout

In the previous version, the middle/right panel was treated like a little Illustrator file inside the SVG — I was dropping pies at very specific places using pixel values but those numbers were baked in. That meant the spacing between “Overall / Super” and “Ultra / Mega” looked fine only at the size I designed it at. In this version, I switched to slots instead of pixels:

- figured out how much space that panel actually has (after the top dashed box and margins)
- 4 pies will be plac ed in 2 columns × 2 rows
- centered each pie inside its slot
- made the pie radius a fraction of the slot size, not a fixed number

That instantly made the vertical gap between the top row and bottom row look more consistent across different panel heights.

---

2. Rebuilt the legend so the two sides line up

The first approach to the legend was first placing the elements, then center each line That’s fine for unknown data, but it causes the exact visual bug we saw: Profit and Labor didn’t start at the same x, and Loss and Material didn’t start at the same x at converting it to 2*2 grid. Each row was doing its own centering. In the final version I did it the way people lay out forms:

- made each column as wide as the widest item in that column.
- centered the *whole* 2-column block.

Because both items in a column now share the same starting x, “Profit” and “Labor” line up perfectly on the left, and “Loss” and “Material” line up perfectly on the right. That’s what you asked for: equal-looking left margin on the left pair and equal-looking left margin on the right pair.

---

3. I tied the legend’s Y position to the pies, not to the SVG edge

Before, the legend was kind of “pinned” near the bottom of that dashed box. That’s why on some sizes it looked like there was a weird big gap between the lower pies and the legend. I put the legend right below that with a small buffer. That way, if the pies ever get a bit taller or the panel height changes, the legend follows them instead of floating too low.

Fixes Summary:

SVG is stretchable; but a pixel number is not. Pixels lock you to your scree. When I fix say x = 120px, I’m really saying to pretend everyone will see this at the same width I saw it at. That’s almost never true. Someone else will have: a wider panel, a narrower sidebar, a different zoom, or just a different browser default. So what looked like 20px from the left on my screen can turn into way too close or way too far on someone else’s. Percentages (or “a fraction of the current width/height”) don’t have that problem — they grow and shrink with the container.

Text doesn’t scale the same way as drawings. Legend items are words. Words don’t all have the same length. “Profit” is short. “Material” is longer. If I place everything with fixed pixels from the left, then center each row, every row ends up centering around a different width — so two rows will look slightly crooked even though the code looks correct. By measuring the text first and then building columns that are as wide as the longest word in that column, I made the geometry respond to the content. Pixels can’t do that on their own.

That right-hand panel is not always the same height: the whole SVG height can change, and I also have to leave room for the top dashed “Profit Margin Indicators” box. If I use pixel Y-values for where the pies go, eventually two things happen: on a short panel, the pies bump into the legend; on a tall panel, the pies look like they’re floating too high. When I set positions as percentage inside the inner panel and legend = bottom of pies + a little, it looks intentional at both sizes.

Pixels don’t know about aspect ratio. If I hardcode radius = 55px, that might look great when the panel is fairly square. But if the panel becomes wide and short, 55px might be too tall; if the panel becomes narrow and tall, 55px might look tiny. Basing the radius on min(cellWidth, cellHeight) (use whichever side is smaller) keeps the circles looking like circles and keeps them from crowding each other.

If someone zooms the page to 125% or 150%, text grows. Fixed pixel offsets don’t. That’s when labels start overlapping, or start looking off-center relative to icons. When I measure the text and then space based on that measurement, the layout is allowed to grow with zoom.

Session Summary -

- replaced hard-placed pie positions with a 2×2 grid that uses the actual panel size.
- replaced auto-wrapped legend rows with a 2-column legend where both rows in a column start at the same x.
- aligned the legend to the pies
- sized things from available width/height instead of pixel numbers.

Now, the bar chart was too close to the left edge. That bottom panel on the right (“Profit by Model”) already had a border, a y-axis with numbers, a rotated y-axis label (“Gross Profit”), and then the actual bars. Because everything started too far to the left, the rotated label and the y-axis ticks looks cramped (on some width) or like the label is almost touching the border.

I introduced a small horizontal offset just for that chart (barShiftX = 16) and applied it to the group that draws the bar chart. Basically, I moved the whole bar chart a bit to the right inside its panel. That created a comfortable space on the left, now:

- the rotated y-axis label has room to breathe,
- the tick labels aren’t smashed into the panel border,
- and the bars don’t feel like they’re starting on the edge of the border

I didn’t move the entire right panel — only nudged the inner chart drawing area. That’s the right surgical fix.

Now, y-axis numbers were too long / too detailed for this small panel. This chart is about per-model profit, so the numbers can easily get into thousands. Showing them as full currency (“$1,250”, “$12,500”, etc.) on a narrow right-side panel makes the axis feel heavy and sometimes forces extra width. It also doesn’t match the “quick-glance” vibe of a comparison chart — users mostly need order of magnitude here, not accounting-grade formatting. I switched the y-axis tick format for this chart to a compact “k” style (1k, 2k, 12k, etc.). That:

- shortens every tick label,
- makes the scale easier to scan,
- reduces the risk of labels overlapping or pushing into the bars,
- and matches the kind of chart this is (comparative, not financial statement).

Because I changed the format, I also re-measured the width of the tick labels using that new format and used that measured width to reserve space on the left of the bars. It is necessary if the label text is changed but the space needed is not recalculated, then it can still end up with overlap.

Session summary -

Putting those two fixes together:

The left side of the “Profit by Model” chart has proper spacing between border → y-axis → label → bars.

The layout is more resilient at different container sizes, because now its not assuming a fixed text width anymore and tshifts the chart accordingly.

Quality Yield Probabilistic Model Changes

Date: *November 9th, 2025 {Dhruv}*

Overview and Motivation

This session marked a significant leap forward in the **mathematical sophistication and realism** of the Factory Flow to Fortune simulation.
The **Quality Yield system**—originally based on exponential decay relationships between stress and quality—was reengineered into a **probabilistic model** that captures workstation-level uncertainty and variability using variance, standard deviation, and probability calculations.
Alongside this core simulation overhaul, several key interface refinements were made across multiple tabs to improve visual clarity, interaction stability, and peer-suggested usability features.

These updates collectively deepen both the **analytical realism** and **user engagement** of the platform.

Related Work

This work builds upon the prior implementation of the **Quality Yield Model** (October 24–25), which introduced dynamic yield degradation based on operational stress and random variation.
The new statistical foundation transforms that model from an empirical approximation into a **quantitatively driven system**, integrating principles of probability, standard deviation, and fatigue modeling across the production chain.

Questions

- How can task variability within workstations be modeled to realistically reflect human performance differences and timing uncertainty?
- What's the most computationally efficient way to approximate production yield without requiring full-scale Monte Carlo simulations?
- How can fatigue, overtime, and location-based factors (like wages) interact to produce meaningful differences in yield outcomes across scenarios?
- How can user-facing charts and metrics better convey uncertainty, risk, and operational efficiency?

Data

No new datasets were introduced directly, but the **QualityYield.js**, **Script.js**, and **Location.js** modules were updated to incorporate probabilistic inputs and new API linkages.
Location-based wage estimation logic now uses regional data via the **FCC Census Block API** and industry wage tables to automatically determine median wages based on the selected factory location.

Design Evolution

1. Quality Yield Calculation Redesign

Problem:
The previous quality yield model treated degradation as a deterministic function of stress and demand. While effective for visualization, it lacked the ability to capture **variance-based uncertainty** inherent to real manufacturing environments.

Solution:
Reformulated the **Quality Yield model** using statistical mechanics:

- **Workstation Variability Modeling:**

    - Each element's labor time now contributes to the **workstation's total variance** (σ^2).
    - The variance of the workstation is computed as the **sum of individual element variances**, and the overall **standard deviation (σ)** is derived as the square root of this sum.
    - This enables realistic differentiation between consistent stations (low variance) and unstable ones (high variance).
- **Probability of Overages:**
    - For each workstation, the system calculates the **probability that total task time exceeds available workstation time** using the mean and standard deviation of its labor times.
    - Overages are penalized according to severity: a **light penalty** for single-time exceedances and a **heavy penalty** for successive overages.
    - This framework allows yield degradation to emerge naturally from production instability rather than from fixed decay constants.
- **Operational Stress and Fatigue Factors:**
    - Conveyor speed is now tied directly to **fatigue modeling**, simulating physical and cognitive exhaustion as line speed increases.
    - Overtime is integrated as an exception variable, reflecting schedule overages or "removed shifts" that create extended working conditions.
    - Demand spikes trigger localized increases in conveyor speed or extended hours, both of which negatively influence quality yield with distinct weighting factors.
- **Location-Based Wages (Automation):**
    - When a factory location is chosen, the system now uses the **FCC Census Block API** to determine the median local wage for the factory's industry classification.
    - This replaces manual wage entry with automated regional lookup, ensuring consistent, data-backed economic modeling.

Outcome:
The Quality Yield system now uses real-world statistical logic to model production uncertainty. It captures the compounding risks of overwork, fatigue, and stochastic variation while remaining computationally efficient enough for real-time simulation.

2. Layout and Schedule Tab Stability Fix

Problem:
After adjusting sliders (e.g., conveyor speed or staffing), pressing any control button (e.g., pause, play, skip) reset the animation completely instead of executing the intended action.

Solution:

- Fixed the event-handling logic so that **control buttons execute their functions immediately** without reinitializing the simulation timeline.
- Adjusted the animation synchronization to ensure that UI and state transitions occur without resetting the playback buffer.

Outcome:
Interactions are now seamless, allowing users to make fine adjustments to simulation parameters without disrupting the overall flow of animations.

3. Location Tab Refinements

Problems:

- “Cost Inputs” text boxes lacked the responsive interactivity and input validation found elsewhere.
- The **Optimal Summary box** clipped on smaller screens, disrupting readability and visual hierarchy.

Solutions:

- Added **full textbox functionality** (numeric step controls, Ctrl+Click resets, and smooth animations) to all cost input fields, ensuring parity with other input modules.
- Adjusted responsive scaling and flexbox layout rules to prevent clipping of the Optimal Summary box at lower resolutions.

Outcome:
The Location tab now functions as a fully interactive configuration panel, with accessible controls and a clean, adaptive layout.

4. Efficiency Tab Enhancements

Problem:
The pie charts lacked interactivity, making it difficult for users to focus on individual efficiency slices or interpret category meanings without reference.

Solutions:

- **Interactive Pop-Out Effect:** When hovering over a slice of any pie chart (across all tabs), that slice now **“pops out”** slightly from the center, providing a clear visual cue for focus.
- **Tooltip System:** Incorporated peer feedback to add **hover tooltips** to all Efficiency tab charts, explaining what each section represents (e.g., “Idle Time,” “Work Time,” “Setup Delay”).

Outcome:
The Efficiency tab now offers a tactile, exploratory visualization experience with consistent tooltip support and motion-based interactivity that reinforces comprehension.

Implementation

- **Files Modified:** QualityYield.js, Script.js, Location.js, Style.css plus visualization logic for Layout, Schedule, and Efficiency tabs.
- **Core Updates:**
    - Overhauled Quality Yield to use variance-based probability modeling.
    - Integrated fatigue, overtime, and conveyor speed weighting.
    - Automated wage estimation via FCC Census Block API.
    - Fixed animation reset glitch on Layout and Schedule tabs.
    - Added interactive pop-out and tooltip features to Efficiency pie charts.

- Improved Location tab cost input interactivity and responsive scaling.

Evaluation

- **Realism:** The Quality Yield system now reflects stochastic and probabilistic characteristics of real manufacturing systems.
- **Performance:** Statistical approximations achieve realism without the computational cost of Monte Carlo simulation.
- **Consistency:** All text input and animation systems now behave uniformly across tabs.
- **Usability:** Hover effects, tooltips, and error fixes improve user comprehension and control precision.
- **Scalability:** Automated wage integration allows the Location tab to act as a bridge between geographic, operational, and economic models.

Session Outcomes

This session represents a critical turning point in the project's evolution—from deterministic simulation toward **probabilistic industrial modeling**.
The new Quality Yield system blends mathematics, data, and human factors into a unified operational model, while interface updates ensure the platform remains intuitive and responsive.
With this foundation, Factory Flow to Fortune now mirrors the complexity of real production environments: unpredictable, data-informed, and dynamically reactive to management decisions.

Modification of Quality Backend

Date: *November 11th, 2025 {Joel}*

Activity: Modified Quality behavior and reduced number of inputs. The specific behaviors of the new quality calculations are detailed in the backend tab on the overview page. Most of this was a backend change to how quality yields were calculated. As I have extensive Manufacturing Management experience, I am more familiar with the mechanics which may drive quality, and I made modifications to how the 4 stress factors were calculated, with the goal of removing linearity and better adjusting behaviors to a Coefficient of Variance (a scaled StdDev).

Technical Implementation: Quality Yield & Stress

Objective: Modify prior quality model to a probabilistic simulation model driven by Coefficient of Variation (CV).

- **Quality Logic (QualityYield.js):**
  - **New Driver:** Replaced multiple inputs with a single window.stDevPercentage (CV).
  - **Calculations:**
    - **Workstation Stress:** Calculated using Normal CDF (probability that variance exceeds Takt Time).
    - **Conveyor Fatigue:** Probability that process variation exceeds max safe speed.
  - **Location Stress:** Moved from simple ratios to a Logistic (Sigmoid) Function for overtime stress (1 / (1 + exp(-k...))).
- **Demand Logic Refactor:**

- **Circular Dependency:** Resolved the conflict between "Daily Demand" and "Yield." dailyDemandInput is now the strict Sales Target. The Auto-Adjuster calculates a *higher* Production Target to compensate for yield, rather than lowering expectations.
- **Wage Stress:** Integrated Async API calls to fetch Local Median Wage based on lat/long for stress calculations.

Quality Yield Integration within Tabs

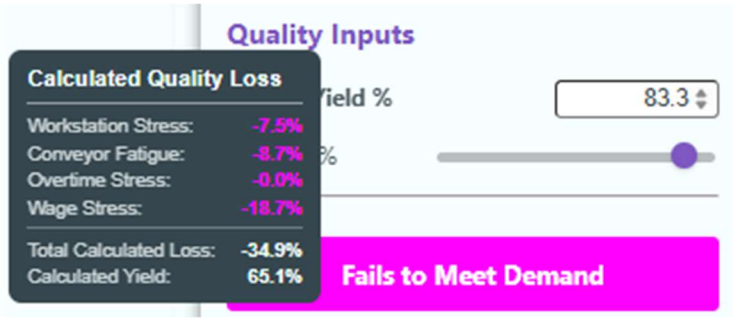
Date: *November 15th, 2025 {Joel}*

Activity: Integrated Quality Yield into other tabs to reflect on financials.

Technical Implementation: Financial Integration

Objective: Decouple Quality Yield from volume constraints and treat it as a financial driver (Cost of Poor Quality).

- **Operational Changes:**
  - **Targeting:** dailyDemandInput is treated as a hard "Total Units to Produce" limit. The system no longer iteratively increases production to compensate for yield in the physical simulation.
- **Financial Logic (COPQ):**
  - **Rework Costs:** Added inputs for superRework, ultraRework, etc.
  - **Formula:** Revenue - COGS - Labor Cost - COPQ = Gross Profit.
  - **Optimization:** The background profit calculator (findOptimalConfigForDemand) was updated to subtract COPQ when determining the "Optimal" configuration.
- **UI Updates:**
  - Updated tooltips to explain that Quality Yield now generates financial penalties rather than reducing volume capacity.
  - Changed the prior Quality Inputs to just a Quality Yield Percentage and a CV slider bar. The Quality Yield will auto-adjust, unless a value is directly typed in. Hovering over the Quality Yield Label will show the anticipated breakdown.



Modifying Tab, Sidebar, and Profit Templating

Date: *November 17th, 2025 {Joel}*

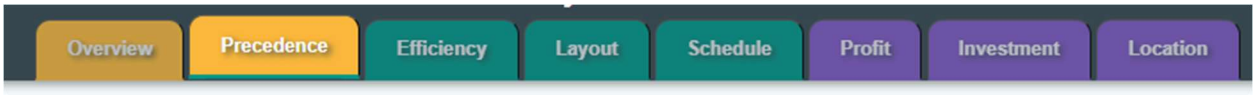
Activity: Compacted sidebar, adjusted Profit templating, and added custom force to precedence legend.

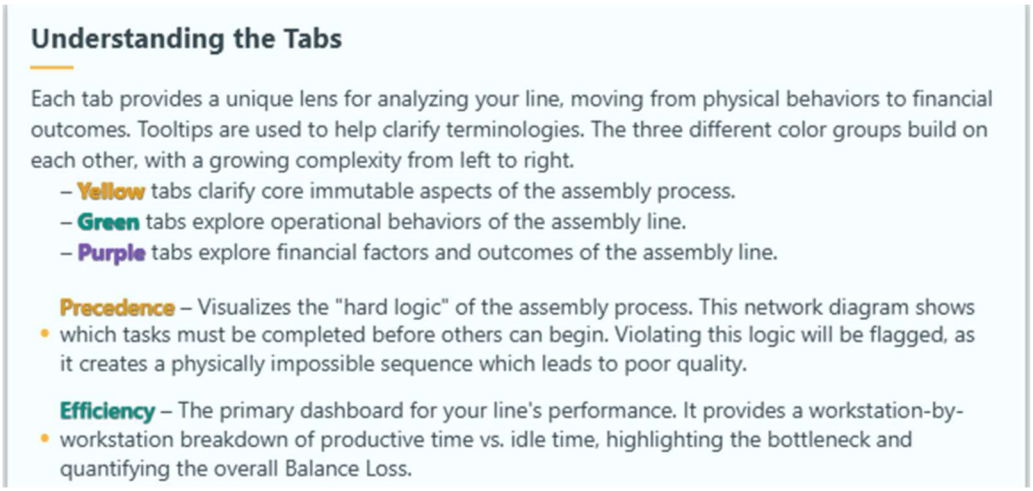
Technical Implementation: UI & Validation

- **Bug Fix (Precedence Validation):**

    - **Issue:** "Meets Demand" button wasn't updating on drag-and-drop.
    - **Fix:** Rewrote updateWorkstationOrder to update the global state based on the DOM state *first*, then trigger a full updateUI(). This ensures the newly rendered list is immediately validated and styled.

- **Custom Force (Precedence)**
    - Constructed a force for the DAG to repel the nodes away from the legends, to help minimize overlap.
- **Styling:**
    - Implemented "Folder Tab" styling with color coding (Primary vs. Secondary categories), updated the overview tab to explain the colors:



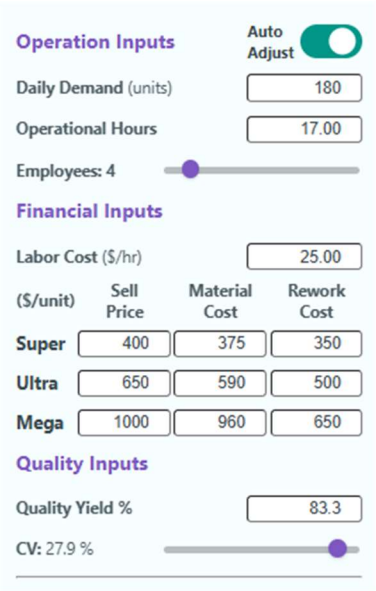


Understanding the Tabs

Each tab provides a unique lens for analyzing your line, moving from physical behaviors to financial outcomes. Tooltips are used to help clarify terminologies. The three different color groups build on each other, with a growing complexity from left to right.

– **Yellow** tabs clarify core immutable aspects of the assembly process.
– **Green** tabs explore operational behaviors of the assembly line.
– **Purple** tabs explore financial factors and outcomes of the assembly line.

- **Precedence** – Visualizes the "hard logic" of the assembly process. This network diagram shows which tasks must be completed before others can begin. Violating this logic will be flagged, as it creates a physically impossible sequence which leads to poor quality.
- **Efficiency** – The primary dashboard for your line's performance. It provides a workstation-by-workstation breakdown of productive time vs. idle time, highlighting the bottleneck and quantifying the overall Balance Loss.



Operation Inputs — Auto Adjust

Daily Demand (units)	180
Operational Hours	17.00
Employees: 4	

Financial Inputs

Labor Cost ($/hr)	25.00

($/unit)	Sell Price	Material Cost	Rework Cost
Super	400	375	350
Ultra	650	590	500
Mega	1000	960	650

Quality Inputs

Quality Yield %	83.3
CV: 27.9 %	

    - Compacted the sidebar layout, to allow more of the output variables to be visible when using the site.

- **Tooltips:**
    - Rewired right sidebar tooltips to attach to text labels rather than numeric values for better hit-testing.

Setting Dynamic Tooltip Positioning

Date: *November 19th, 2025 {Joel}*

Activity: Modified tooltip positioning for right-side of screen behaviors, and added tooltip notes to (Location/Investment). Added export functionality to the Location Tab, and did some bug-fixes of Investment.

Technical Implementation: UX Refinements

- **Smart Tooltips:** Implemented positionTooltip helper that detects screen edges and flips the tooltip to the left if it's in the rightmost quarter of the screen.
- **Profit Tab Layout:**
    - Split the chart area vertically (60% Pie / 40% Bar).

- Inserted a horizontal legend in the gap between charts.

- **Location Tab:**
  - Added "Export CSV" functionality to the simulation ribbon. Which outputs the build schedule as a spreadsheet.
- **Investment Tab:**
  - Refactored the draw function to use the D3 Update Pattern (transitions) rather than destroying/recreating the DOM, reducing flicker.
  - Fixed background update logic so NPV/IRR calculate even when the tab is closed.

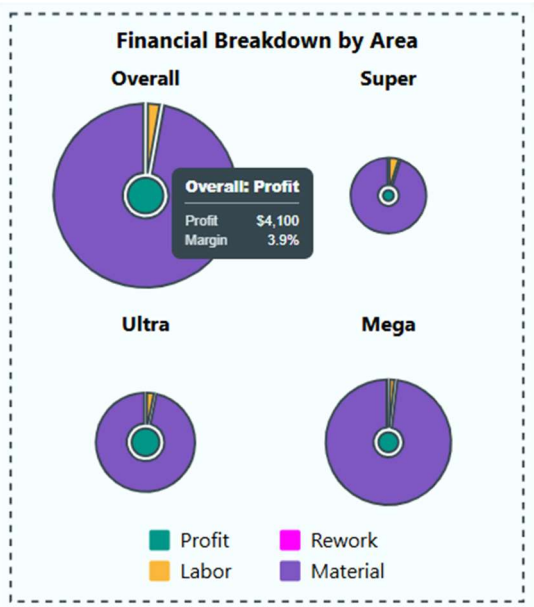
Modify Pie Charts to better Transition between Profit and Loss

Date: *November 20th, 2025 {Joel}*

Activity: Based on a conversation with the professor, I modified the behaviors of the pie-charts to instead resemble a sunburst chart (or more accurately a hierarchical set of donut charts). Tweaked the pie chart to be consistent between modes.

Technical Implementation: SVG & Chart Fixes

- **Pie Chart Dynamic Scaling:**
  - **Issue:** Vertical overlap of titles and legends.
  - **Fix:** Implemented strict height reservations. The maximum radius is now calculated as a function of the minimum cell width and the *remaining* vertical height after subtracting titles.
- **Tooltip Targets:**
  - Fixed an issue where the center "Gross Profit" tooltip was blocked by layering. Added an invisible circle (profit-tooltip-target) on top of the stack to handle mouse events exclusively.
- **SVG Markers:**
  - Fixed color conflicts in the Location tab arrows. Added .attr("fill", "currentColor") to the marker definition to force it to inherit the parent line's stroke color.
- **Profit and Loss Mode Unity:**
  - Changed the pie charts to become a 3-part nested donut chart system in which the gross profit forms the center circle, with the middle donut chart being the ratio of costs (collectively forming net revenue), and then an outer donut chart which shows unrecovered costs (when operating at a loss). This forms a smooth transition between profitable to non-profitable configurations without fundamentally changing what the chart means.



Embedding Screencast in Overview

Date: *November 24th, 2025 {Joel}*

Activity: Recorded screencast and embedded it into the overview page.

Recorded and embedded a screencast video into the overview tab.

Fixing Bugs Across the Entire Webpage

Date: *November 26th, 2025 {Dhruv}*

Overview and Motivation

This session focused on **final-stage debugging and polish**, addressing small but impactful issues across nearly every tab in the Factory Flow to Fortune application.
The goal was to ensure **visual consistency, animation synchronization, and configuration reliability** before final deployment.
Most of the changes involved correcting alignment, animation triggers, and user interaction logic — ensuring that every tab behaves predictably, looks polished, and aligns with the cohesive visual and interaction standards established throughout the project.

Related Work

This session builds upon prior stabilization efforts, particularly those involving sidebar configuration saving, animation timing, and responsive layout management.
While previous work focused on functional expansion and realism (such as the Quality Yield Model), this session finalized the **usability, consistency, and performance quality** of the overall experience.

Questions

- How can we eliminate redundant animation triggers without breaking existing transitions?
- What checks are necessary to prevent premature interactivity or inconsistent data states in configuration saving?
- How can cross-tab visual standards (legends, text styles, animation speeds) be unified without introducing regressions?

Design Evolution

1. Overall Animation Synchronization

Problem:
Both the **Layout** and **Schedule** tabs were experiencing an issue where animations were being triggered twice. This led to jittery visuals, duplicate transitions, and unnecessary processing overhead.

Solution:

- Debugged the main rendering loop to identify duplicate animation calls caused by overlapping event triggers.
- Consolidated start functions and ensured that **each animation sequence initializes only once per user action**.
- Added safeguards to prevent redundant requestAnimationFrame calls during quick sidebar interactions or tab switches.

Outcome:
Animations across both Layout and Schedule tabs now start cleanly, run smoothly, and maintain proper synchronization, eliminating flicker and performance artifacts.

2. Schedule Tab Refinements

Problems:

- The **legend** was misaligned and appeared slightly off relative to the visualization area.

- The **cycle time label** had inconsistent text formatting compared to the rest of the site.

Solutions:

- Repositioned the legend elements using precise translation offsets and alignment logic to ensure consistent placement across different resolutions.
- Updated cycle time text styling — matching the site-wide typographic standards for font size, weight, and color variables.

Outcome:
The Schedule visualization now aligns perfectly with the design system, maintaining clear, consistent labeling and balanced composition.

3. Right Sidebar Finalization

Problems:

- **Ctrl+Click reset** was reverting to incorrect baseline values on certain parameters.
- The **"Save and Compare"** feature lacked contextual guidance and proper interactivity handling.
- The **"Compare Configuration"** button was clickable even before saving a configuration, leading to undefined comparisons.
- Saved configurations were not properly restoring parameters or triggering corresponding animations, resulting in inconsistent states.

Solutions:

- Fixed the **reset-to-default logic**, ensuring all parameters correctly reference the master default dataset.
- Added a **tooltip** for the "Save and Compare" section to clearly explain its purpose and workflow.
- Updated the **Compare button logic** so that it only becomes active **after a configuration is saved**.
- Refactored the save/load system to ensure parameters persist correctly across sessions and animations update dynamically upon configuration load.

Outcome:
The Save and Compare feature now behaves intuitively and reliably. Configurations persist correctly, controls are context-aware, and tooltips help users understand comparison workflows without confusion.

4. Location Tab Enhancements

Problems:

- The **arrow animation** started prematurely (before both location markers were placed).
- Animations were being interrupted or cut short when other animations (such as map updates) concluded.
- There was no clear visual legend explaining the meaning of map elements, leaving users unsure of how to interpret the visualization.

Solutions:

- Adjusted animation triggers to **delay arrow animation until both markers are placed**, ensuring logical sequencing.
- Introduced event handling isolation so that arrow animations **run independently** and are no longer interrupted by unrelated transitions.
- Added a **legend** clarifying that the **star represents the optimal factory location** and that **arrow width corresponds to annual cost**.

Outcome:
The Location tab now offers a fully self-contained, polished visualization experience — with clearly defined visual semantics and reliable, uninterrupted animations.

5. Efficiency Tab Adjustments

Problem:
The **idle clock text** was static and not updating during animation transitions, causing a visual disconnect between the clocks and their numerical labels.

Solution:

- Implemented proper animation binding for the idle clock text to dynamically update in real time as the clock transitions.

Outcome:
The Efficiency tab now maintains smooth synchronization between visual clock motion and textual feedback, improving coherence and user comprehension.

6. Profit Tab Refinements

Problems:

- The Profit tab lacked animation consistency compared to other visual tabs, resulting in static or delayed transitions.
- The pie chart resizing did not occur simultaneously with page resizing, leading to temporary misalignment.
- A glitch caused **multiple chart titles** to appear during dynamic resizing events.

Solutions:

- Added **animations for chart elements** including axes transitions, color fades, and bar chart element movement — unifying the visual rhythm with other tabs.
- Implemented simultaneous **responsive resizing logic** for pie charts and axes to ensure proportional scaling.
- Fixed the title duplication glitch by consolidating the title creation call and preventing redundant re-renders on window resize events.

Outcome:
The Profit tab now visually aligns with the rest of the platform, featuring smooth transitions, consistent chart behavior, and clean responsiveness without rendering artifacts.

Implementation

- **Files Modified:** script.js, style.css, and tab-specific visualization modules for Schedule, Location, Efficiency, and Profit.
- **Core Updates:**
    - Unified animation timing to eliminate redundant triggers.
    - Fixed right sidebar configuration save/load logic and tooltip explanations.
    - Corrected text alignment and legend formatting in Schedule tab.
    - Added legends and independent animation logic to Location tab.
    - Synced clock text animation with Efficiency visualization.
    - Standardized Profit tab chart animations and responsive behavior.

Evaluation

- **Performance:** Eliminated redundant animation starts, improving performance and visual smoothness.

- **Usability:** Clearer tooltips, improved configuration management, and accurate button interactivity enhance user experience.
- **Visual Consistency:** Unified text formatting, legend alignment, and animation timing across all tabs.
- **Stability:** All known rendering glitches, alignment errors, and animation interruptions were resolved.
- **Professional Polish:** The application now presents a cohesive, refined aesthetic across all visual and analytical modules.

Session Outcomes

This session represents the **final stabilization and refinement phase** of the Factory Flow to Fortune application.
By resolving animation redundancies, tightening alignment and text consistency, and finalizing interactivity logic across the sidebar and visualizations, the system has reached a level of polish suitable for final submission and demonstration.
Every tab now delivers a **smooth, synchronized, and reliable experience**, showcasing the project's evolution from concept to a professional-grade, data-driven visualization platform.

Modified Screencast and Additional Bug Fixes

Date: *November 29th, 2025 {Joel}*

Activity: Implemented a series of bug-fixes across the Location and Profit Tabs, and then redid the screencast video to be less about how the tabs worked, but rather to be why our website should be used.

Bugfix Technical Implementations:

- **Location Tab** (LocationTab.js) – There were several CSS rendering issues which would occur after initial load and between switching tabs.
    - **Arrowhead Visibility**: Fixed an issue where the connection line arrowheads appeared dark/invisible when changing back and forth between tabs by switching from using .attr("fill") to .style("fill") to ensure the correct yellow color took precedence over global CSS rules.
    - **Animation Continuity:** Fixed an issue where existing lines would "shrink and regrow" upon updates, by adding a proximity check; if a line is already at its target destination, the animation reset is skipped. This prevents the double draw on initial factory allocation.
    - **Legend Layout:** Refactored the legend to place the "Low Cost" and "High Cost" line indicators side-by-side rather than stacked vertically. Added additional clarifying descriptions to the meaning of gaps in the 'marching ants' animation and the size of city nodes.
    - **Redraw Artifacts:** Fixed a bug where leaving and returning to the tab caused the legend arrows to appear thick or discolored. This was caused by the legend grouping not being cleared on redraw; we added an explicit .remove() call.

- **Profit Tab** (ProfitTab.js) - The prior changes made by Dhruv had modified the behavior of the "Sunburst" chart in a way which didn't accurately visualize financial losses.
    - **Outer Ring Logic:** Modified the logic for the "Unrecovered Costs" outer ring. It is now only calculated and drawn if Profit is negative. Previously it was showing as a constant outer band, showing unrecovered losses when the production was profitable—which doesn't make sense.
    - **Ring Scaling:** Adjusted the radius calculation so the outer ring scales based on Total Expenses, making the visible area proportional to the magnitude of the loss. The middle ring is sized to total revenue, which makes the unrecovered losses show a proportional area to revenue in a similar way that profit is shown to revenue. This had been changed to a simple outline, which misrepresented true negative profit margins.

- **Console Errors:** when quality variables were changed, while visually nothing occurred, a console reference error would occur. This was due to an invalid line which was trying to attach tooltips in the main script file, outside an event-listener, which was removed.
- **Dynamic Updates:** Fixed a D3 lifecycle bug where changing the Labor Cost input didn't update the outer ring's color or visibility. This would only be visible after switching tabs and changing multiple variables. This was solved by moving the style application from the .enter() block to the .merge() block to ensure existing elements update correctly.

- **Screencast Modification –** upon discussion within our team, and with the professor, it was determined that it wasn't feasible to detail the full functionality of all our tabs, and that the screencast should instead be used to explain the overall utility and why our visualization is useful. This was structured as an "elevator speech" in which the impact of poor balancing was explained and why the problem space mattered, and how our website provided utility which would help resolve these. I then explained the 3-tab groups by color, and how they would be used by Engineers, Managers, and Executives. I then concluded by explaining that our website was more than just a calculator, but rather a complex intersection of multiple relevant fields in manufacturing, and how impactful it would be.

Embed Documentation

Date: *November 30th, 2025 {Joel}*

Activity: Reviewed the Process Document and organized entries to be chronological, printed it out to be located as a pdf -> /Pages/ProcessBook.pdf. Created a Container at the end of the overview tab to fit the Backend Write-Up, our ReadMe file, and our Process Book.

Created a README.md file, and a readme_viewer.html to read in that file into a readable document for the "Code Architecture ReadMe" button to reference.